

Detailed explanation of the source code of the LwIP protocol stack

- Implementation of the TCP/IP protocol



Created by.. 老衲五木

at.. UESTC

Contact me.. for_rest@foxmail.com

540535649@qq.com



preface

A recent project used LwIP, and I happened to see a lot of people discussing it online, so I had the urge to write this study note, one is to pass the time in a daze, and the other is to blow the NB. Often deciding to do something is simple, and insisting on finishing it is a long and tortuous one, but after all, it is still written, the time cost is about four months, and the memory cost cannot be estimated.

This article covers most of the LwIP protocol, but it is not comprehensive. It mainly explains the most important and commonly used parts of the LwIP protocol, including memory management, underlying network interface management, ARP layer, IP layer, TCP layer, API layer, etc., which are often involved in typical LwIP applications. Other parts of the LwIP protocol, including UDP, DHCP, DNS, IGMP, SNMP, PPP, and other parts that do not have common use, are not covered in this document for the time being.

It turned out that the article was posted in space, updated in turn in each section, and later changed to a blog, and then it was simply too lazy to post. Now it's finally done, so I've summarized all the articles. I racked my brains to write an unprecedented and loved preface, but the more I wrote, the more I felt like a cat scratching it. In this way, PS: Because my writing is limited, my emotional intelligence is low, and I am a low-level person, so many sentences in the article may make you very entangled, you can contact me through email. Joint discussion is the key to progress.

Finally, readers are welcome to use and reprint in any way, but please keep the author's relevant information, sauce purple! Code words... There is nothing more painful in the world than this...

——Lao Na Wumu

directory

1	Transplant Review	
-- 42	Dynamic memory management	
----- 63	packet pbuf	
----- 94	pbuf Release	
135	Network interface structure	
----- 166	Ethernet data reception	
----- 207	ARP table	
----- 238	ARP table query	
-269	ARP Layer Process	
281	IP Layer Input	
-311	IP Shard Reinstall 1	
-341	IP Shard Reinstall 2	
371	ICMP handles	-4014TCP
establish and disconnect		
-431	TCP State Transition	
461	TCP Control Block	
491	TCP Establish Flow	
-531	TCP State Machine	
561	TCP Input/Output Functions 1	
-602	TCP Input/Output Functions 2	
632	TCP Sliding Window	
-662	TCP Timeout and Retransmission	
----- 692	TCP Slow Start & Congestion Avoidance	
----- 732	TCP Fast Recovery Retransmission and Nagle Algorithm	
----- 762	TCP Persistence and Keep Alives Timer	
----- 802	TCP Timer	
----- 842	TCP Termination and Summary	
----- 882	API Implementation and Related Data Structures	
----- 912	API Message Mechanism	
----- 943	API Functions and Programming Examples	
----- 97		

1 移植综述移植综述移植综述移植综述

If you think that the so-called perseverance is a "hard and patient" struggle every minute and every second, then this is a very insufficient psychological state. Perseverance is a habit, perseverance is a state, perseverance is a life. After reading the code for so long, I feel that it is time to write something, otherwise how can I be worthy of the glorious title of scientific researcher that someone says. When I first saw this mountain-like code, I really saw a cold sweat. Now that I think about it, it's actually not that difficult, how can so many revolutionary ancestors come up with Dongdong after N long hours make it possible for a hairy boy to understand it within a few weeks. All I saw was a small corner of a glacier, a small step of the Long March, and a small hair on the body of nine cows... In someone's words, write, write in a daze, write blindly, write nonsense, scribble, write and write, and you will understand.

I think I am very suitable to be a singer, youth is floating in the wind. You know, even if the rain turns the city upside down, I will give you a hug; I can't stand it, I see your back coming, and write down my unbearable sorrow; Even if the whole world is tied up by loneliness, I will not run; I can't escape, and in the end everyone will be old, write about me, the castle where time and the sound of the piano are intertwined. The song I'm listening to. Far from it...

To get to the point, embedded products are connected to the Internet network, and this MS is an intensifying trend. Think about how great it would be to be able to configure your products without leaving home and get the data information you care about. This is perhaps the most basic prototype of the Internet of Things world. Of course, it is not easy for your product to have such functionality, at least it must have a TCP/IP protocol stack that is currently very fashionable. LWIP is an open-source TCP/IP protocol stack for embedded systems. In cases where your embedded processor is not very NB, and the internal Flash and RAM are not very powerful, it is still very suitable to use.

The designers of LWIP have provided detailed porting documentation for lazy people like me, but that's not enough, they have done most of the work as much as possible, and lazy people only need to do very little work to complete the merit. Throughout the porting process, users need to complete the following things:

The first is the definition of the data types used internally in the LWIP protocol, such as `u8_t`, `s8_t`, `u16_t`, `u32_t`, etc. These data types must be redefined due to different processors on the ported platform and different compilers used. Think of an int-type data that is 8 bytes long on a 64-bit processor, 4 bytes on a 32-bit processor, and only two bytes on a 16-bit processor. Therefore, this part needs to be written according to the number of processor bits and the characteristics of the compiler used. So the typedef unsigned int `u32_t` porting statement used on ARM7 processors will definitely not work during the porting process of 64-bit processors.

The second is to implement functions related to semaphore and mailbox operations, such as establishing, deleting, waiting, and releasing them. If you run LWIP directly on bare metal, this is more troublesome to implement, and users must establish a set of semaphore and mailbox-related mechanisms by themselves. Normally, embedded systems using LWIP are supported by the operating system, and semaphores and mailboxes are often the most basic process communication mechanisms in the operating system. UC/OSII should be regarded as the simplest embedded operating system, and it can also provide semaphores and mailbox mechanisms without exception, as long as we encapsulate the relevant functions in UC/OSII accordingly, it can meet the needs of LWIP. LWIP uses mailboxes and semaphores to enable information interaction between upper-layer applications and protocol stacks, and between lower-layer hardware drivers and protocol stacks. The LWIP protocol simulates the layering idea of the TCP/IP protocol, and on the surface, LWIP also has a layered idea, but from the implementation point of view, LWIP only implements all the work at all levels in one process. The details are as follows: After the LWIP completes the relevant initialization, it will be blocked on a mailbox waiting for the received data to be processed. The data in this mailbox may come from packets received by the underlying hardware driver, or from the application. When the data is obtained in the mailbox, LWIP will parse the data, and then call the relevant processing functions on the upper layer of the stack to process the data. After processing is over, LWIP continues to block on the mailbox waiting for the next batch of data. Of course, LWIP also has a large set of memory management mechanisms to avoid a lot of time and memory overhead when interacting data between layers, which will be explained in a later discussion. Of course, this design makes it more difficult to understand the code, which is a big headache. It is a semaphore

It can be used to communicate with each other between the application and the protocol stack. For example, when an application is about to send data, it first sends the data to a LWIP blocked mailbox, and then it hangs on a semaphore. After LWIP takes the data from the mailbox and processes it, it releases a semaphore and tells the application that I have completed the data you want to send; After that, the application gets the semaphore and continues to run, while the LWIP continues to block on the mailbox waiting for the next batch of data to be processed.

Secondly, there are functions related to waiting timeouts. As mentioned above, the LWIP protocol stack will be blocked on the mailbox waiting for the arrival of receiving data. This kind of waiting seems to be going on all the time on the outside, but it is not. Generally, when initializing the LWIP process, some timeout events will be initialized at the same time, that is, when some events wait for the timeout, they will automatically call some timeout processing functions to do related processing to meet the requirements of the TCP/IP protocol stack. In this way, when the LWIP stack blocks the waiting mailbox, it will shrewdly calculate how long it should wait, if there are no timeout events initialized in the LWIP process, then the situation is the simplest, just hang the process forever, and the wait at this time can be regarded as eternal....It's a bit ambiguous. If there is an initialized timeout event in the LWIP process, you can't wait forever, because then the timeout event has no chance of being executed. LWIP does this, the waiting time for the mailbox is set to the length of time for the first timeout event, if the time is up, and the data has not yet been waited, then it jumps out of the mailbox and waits for the timeout event to be executed, and then continues to block the mailbox according to the above method after the execution of the timeout event is completed. It can be seen that for an LWIP process, a linked list is required to manage these timeout events. Most of the work on this linked list has been done by the designers of LWIP, and the user only needs to implement one function: this function returns the first address of the current process's timeout event linked list. The LWIP internal protocol uses this first address to find the completion timeout event.

Secondly, if LWIP is built on a multithreaded operating system, it is necessary to implement a function to create a new thread. Operating system that does not support multi-threading, sweat... said he hadn't heard of it yet. However, UC/OSII obviously supports multithreading, and everyone on Earth knows it. A typical LWIP application system consists of three processes: the upper-level application process is started first, then the LWIP stack process, and finally the underlying hardware packet receive and send process. Typically LWIP stack processes are created by calling LWIP stack initialization functions in an application. Note that LWIP stack processes generally have the highest priority in order to respond correctly to data in real time.

Secondly, some other details. For example, the critical zone protection function is used when the LWIP protocol stack handles certain critical zones, which is generally implemented by turning off interrupts in the critical zone and opening interrupts out of the critical zone. For example, the structure encapsulation macro used when defining the structure, the implementation of LWIP is based on such a mechanism, that is, the upper layer protocol has clearly known the structural characteristics of the data transmitted by the lower layer, and the upper layer directly uses the relevant address to calculate the desired data, avoiding copying and buffering when the data is submitted, so it is necessary to define the structure encapsulation macro and prohibit the compiler's address from being automatically aligned; There are also macros such as debugging output and measurement records.

Finally, it is also a more important place. Implementation of the underlying network-driven function. It depends on the network interface chip used by your embedded hardware system, that is, the network card chip, the common ones are RTL8201BL, ENC28J60, etc. Different interface chip manufacturers will provide rich driver functions. As long as we encapsulate these transmit and receive interface functions accordingly, we can encapsulate the received data packets into a familiar data structure of the LWIP protocol stack, and decompose the sent data packets into familiar data structures on the chip. At a minimum, sending a packet function and receiving a packet function need to be implemented.

That's it, although it was written hastily, but finally got it done before retreating. A good start is half the battle, so let's count it as a quarter for now. I don't know if I can finish writing in a month, two months, or more. To predict what will happen next, please see the next breakdown.

2. Dynamic memory management, dynamic memory management, dynamic memory management, dynamic memory management

Recently, the power bureau is very incompetent, and the power outages every now and then every now and then, causing us to always be painful double-buckle landlords, don't bring this! Still writing today? Write, must.

Yesterday, I talked about the LWIP porting framework, and there is a large basket of documents on the Internet about the details of the port. Interested children's shoes may wish to look for them. Here, I'm curious to delve into the details of LWIP's internal protocol implementation, as well as the ins and outs of all the entangled issues. Future discussions and studies will be organized according to the structure of the LWIP English document, "Design and Implementation of the LWIP: TCP/IP Stack".

The dynamic memory management mechanism of LWIP is discussed here.

In general, LWIP can have three dynamic memory management mechanisms: the memory allocation policy that comes with the C runtime library, the dynamic memory heap (HEAP) allocation policy, and the dynamic memory pool (POOL) allocation policy.

The dynamic memory heap allocation policy is very similar to the memory allocation policy that comes with the C runtime library, which is implemented by LWIP simulation runtime library allocation policy. The consumer of these two policies can only choose one of them, which is achieved by the macro definition `MEM_LIBC_MALLOC` in the header file `lwippools.h`, which uses the memory allocation policy that comes with the standard C runtime library when it is defined as 1, and LWIP its own dynamic memory heap allocation policy when it is 0. In general, we choose to use LWIP's own dynamic memory heap allocation policy, and we will not discuss the memory allocation policy that comes with the C runtime library here.

At the same time, the dynamic memory heap allocation policy can be implemented in two ways, and the first is to open up a memory heap and then implement it by simulating the memory allocation policy of the C runtime library. The second is to implement it through dynamic mempools, that is, the dynamic memory heap allocation function completes its function by simply calling the dynamic memory pool (POOL) allocation function (too perfunctory), in this case, the user needs to define the macro `MEM_USE_POOLS` and `MEM_USE_CUSTOM_POOLS` as 1 in the header file `lwippools.h`, and also open up some additional buffer pool areas, as follows:

```
LWIP_MALLOC_MEMPOOL_STARTLWIP_MALLOC_MEMPOOL(20,  
256)LWIP_MALLOC_MEMPOOL(10,  
512)LWIP_MALLOC_MEMPOOL(5, 1512)
```

`LWIP_MALLOC_MEMPOOL_END` these sentences are taken from the LWIP source code comments, which indicate that 20 256-byte memory blocks, 10 512-byte memory blocks, and 5 1512-byte memory blocks are allocated to dynamic memory heap-related function functions. Memory pool management automatically defines a large amount of memory in memory statically based on the above macros for the memory pool. If macro `MEM_USE_POOLS_TRY_BIGGER_POOL` is enabled, then if there is no space in the pool with the best length mentioned above, the allocator will apply from the larger length pool, but this will waste more memory. Dizzy.... that's it, this method is generally not used. Hey, just this last sentence is powerful.

The first implementation of the dynamic memory heap allocation strategy is discussed below, which is also commonly used. This part of the discussion is mainly based on the online Oldtom's Blog, which is well written (but also has a small mistake), so I accidentally borrowed it.

The principle of dynamic memory heap allocation strategy is to manage in a memory block of a predefined size, and its memory allocation strategy is to adopt the first fit method, as long as it finds a free block larger than the requested memory, it will cut out the appropriate block from it, and return the remaining part to the dynamic memory heap. There is a minimum size limit for the allocated memory blocks

The allocation size of the request must not be less than MIN_SIZE, otherwise the request will be allocated to MIN_SIZE size of memory space. Generally MIN_SIZE is 12 bytes, and the first few bytes of these 12 bytes will store private data for memory allocator management, which cannot be modified by user programs, otherwise it will cause fatal problems. The process of freeing up memory is the opposite, but the allocator looks at whether adjacent memory blocks before and after that node are free, and if so, merges into one large free memory block. The advantage of adopting this allocation strategy is that the memory waste is small, relatively simple, and suitable for the management of small memory, but its disadvantage is that if frequent dynamic allocation and release, it may cause serious memory fragmentation, and if the fragmentation situation is serious, it may lead to unsuccessful memory allocation. For the use of dynamic memory, the recommended method is allocation-> release

-> allocation -> release, this method of use is able to reduce memory fragmentation. Let's take a look at how LWIP implements these functions. `mem_init()` The initialization function of the memory heap mainly informs the start and end address of the memory heap and the initialization idle list, which is called by LWIP when initializing. `mem_malloc()` Request to allocate memory. Passing the total number of bytes required as a parameter to the function, the return value is a pointer to the latest allocated memory, while if the memory is not allocated, the return value is NULL, and the allocated space size is affected by memory alignment and may be slightly larger than the requested one. The returned memory is initialized "0". This memory may contain any random junk, and you can immediately initialize it with valid data, or at least zeros. The allocation and release of memory cannot be carried out in the interrupt function. If there are multiple threads requesting and releasing memory at the same time, it may take a long time to apply due to the waiting of semaphores. `mem_calloc()` is a simple wrapper of the `mem_malloc()` function, which has two parameters, namely the number of elements and the size of each element, and the product of these two parameters is the size of the memory space to be allocated, unlike `mem_malloc()` that it will clear the dynamically allocated memory to zero. Experienced programmers prefer to use `mem_calloc()` because then there will be no problem with the contents of the newly allocated memory, calling `mem_calloc()` will definitely clear 0s, and calling `memset()` can be avoided. Rest..... The dynamic memory pool (POOL) allocation strategy can be said to be a relatively stupid allocation strategy, but its allocation strategy is simple to implement, and the allocation and release of memory are efficient, which can effectively prevent the generation of memory fragmentation. However, its disadvantage is that it wastes some of the memory. Why is it called POOL? This is interesting, there are many types of POOLS, and this depends on how the user configures the LWIP. For example, if the user defines `LWIP_UDP` as 1 in the header file `opt.h` file, the UDP type mempool will be established at compile time; If `LWIP_TCP` is defined as 1, the TCP type mempool will be established at compile time. There are also many other types of mempools, such as `PBUF_POOL` that store network packet data, `CUSTOM_POOLS` mentioned in the dynamic memory heap allocation strategy above, and so on. The individual size of a certain type of POOL is fixed, and the number of POOLS allocated to this type of POOL is user-configurable, and users should configure it according to the actual usage of the stack. By putting all the POOLS in the stack together and placing them in a contiguous memory area, this presents a large buffer pool to the user. Therefore, the internal organization of the so-called buffer pool should be like this: at the beginning, a type A POOL pool is placed, followed by type B POOL b, and then a type C POOL pool c... until finally n POOL pools of type N. This is very similar to the process control block and event control block in UC/OSII, where you can open up a bunch of various types of control blocks first, and you just need to use them directly. Note that the allocation here must be based on a single buffer pool, which can lead to memory waste. This is obvious, no explanation. Let's take a look at how the LWIP implementation opens up the large buffer pool discussed above (the word 'of' has made a bunch of people laugh for a long time today). Basically, most people will be dizzy when they see this part of the code, and they don't know what the author is doing at all, but after careful understanding, you have to admire the author's otherworldly code writing ability, and almost used the word sinking fish and falling geese. On the code:

```
static u8_t memp_memorystatic u8_t memp_memorystatic u8_t memp_memorystatic u8_t memp_memory [[[[
MEM_ALIGNMENTMEM_ALIGNMENTMEM_ALIGNMENTMEM_ALIGNMENT ---- 1111#define LWIP_MEMPOOL(name,num,size,desc) +#define
LWIP_MEMPOOL(name,num,size,desc) +#define LWIP_MEMPOOL(name,num,size,desc) +#define LWIP_MEMPOOL(name,num,size,desc) + (
.....) * (MEMP_SIZE + MEMP_ALIGN_SIZE(size)) \\\(.....) * (MEMP_SIZE + MEMP_ALIGN_SIZE(size)) \\\(.....) * (MEMP_SIZE +
)]; The above code defines the memory buffer used by the buffer pool, and many people will
definitely wonder if this is the definition of an array. Define an array with the define and
include keywords. The key to solving the problem is the header file memp_std.h, which can be
reduced to many LWIP_MEMPOOL (name, num, size, desc). And since the define keyword is used to
define LWIP_MEMPOOL (name, num, size, desc) as +((num) * (MEMP_SIZE + MEMP_ALIGN_SIZE(size))), the
memp_std.h is compiled into +(), +(), +(), +(), +()....so the final array memp_memory
equivalent is defined as: static u8_t memp_memory [ MEM_ALIGNMENT - 1
```

+ () + () ...];

If you don't understand it at the moment, it can only mean that there is a problem with my ability to express myself. Of course, there is a small legacy question, why should the array be larger than the actual need MEM_ALIGNMENT - 1? The author is considering the word alignment problem of the compiler, so stop here, this problem cannot be studied in depth, and I will talk about it slowly later.

Copying the method of array creation above, the stack also establishes some global variables related to buffer pool management: memp_num: This static array is used to store the number of members of various types of buffer pools memp_sizes: This static array is used to store the structure size of various types of buffer pools memp_tab: This pointer array is used to point to the currently idle nodes of various types of buffer pools

The next step is the implementation function: memp_init(): The initialization of the mempool is mainly to create a linked list for each mempool, memp_tab and the linked list is reversed, in addition, if the statistical function is enabled, the number of various mempools is also recorded. memp_malloc(): If the corresponding memp_tab linked list has free nodes, cut a node from it and return, otherwise it returns empty. memp_free(): Add the released node to the corresponding linked list memp_tab header.

As you can see from the above three functions, the dynamic mempool allocation process is quite simple and intuitive. HC: Baidu means nonsense. Ha ha.....

3 Packet Packet Packet Packet PBUF

High places, it's always cold. Loneliness can make people crazy. No one can understand you! Yesterday we talked about the memory allocation mechanism of LWIP. In summary, there are two commonly used memory allocation strategies in LWIP, one is memory heap allocation, and the other is memory pool allocation. The former can be said to be able to allocate the memory blocks of reasonable size we need (again, 'the') as we want, but the disadvantage is that when after multiple allocation releases, there will be many fragments in the middle of the memory heap, making the allocation fail when larger memory blocks need to be allocated; The latter is fast and simple linked list operations, because various types of POOLS are established in advance, but using POOLS will sometimes waste a certain amount of memory space. In LWIP, the two allocation strategies are mixed to achieve good memory efficiency.

Let's take a look at how these two allocation strategies are used in LWIPs. This brings us to the topic of discussion in this section: the implementation of packet buffering for LWIPs.

The packets that move through the stack are undoubtedly the most important part of the overall memory management. The types and sizes of data packets can also be said to be varied, counting, first of all, the original packets from the network card, it can be TCP packets up to thousands of bytes, or ICMP packets with only a few bytes; In terms of the packets to be sent, the upper-layer application may submit the packets it wants to send in various shapes to the LWIP stack, which may exist in the memory space managed by the application or on a ROM. Note that the core thing here is that LWIP strongly prohibits copying data when it is passed between layers, as it consumes a lot of time and memory. In summary, LWIP must have an efficient packet management core that can accommodate all types of data and avoid the huge overhead of copying data between layers.

The packet management authority adopts the data structure pbuf to describe the packet, the source code of which is as follows:

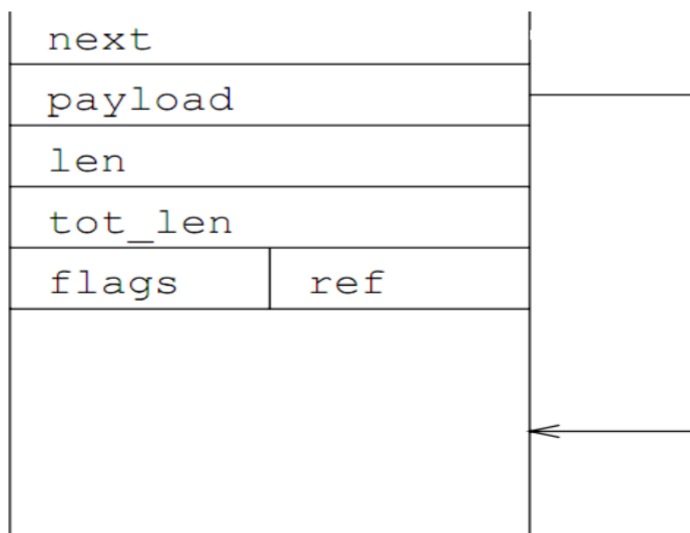
```
struct pbuf {  
  
    struct pbuf *next;  
  
    void *payload;  
  
    u16_t tot_len;  
  
    u16_t linen;  
  
    u8_t  type;  
  
    u8_t flags;  
  
    u16_t ref;  
  
};
```

This seemingly simple data structure is enough for me to take a break! The next field pointer points to the next PBUF structure, because the packets actually sent or received can be large and each PBUF may be able to manage very little, so it often takes multiple PBUF structures to fully describe a packet. So, all the pbuf structures that describe the same packet

It needs to be chained to a linked list, which is implemented with next. The payload is a data pointer to the starting address of the data managed by the pbuf, where the starting address of the data can be the RAM immediately following the pbuf structure, or it may be an address on the ROM, and what determines this is what type of pbuf is the value of the type field, which will be discussed below. The len field represents the length of valid data in the current PBUF, while tot_len represents the length of valid data for the current PBUF and all subsequent PBUs. Obviously, the tot_len field is the sum of the len field and the tot_len field of a subsequent PBUF in the PBUF chain; The tot_len field of the first PBUF in the PBUF chain represents the length of the entire packet, while the tot_len field of the last PBUF must be equal to the LEN field. The type field represents the type of PBUF, there are four main types, which basically involves the most difficult part of PBUF management, which will be discussed in detail in the next section. The documentation says that the flags field also represents the type of pbuf, I don't understand, doesn't the type field explain the type of pbuf? However, in the source code, when initializing a PBUF, the value of the field is set to 0, and the field is not used anywhere else, so it is simply ignored here. The last ref field indicates the number of times that pbuf has been referenced. This is another entangled place. When initializing a PBUF, the ref field value is set to 1, and when the next pointer of another PBUF points to the PBUF, the ref field value of the PBUF is added by one. Therefore, to delete a pbuf, the value of the ref must be 1 for the deletion to succeed, otherwise the deletion will fail.

The type of PBUF, something dizzying. There are four categories of PBUFs: PBUF_RAM, PBUF_ROM, PBUF_REF, and PBUF_POOL. Below, let's take a look at the characteristics of each type one by one.

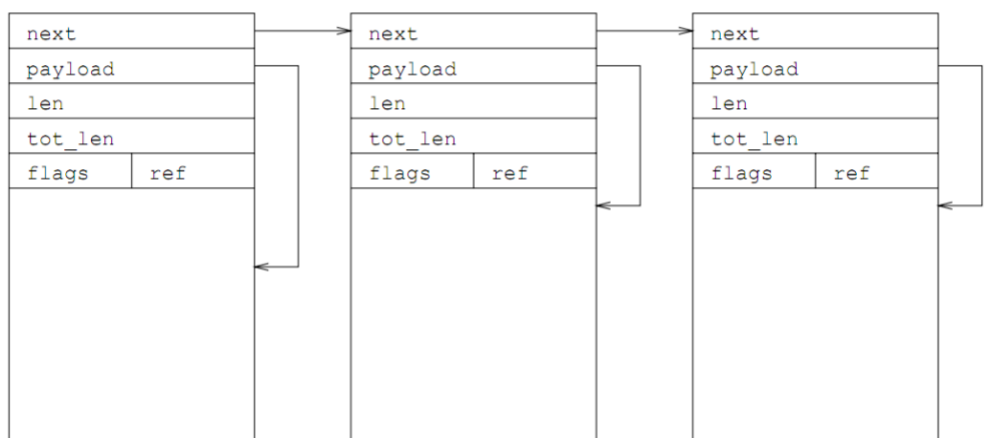
PBUF_RAM type PBUFs are mainly allocated through the memory heap. This type of PBUF is the most used in the protocol stack. The data that the stack wants to send and the data that the application wants to pass are generally in this form. When applying for the PBUF_RAM type, the stack allocates the corresponding size in the memory heap, note that the size here includes the pbuf structure header size and the corresponding data buffer as mentioned earlier, which are in a contiguous memory area. Let's take a look at how the source code applies for type PBUF_RAM. where p is a PBUF-type pointer. `p = (struct pbuf*)mem_malloc(LWIP_MEM_ALIGN_SIZE(SIZEOF_STRUCT_PBUF + offset) + LWIP_MEM_ALIGN_SIZE(length));` It can be seen that the system calls the memory heap allocation function mem_malloc for memory allocation. The size of the allocated space includes: the size of the PBUF structure header SIZEOF_STRUCT_PBUF, the size of the required data storage space length, and an offset. There's a lot to discuss about this offset, but that's where it starts. In summary, the PBUFs of the PBUF_RAM types that are successfully assigned are as follows:



From the figure, it can be seen that the PBUF header and the corresponding data are in the same continuous memory area, and note that the payload does not point to the end of the PBUF header, that is, after the ref field, but separates a certain area. This area is the size of the offset above, which is used to store data headers, such as TCP headers, IP headers, etc. Of course, offset can also be 0, what the specific value is, it depends on how you apply for it. If you want to dig deeper, you will definitely be even more dizzy.

PBUF_POOL type and PBUF_RAM type PBUF have a lot of similarities, but they are mainly allocated through mempools. This type of PBUF can be allocated in a very short time. LWIPs typically wrap data in this way when accepting packets. When applying for the PBUF_POOL type, the stack allocates the appropriate number of mempools in the mempool to meet the required application size. Let's take a look at how the source code applies for type PBUF_POOL. where p is a PBUF-type pointer.

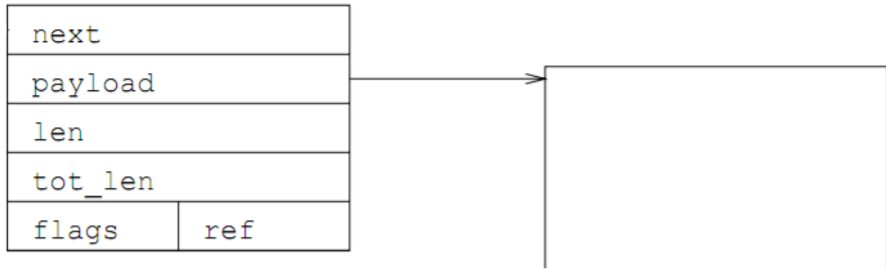
`p = memp_malloc(MEMP_PBUF_POOL);` It can be seen that the system calls the memory pool allocation function `memp_malloc` for memory allocation. The PBUF of the PBUF_POOL type that was successfully assigned is shown below:



The system call allocates multiple fixed-size PBUF_POOL type PBUs and chains these PBUs into a linked list to meet the user's allocation space request.

The PBUs of type PBUF_ROM and PBUF_REF are basically the same, and their applications are to allocate a corresponding PBUF structure header in the memory heap without requesting space in the data area. This is the biggest difference between them and PBUF_RAM and PBUF_POOL. The difference between the PBUF_ROM and PBUF_REF types is that the former points to a certain piece of data within the ROM space, while the latter points to a certain piece of data within the RAM space. Let's take a look at how the source code applies for the PBUF_ROM and PBUF_REF types. where p is a PBUF-type pointer.

`p = memp_malloc(MEMP_PBUF);` It can be seen that the system calls the memory pool allocation function `memp_malloc` for memory allocation. And the type of memory pool requested at this moment is `MEMP_PBUF`, not `MEMP_PBUF_POOL`, halo... This is so depressing. The `MEMP_PBUF` type mempool size is exactly the size of a pbuf header, as this pool is made by LWIP for PBUF_ROM and PBUF_REF types of pbufs. LWIP is still really thoughtful, it will tailor different types of pools for different data structures. A properly assigned PBUF of type PBUF_ROM or PBUF_REF is structured as follows:



Note: All the above images are from the document "Design and Implementation of the LWIP: TCP/IP Stack", and these diagrams have a common error, that is, the positions of the len and tot_len fields are reversed, which is secretly happy.

Finally, for a packet, it may use any of the PBUF types described above, most likely a large string of different types of PBUFs concatenated together to hold data for a packet.

Ad..... The next section focuses
on the memory release of PBUF.

4 PBUF release, release, release

After complaining, go on. Yesterday we talked about the memory application of the data buffer pbuf, and today we will continue to explore its memory release process. Since PBUF applications are mainly implemented through memory heap allocation and memory pool allocation, the release of PBUF must also be discussed separately in these two cases.

Don't panic, before we start the discussion, let's talk about the premise that a certain PBUF can be released. This is easy to judge in LWIP, because as mentioned in the previous section, the ref field of the pbuf represents the number of times the pbuf has been referenced, and when the pbuf is created, the initial value of the field is 1, so it can be judged that the pbuf can only be deleted when the ref field of the pbuf is 1, so the pbuf structure in the middle of the pbufs linked list will not be successfully deleted because their ref value is at least 2. Therefore, the PBUF that can be deleted must be the first node of a PBUF chain. Of course, the deletion of PBUF is far from simple, and another point to pay special attention to is that if the first node of a PBUF chain is successfully deleted, the second node of the PBUF chain will naturally become the first node of the PBUF chain, at this time, the ref value of the node may become 1 (the node is not referenced), in this case, the node will also be deleted, because LWIP thinks that it stores the same packet as the first node. When the second node is also deleted, LWIP will go to see if the third node meets the deletion criteria... Just keep deleting it. Of course, if the ref value of the second node is greater than 1 after the first node is deleted, it means that the node is still referenced elsewhere and can no longer be deleted, and the deletion work ends here. This passage is very saliva-like, so let's take an example to see this deletion process. If our pbufs linked list now has three pbuf structures connected by A--->B--->C, using the pbuf_free(A) function to delete the pbuf structure, let's use several different sets of ref values of ABC to see the deletion results:

- (1) 1->2->3 After the function is executed, it becomes ... 1->3, node BC is still there;
- (2) After the 3->3->3 function is executed, it becomes 2->3->3, and node ABC is still there;
- (3) 1->1->2 The function becomes after execution. 1, node C is still there;
- (4) After the 2->1->1 function is executed, it becomes 1->1->1, and node ABC is still there;
- (5) 1->1->1 becomes after the function is executed, and all nodes are deleted. If you could say Daigo empowerment, that would be my greatest motivation.

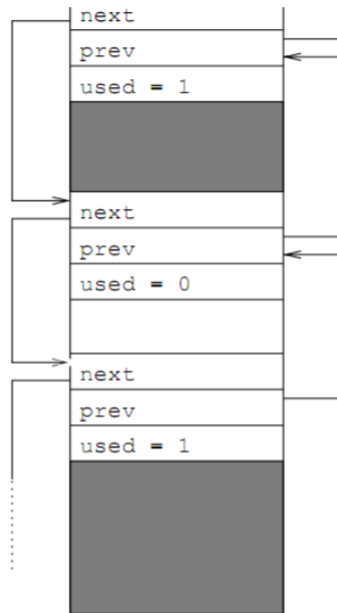
When a pbuf structure can be deleted, LWIP first checks which of the four types of pbuf belongs to the previous section, and calls different memory release functions to delete it depending on the type. PBUF_POOL and PBUF_ROM types, PBUF_REF types need to be deleted by the memp_free() function, and PBUF_RAM types need to be deleted by the mem_free() function for unexplained reasons.

The PBUF_RAM type comes from the heap, so you need to release the pbuf back to the heap via the mem_free() function. Here, let's first look at the organizational structure of the memory heap, see the figure below, inside the memory heap, the memory heap management module stores the memory allocation record by placing a relatively small structure on top of each memory allocation block (note that this small structure is automatically attached by the memory management module, independent of the user's application size). This struct has three member variables, two pointers and a flag, as shown in the figure. next and prev point to the next and previous allocated blocks of memory, respectively, and the used flag indicates whether the memory block has been allocated. The two places to note in the diagram, first, are different in the size of each memory block in the diagram and can change at any time. Second, when the system is initialized, the entire memory heap is a memory block, and the following figure is the result of the memory heap after multiple allocations and releases.

The memory heap management module searches all unused memory allocation blocks according to the size of the allocation requested, and the first memory block that meets the conditions will be allocated to the applicant. When the allocation is successful, the memory management module will immediately insert a small struct behind the allocated data area, and string this struct together with the next and prev pointers for convenience

Next time allocation. After several applications and releases, we see the memory heap organization model in the figure.

When memory is freed, the usage flags of the previous and next allocated blocks are checked to prevent memory fragmentation, and if any of them are not already used, the memory block will be merged into a larger unused memory block. The memory heap management module does this, it goes back a few bytes according to the release address provided by the user to find this small structure, and uses this structure to achieve operations such as memory heap merge. After the allocated memory blocks are reclaimed, they are cleared with the flag used. Of course, if both the previous and the next allocation block have been used, then freeing is the easiest case, but this is also the root cause of the heap fragmentation problem.



Hey, it's dead.

Next, let's talk about the other three structures that release PBUF back into the mempool through the `memp_free()` function. The structure of mempool POOLS has been discussed in the previous article, with PBUF_POOL-type pbufs mainly using MEMP_PBUF_POOL-type pools, and PBUF_ROM and PBUF_REF-type pbufs mainly using MEMP_PBUF-type pools. This sentence is too roundabout and you should read it twice. The POOL structure has a next pointer at the beginning of the POOL structure, which is used to point to the next POOL of the same type, and is used to connect POOLS of the same type into a one-way linked list.

```
memp = LWIP_MEM_ALIGN(memp_memory); for (i = 0; i < MEMP_MAX;
++i) { // Operate on various types of POOLS in turn
```

```
    memp_tab[i] = NULL;
```

The idle linked header is initially empty

```
    for (j = 0; j < memp_num[i]; ++j) { Chain homogeneous POOLS into a linked list
```

```
    memp->next = memp_tab[i]; memp_tab[i] = memp; memp = (struct memp *)
((u8_t *)memp + MEMP_SIZE + memp_sizes[i]); Get the address of the
next} //a POOL}
```

There are several important global variables in the above code, `memp_memory` which is the start address of the buffer pool, which has been discussed earlier; `MEMP_MAX` is the number of POOL types; `memp_tab` Used to point to the starting node of a certain type of POOL idle linked list; `memp_num` represents the number of various types of POOL; `memp_sizes` represents the size of a single POOL of various types, and for POOLS of type `MEMP_PBUF_POOL` and `MEMP_PBUF`, its size is the sum of the pbuf header and the size of the pbuf-mountable data.

On this basis, the release of the POOL pool is simple, first find the corresponding free linked header `memp_tab` according to the type of POOL, insert the POOL into the linked header, and point the `memp_tab` to the linked header, which is simple and fast. As for the application of the POOL pool, it is naturally the operation of the `memp_tab` head, I believe you understand.

Well, at this point, the memory-related mechanism of LWIP is basically introduced. Of course, it can't be that simple, and I'll explain it again where I use it in the future.

Organize and finish. The weekend is coming....a day of hibernation is essential....haha

5 Network interface structure, network interface structure, network interface structure, network interface structure

I just don't want to put this heart into words. There is also a sentence in front of it: trusting others does not mean weakness. I just pretend to know nothing about all things, so that I can gain all your tenderness. Thank you for everything you have done, and now it will start all over again. I can only give you this unforgettable thought.

People always say that "the night will pass", but that is just a white lie. I think even one person should be able to survive, because your smile has been engraved in my heart forever, and the heart of trusting others that should have been abandoned by me.

The above content is plagiarized from the lyrics of a certain beauty. Today we will discuss how LWIP handles the relationship with the underlying hardware, i.e., the network card chip. Why discuss this issue in the first place? Like many other TCP/IP implementations, LWIP is designed to implement TCP/IP with a reference to a layered protocol. LWIP is logically divided into four layers: link layer, network layer, transport layer, and application layer. Note that although LWIP also uses a layering mechanism, it does not have a strict division between layers, and there can be more or less cross-access between layer protocols, that is, the upper layer can be aware of the caching mechanism used by the lower layer protocols. As a result, the layers can reuse buffers more efficiently. In addition, the application process and stack code can use the same memory, and the application can read and write directly to the internal cache, thus saving the overhead of executing copies. We'll start with the lowest link layer of LWIP and start the entire LWIP internal protocol journey.

In LWIP, a hardware network interface is described through a network structure called netif. This interface structure is relatively simple, let's analyze this structure from the source code structure:

```
struct netif {  
    struct netif *next;    A pointer to the next netif structure  
    struct ip_addr ip_addr;    IP address-related configuration  
    struct ip_addr netmask;  
    struct ip_addr gw;  
  
    err_t (* input)(struct pbuf *p, struct netif *inp);    Calling this function can get one from the network card  
                                                         Data packets  
    err_t (* output)(struct netif *netif, struct pbuf *p, // IP layer calls this function to send to the network card  
                    struct ip_addr *ipaddr);              One packet  
  
    err_t (* linkoutput)(struct netif *netif, struct pbuf *p);    The ARP module calls this function to send  
                                                         a packet to the network/card  
  
    void *state;    Users can play this pointer independently to point to the information of the network card that the user is concerned  
    u8_t hwaddr_len;    The hardware address length, for Ethernet, is the MAC address length,  
                        which is 6 bytes u8_t hwaddr[NETIF_MAX_HWADDR_LEN]; MAC address  
    u16_t mtu;    The maximum number of bytes that can be transmitted at a time is generally set to 1500  
    u8_t flags;    The status information of the network card is marked bit  
  
    char name[2];    The type of device driver used by the network interface  
    u8_t num;    Used to indicate different network interfaces using the same driver type};
```

The next field is a pointer to the next netif structure. One of our products may have multiple network card chips, LWIPs

The structure of all the network card chips will be chained into a linked list for management, with a `netif_list` global variable pointing to the head of the linked list. `next` field is used for linked lists.

The three fields `ip_addr`, `netmask`, and `gw` are used to send and process packets, and represent the IP address, subnet mask, and gateway address, respectively. The first two fields play an important role when sending packets, and the third field seems to be of little use. The IP address and NIC device must correspond one by one. If you don't even know what IP addresses are, subnet masks, and what they do, then you need to check out Chapter 3 of TCP/IP Protocol Detailed Volume 1.

`input` field points to a function that submits the packets received by the NIC device to the IP layer, and when used, the input pointer can be pointed to the function, which will be discussed in detail later. The function's two parameters are of type `pbuf` and type `netif`, and the return arguments are of type `err_t`. where PBUF represents the received packet.

`output` field to a function that is closely related to the specific network interface device driver and is used at the IP layer to send a packet to the network interface. The user needs to write the function based on the actual NIC and point the output field to the function. The three parameters of the function are `pbuf` type, `netif` type, and `ip_addr` type, and the return parameter is `err_t` type. where PBUF represents the packet to be sent. `ipaddr` represents the address to which the NIC needs to send the packet to, which should be the IP address of the host that receives the actual link layer frame, not necessarily the IP address that the packet ultimately needs to reach. For example, when an IP packet is to be sent to a host that is not on the local network, the link layer frame is sent to a router in the network. In this case, the IP address given to the output function will be the address of this router.

The `linkoutput` field basically does the same thing as the output above, but this function is called in the ARP module, so I won't go into detail here. Note that this function only has two parameters. In fact, the implementation of the output field function ultimately calls the `linkoutput` field function to send the packet out.

The `state` field can point to some information about the device that the user cares about, and the user can play with it or not. `hwaddr_len` and `hwaddr[]` represent the MAC address length and MAC address, with a typical MAC address length of 6.

The `mtu` field represents the maximum number of bytes that the network can transmit at one time, which is generally set to 1500 for Ethernet, not to mention. The `flags` field is an important control field, including NIC function enablement, broadcast enablement, ARP enablement, and other important control bits.

The `name[]` field is used to hold the name of each network interface. Use a two-character name to identify the type of device driver used by the network interface, which is set by the device driver and should reflect the type of hardware represented by the network interface. For example, the network interface name of a Bluetooth device can be `bt`, and the name of an IEEE 802.11b WLAN device can be `wl`. Of course, if two network interfaces have the same network name, we use the `num` field to distinguish between different network interfaces of the same category.

At this point, you may be confused, too abstract things are too easy to get entangled. Well, let's take an example of how an Ethernet network card interface structure is initialized and how packets are received and sent. Let's take a look at the initialization process first, source code:

```
static struct netif enc28j60;           (1)
struct ip_addr ipaddr, netmask, gw;    (2)

IP4_ADDR(&gw, 192, 168, 0, 1);         (3)
IP4_ADDR(&ipaddr, 192, 168, 0, 60);    (4)
IP4_ADDR(&netmask, 255, 255, 255, 0); (5)

netif_init();                          (6)
netif_add(&enc28j60, &ipaddr, &netmask, &gw, NULL, ethernetif_init, tcpip_input); (7)
netif_set_default(&enc28j60);          (8)
netif_set_up(&enc28j60);               (9)
```

The above (1) declares a variable of the netif structure enc28j60, which I chose to use since the network card chip enc28j60 is used on my board. (2) Declares three variables for staging IP addresses, subnet masks, and gateway addresses, which are 32 bits long. (3) ~ (5) is the initialization of the above three address values, and the process is simple.

(6) It's simple, it just needs to initialize the global variable netif_list as described above: netif_list = NULL. (7) Call the netif_add function initialization variable enc28j60, the two more important parameters are ethernetif_init and tcpip_input, the former is the underlying interface initialization function defined by the user, and the tcpip_input function is the function of submitting packets to the IP layer. Let's take a look at the source code: struct netif *

```
netif_add(struct netif *netif, struct ip_addr *ipaddr, struct
ip_addr *netmask, struct ip_addr *gw, void *state, err_t (* init)
(struct netif *netif), err_t (* input)(struct pbuf *p, struct
netif *netif)){
```

```
static u8_t netifnum = 0;
netif->ip_addr.addr = 0;    The value of each field in the reset variable enc28j60
```

```
netif->netmask.addr =
0; netif->gw.addr =
0; netif->flags = 0;    This NIC does not allow any function to be enabled
```

```
netif->state = state;    Points to information that the user cares about, here NULL
netip->= nedipnum++; 设置 also 字段, nettip->input=input; //如前所诉, input 函数被赋值
```

```
netif_set_addr(netif, ipaddr,          Set the three addresses of the variable enc28j60
netmask, gw);
if (init(netif) != ERR_OK) {    The user's own underlying interface initialization function
return NULL;}
```

```
netif->next = netif_list; Insert the initialized node into the linked list netif_list
netif_list = netif;    netif_list Points to the linked header
```

return netif; } The above initialization function calls the underlying interface initialization function defined by the user himself, here it is ethernetif_init, let's take a look at its source code:
err_t ethernetif_init(struct netif *netif){

```
net->name[0] = IFNAME0;    Initialize the name field of the variable enc28j60
net->name[1] = IFNAME1;    IFNAME is defined outside of the file, so you don't have to
worry about its specific value here
```

```
netif->output = etharp_output;    The IP layer sends packet functions
netif->linkoutput = low_level_output; //ARP module sends packet
function low_level_init (netif); return ERR_OK the underlying
hardware initialization function; }
```

Oh my God, and function calls! low_level_init functions are functions that are closely related to the hardware we use.
Ahhh

6 Ethernet data reception Ethernet data reception Ethernet data reception Ethernet data reception

Young people don't work hard, and when they grow up, they write programs. Tragic!
Yesterday I said that the `low_level_init` function is a hardware-related initialization function that we use, take a look: `static void low_level_init(struct netif *netif) {`

```
netif->hwaddr_len = ETHARP_HWADDR_LEN; Set the hwaddr_len field of the
variable enc28j60 netif->hwaddr[0] = 'F'; MAC address netif->hwaddr[1] = 'O' for
the initialization variable enc28j60; Whatever address the user wants, but don't
use it with netif->hwaddr[2] = 'R'; Duplicate MAC addresses for network devices.
netif->hwaddr[3] = 'E'; netif->hwaddr[4] = 'S'; netif->hwaddr[5] = 'T';
```

```
netif->mtu = 1500; The maximum allowed transport unit // allows the broadcast and ARP functions of the
network card, and the network card allows hardware link connections
```

```
netif->flags = NETIF_FLAG_BROADCAST | \
NETIF_FLAG_ETHARP | NETIF_FLAG_LINK_UP;
```

```
enc28j60_init(netif->hwaddr); Hardware initialization functions that are closely related to the underlying driver hardware
```

At this point, finally the variable `enc28j60` is initialized, and the network card chip `enc28j60` it describes is also initialized, and the variable `enc28j60` is also linked to the linked list `netif_list`.

Then call the `netif_set_default` function to initialize the default network interface in the above statement (8). In addition to a linked list of `netif_list` global variables pointing to the `netif` network interface structure, the stack also has a global variable `netif_default` a global variable pointing to the default network interface structure. When the IP layer has data to send, it will first select a network interface that meets a certain condition with `netif_list` as the index to send packets, but when such an interface is not found, the stack will call the default network interface to send the packet directly, so the meaning in (8) is to set the network interface described by the variable `enc28j60` as the default network interface.

(9) Call the function `netif_set_up` enable the network interface, which is achieved through a simple statement:

```
netif->flags |= NETIF_FLAG_UP;
```

At this point, the initialization of the network card is complete, and it can receive and send data packets normally. Let's discuss the receipt and transmission of network card packets.

LWIP implements a framework for receiving a packet and sending a packet function, which are `low_level_input` and `low_level_output`, and the user needs to complete these two functions with the actual NIC driver. As mentioned in the first article, a typical LWIP application system consists of three processes: first the upper-level application process, then the LWIP stack process, and finally the underlying hardware packet receiving process. Now let's talk about the third process to see how packets are received and passed to the upper layer. But before that, it is necessary to talk about the format of the packets received by the Ethernet card. As shown in the figure below,



LWIP uses a `eth_hdr` data structure to describe the 14 bytes of an Ethernet packet header. as follows

```
PACK_STRUCT_BEGIN
    struct eth_hdr {PACK_STRUCT_FIELD(struct
        eth_addr dest);
        PACK_STRUCT_FIELD(struct eth_addr src);
        PACK_STRUCT_FIELD(u16_t type);
    } PACK_STRUCT_STRUCT; PACK_STRUCT_END PACK_STRUCT_XXX
```

of these are macro definitions related to compiler word alignment, which I won't go into detail here. The above three fields of `dest`, `src`, and `type` correspond to the destination MAC address, source MAC address, and type domain fields in the figure above.

Based on the above discussion, let's take a look at this packet receiving process, the source code is as follows: `void ethernetif_input(void *arg)` When creating the process, the `netif` structure of a network interface structure is passed in as a parameter

```
struct eth_hdr *ethhdr;
struct pbuf *p;

struct netif *netif = (struct netif *)arg;

while
(1) {
    p = low_level_input (netif);    Receive a packet
    if (p == NULL)                  If the packet is empty,
        continue;                  The loop ends and the next receiving process is initiated

    ethhdr = p->payload;             Retrieve the data in the packet
    switch (htons(ethhdr->type)) // 判断数据包类型
    {
        Only IP and ARP packets are processed
        case ETHTYPE_IP:           // IP packets
        case ETHTYPE_ARP:          // ARP packets
            if (netif->input(p, netif) != ERR_OK) Sends the packet to the parent application
                {                             function
                    pbuf_free(p); p =
                    NULL;} break;
            default:
                pbuf_free(p);
    }
```

```

        p = NULL;
        break;
} //switch} //while} //main function To create the above process, you need to pass
in the netif structure pointer of the network interface structure as a
parameter, and use the following statement to implement it in UC/OSII.

```

```

OSTaskCreate(ethernetif_input, (void *)&enc28j60,
             &T_ETHERNETIF_INPUT_STK[T_ETHERNETIF_INPUT_STKSIZE-1]
             ETH_IF_TASK_PRIO);

```

There are three things to note in the packet receiving process. First, the method of receiving data packets is the query method, that is, the processor continuously reads data to the network card chip, and if the data cannot be read, the controller will restart a reading timing; If the data can be read successfully, the data is communicated to the upper level through the input function registered with the NIC for processing. The packet receiving process implemented using the query method must have a lower priority than the priority of other processes in the system, otherwise it will block the operation of processes with a lower priority than it. The above program has an improvement, that is, when the read packet is empty, the receiving process calls the system function to delay itself for a period of time before starting the next read process, which can prevent the operation of the lower priority process, and the disadvantage is that the receiving of the packet cannot be responded to in time. In fact, the reception of data packets can be achieved in the form of interruption, which is a better way. Generally, the network card chip has an interrupt function, that is, when the network card receives a data packet, it can generate an interrupt signal to tell the controller that it has received a data packet. The controller starts a read packet timing at this time, and it can effectively read non-null packets. So it can be done this way to achieve a packet receiving process: when no packet is received, the packet receiving process is blocked under a semaphore, and when a packet arrives, the NIC chip generates an interrupt signal, and the processor enters the interrupt processing and releases a semaphore. After interrupting and exiting, the packet receiving process obtains the semaphore, reads the packet from the network card chip, and submits the packet to the upper layer for processing.

The second thing to note is the use of the `htons(ethhdr->type)` function, which converts a half-word long of data from a network byte order to the byte order supported by our processor. To explain, in the field of computer architecture and computer communication, the storage mechanism for half characters, characters, etc. may be different. There are two main storage mechanisms commonly used: big-endian and little-endian, i.e., large and small. For the large-end mode, the high-level bytes of a half-word or word data are placed at the low address side of memory, and the low bytes are discharged at the high address side of memory. For the small end mode, it's just the opposite. Since the ARM processor we use uses the small-end mode, and the received network byte data uses the large-end mode, here we call the function `htons` to convert the large end and the small terminal, which is actually to swap the two bytes in order. When you call `htons(ethhdr->type)`, the value of `ethhdr->type` is 0x0800 or 0x0806, etc.

Finally, it should be noted that `netif->input` is already set to point to the `tcpip_input` function when the structure `enc28j60` is initialized, so it is actually calling the `tcpip_input` function to submit packets to the upper layer. `tcpip_input` is an IP layer function, and from this we can see that a big feature of LWIP is that there is no obvious boundary between the layers. As mentioned earlier, the LWIP stack process will block a mailbox waiting for packet input after completing the initialization work, which is right, the `tcpip_input` function sends a message to this mailbox containing the address of the received packet store. Once the LWIP stack process has obtained the address from the mailbox, it can process the packets.

At this point, the packet reception can be considered a great success, and the sending of the packet is very simple, because it does not need to use a special process to implement it like packet reception, but it is like this: when there is a packet to be sent at the upper level, it can directly call `netif->linkoutput` to send the packet. `netif->linkoutput` is already set to point to the `low_level_output` function when the structure `enc28j60` is initialized, which is closely related to the underlying hardware driver and is used to send a packet. The user should implement this function in conjunction with the specific NIC driver.

7 ARP table

Hahaha..... I wish my buddies and sisters all the best in the new year. Last year (well, that's exactly the word) we've talked about all sorts of things, including LWIP porting points, memory management, packet management, network interface management, and more. The new year continues to be strong.

ARP, which stands for Address Resolution Protocol, is a protocol located at the bottom of the TCP/IP protocol stack. The communication of any network is based on the underlying hardware link, and the underlying data link has its own set of addressing mechanisms, and in Ethernet, different network communication devices are often marked through a 48-bit MAC address. The upper layer of the TCP/IP protocol uses IP addresses as the communication addressing mechanism between each host. When the source host wants to send data to the target host, it only knows the IP address of the target host, and at this time, the source host needs to convert the IP address to the MAC address corresponding to the destination host, so that the correct channel can be selected on the data link to transmit the data, which is the role of ARP. Hey, it's complicated!

A description in the protocol may be clearer: The basic concept behind ARP is that each network interface has a hardware address (a 48-bit value identifying a different Ethernet or token ring network interface), and data frame exchange at the hardware level must have the correct hardware interface address. However, TCP/IP has its own address: an IP address for 32bit. Knowing the IP address of the host does not allow the kernel to send a frame of data to the host. The kernel, such as the Ethernet driver, must know the hardware address of the destination to send data. The function of ARP is to provide a dynamic mapping between 32-bit I-P addresses and hardware addresses with different networking technologies. The basic function of the ARP protocol is to query the MAC address of the target device through the IP address of the target device to ensure the progress of communication.

The core of the ARP protocol implementation is the ARP cache table, which is the establishment, update, query and other operations of the cache table. The ARP cache table is composed of a cache entry (entry), and the data structure that describes the cache table entry in LWIP is called `etharp_entry`, which is the source code:

```
struct etharp_entry {  
    #if ARP_QUEUEING  
        struct etharp_q_entry *q;    Packet buffer queue pointer  
    #endif  
    struct ip_addr ipaddr;    The destination IP address  
    struct eth_addr ethaddr;    // MAC address  
    enum etharp_state state;    Describe the status of that entry  
    u8_t ctime;    Describe the time information for that entry  
    struct netif *netif;    Information about the corresponding network interface  
};
```

`ARP_QUEUEING` is the compile option, which indicates whether to allow cache entries to have a packet buffer queue, which is set in `opt.h`. Why buffer the queue pointer with packets, and then slowly talk about it. The `ipaddr` and `ethaddr` fields are used to store IP addresses and MAC addresses, respectively, and they are the core part of the ARP cache entry. `state` is an enum type, which represents the state of the cached entry, and there are three possible states for a table item, which we describe with the enumeration type `etharp_state`.

```
enum etharp_state {  
    ETHARP_STATE_EMPTY =  
    0, ETHARP_STATE_PENDING, ET  
    HARP_STATE_STABLE};
```

The LWIP kernel creates an ARP cache table in the form of an array, as follows:

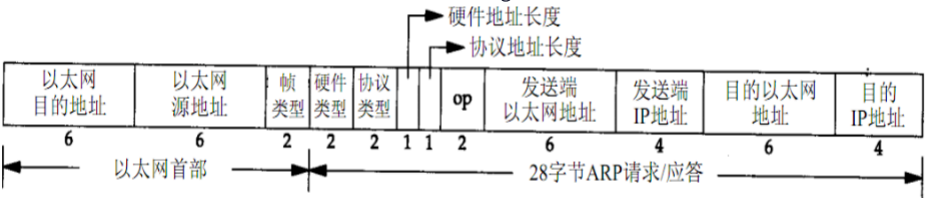
```
static struct etharp_entry arp_table[ARP_TABLE_SIZE];
```

Each cache entry in the initialized cache table is in the initial state, no information is recorded, at this time each entry is in the ETHARP_STATE_EMPTY state, the ETHARP_STATE_PENDING state indicates that the entry is in an unstable state, it is likely that the entry only records the IP address, but also records the MAC address of the corresponding IP address, at this time the entry is in ETHARP_STATE_PENDING state. In this state, the LWIP kernel makes a broadcast ARP request to the data link to have the host of the corresponding IP address respond to its MAC address, and when the source host receives the MAC address, it updates the corresponding ARP entry. When an ARP entry is updated, it fully records a pair of IP addresses and MAC addresses, at which point the entry is in ETHARP_STATE_STABLE state. Note that when a table item is in the PENDING state, the packets to be sent to the IP address in the table entry will be connected to the packet buffer queue corresponding to the table key, and these packets will not be sent until the table entry is stable. That's why each table key needs to have a packet buffer queue pointer.

The ctime field records the time a table entry is in a certain state, and when the ctime value of a table key is greater than the specified maximum survival value of the table key, the table entry will be deleted by the kernel. In the first lecture, we talked about LWIP timeout events, to use the ARP function, you must set an ARP timeout event, the basic function of this timeout event is to add 1 to the ctime field value of each table item, and then delete those table entries with a survival time greater than the maximum survival value.

Now, let's talk about the basis for properly establishing ARP caches: ARP packets. To establish a table entry on the source host that corresponds to the IP address and MAC address of the target host, the basic information interaction between the source host and the destination host is necessary, simply put, how the source host tells the destination host: I need your MAC address; And how the destination host replies: This is my MAC address. ARP packets, which comes in handy.

ARP packets can be divided into ARP request packets and ARP reply packets, ARP packets will be sent with Ethernet packet headers when they reach the underlying link, and finally the data header format on the link is as shown in the figure below.



The first two fields in the Ethernet header are the destination MAC address and the source MAC address of Ethernet, which have been explained in the previous chapter. A special address with a destination address of all 1 is the broadcast address. Before the ARP table entry is established, the source host only knows the IP address of the destination host, but does not know its MAC address, so on the data link, the source host can only send ARP request packets by broadcasting. All Ethernet interfaces on the cable receive the broadcast packet and detect whether the packet is sent to itself, which is achieved by checking the IP address of the destination, and if it is sent to itself, the destination host needs to reply with an ARP reply packet to the source host to tell the source host its MAC address.

The two-byte-long Ethernet frame type indicates the type of data behind. The field has a value of 0x0806 for ARP request or response packets and 0x0800 for IP packets.

After talking about the Ethernet data header, let's talk about the ARP data header. The Hardware Type field represents the type of hardware address, and its value of 1 represents the Ethernet MAC address, which is 6 bytes long. The Protocol Type field indicates the type of protocol address to be mapped. Its value of 0x0800 means to map as an IP address. It has the same value as the value of the type field in the Ethernet data frame header that contains the IP-datagram.

The next two 1-byte fields, Hardware Address Length and Protocol Address Length, indicate the length of the hardware address and protocol address in bytes, respectively. For an Ethernet ARP request or answer, they have values of 6 and 4, respectively.

The action field op indicates four types of actions, which are a RP request (value 1), a RP reply (value 2), a RP request (a value of 3), and a RP response (a value of 4), here we only care about the first two types. This field

Required because the frame type field value for the ARP request and the ARP reply is the same. The next four fields are the Ethernet MAC address of the sender, the IP address of the sender, the Ethernet MAC address of the destination, and the IP address of the destination.

Note that there is some duplication here: both the sender's Ethernet MAC address is present in the Ethernet dataframe header and in the ARP request dataframe. For an ARP request, all fields except the destination MAC address have padded values. When the destination host receives an ARP request packet to itself, it fills in its hardware address, then exchanges the source host information and destination host information of the request packet, sets the operation field op to 2, and finally sends the newly constructed packet back, which is the ARP response.

Finally, let's use the source code to see how LWIP describes this header above: struct etharp_hdr {

PACK_STRUCT_FIELD(struct eth_hdr ethhdr);	14-byte Ethernet data header
PACK_STRUCT_FIELD(u16_t hwtype);	2 bytes of hardware type
PACK_STRUCT_FIELD(u16_t therefore);	2 bytes of protocol type
PACK_STRUCT_FIELD(u16_t _hwlen_protolen);	Two 1-byte length fields
PACK_STRUCT_FIELD(u16_t opcode);	2 bytes of operation field op
PACK_STRUCT_FIELD(struct eth_addr shwaddr);	6-byte source MAC address
PACK_STRUCT_FIELD(struct ip_addr2 sipaddr);	4 byte source IP address
PACK_STRUCT_FIELD(struct eth_addr dhwaddr);	MAC address for the 6-word program
PACK_STRUCT_FIELD(struct ip_addr2 dipaddr);	The IP address of the 4-word program

} PACK_STRUCT_STRUCT; No Tang Seng, which is completely consistent with the previous descriptions. PACK_STRUCT_FIELD() is a macro definition that prevents compiler word alignment, as I have already said.

8 ARP tables, table queries, queries, queries, and queries

ARP attack is an attack technique that targets the Ethernet Address Resolution Protocol (ARP). In the LAN, the ARP virus receives the broadcast ARP request packet, which can resolve the (IP, MAC) addresses of other nodes, and then the virus disguises itself as the destination host and tells the source host a fake MAC address, so that all the data packets sent by the source host to the destination host are intercepted by the virus software, and the source host and destination host are unaware. ARP attacks achieve ARP spoofing by forging IP addresses and MAC addresses, which can generate a large amount of ARP traffic in the network to block the network, and the attacker can change the IP-MAC entry in the ARP cache of the target host as long as the attacker continuously sends forged ARP response packets. The ARP protocol is designed without cybersecurity in mind, making it vulnerable to ARP attacks. Hackers can eavesdrop on all (IP, MAC) addresses on the network by reading the broadcast ARP request packets sent to the door within the LAN. When the source node receives an ARP response, it will not question it, making it easy for hackers to impersonate others.

This section mainly explains the operations of ARP table creation, update, query, etc. for ARP. Here we will first explain the functions of each submodule of ARP from a few simple functions, and then combine each module with the upper protocol to explain the ARP module in a macro way.

The first function that needs to be impatiently mentioned is `find_entry`, the most important input of which is an IP address, and the return value is the ARP cache key index corresponding to that IP address. The function declaration prototype is as follows,

```
static s8_t find_entry(struct ip_addr *ipaddr, u8_t flags)
```

Here, it's important to translate the comments in the source code: the function mainly functions to find a matching ARP entry or create a new ARP entry and return the index number of that entry. If the parameter `ipaddr` is a given non-empty content, the function needs to return an index key that is in pending or stable, and if there is no matching entry, the function needs to return an empty key, but the value of the IP field of the item is set to the value of `ipaddr`, in which case `find_entry` the caller needs to change the item from state empty to pending after the function returns. There is also a case where if the parameter `ipaddr` is null, it also returns an entry with a status of empty.

If all of these entries are used up and the value of the parameter `flags` is set to `ETHARP_TRY_HARD`, then `find_entry` recycles the oldest ARP entry and returns it with the empty status set.

This function is relatively large, with nearly 200 lines of code, so I won't post it here, just talk about its workflow. This part of the discussion still refers to the blog of a certain hero on the Internet, I can't remember the name, I'm sorry! The Internet is sometimes a good thing, and it is becoming more and more clear. Well, take a look at the `find_entry` workflow.

First of all, LWIP has a clever thing, it doesn't just rush up and search all the table entries in the ARP cache, but makes an assumption that this time the item index is still the same as the last time (in many cases). Therefore, there is a global variable `etharp_cached_entry` in LWIP, which always holds the index number used last time, if this index happens to be the content we are looking for, and the index entry is already in a stable state, then return the index number directly and you're done, we're really fast!

If the situation is not ideal, the entire ARP table must be searched, and the retrieval process starts from the first entry of the ARP table and then searches back to the last item in turn, which is a more complex process. For each item to determine whether it is in the empty state first, `find_entry` only cares about the index value of the first item with the empty state, and is not interested in the empty entries after the index value, so ignore it. If a table key is not in an empty state, it is determined whether it is in a pending state. For the table entry in the pending state, you need to do the following, first check whether the IP address stored in it matches our `ipaddr`, if it matches, return the index value, remember to update `etharp_cached_entry` to the index value, if it does not match, then judge whether the packet pointer of the index is empty, `find_entry` try to record the pending with the longest survival time

state with or without data buffered table key indexes. If a table key is also not in a pending state, it is determined whether it is in a stable state. For entries in a stable state, similar to the entries in the pending state, find_entry attempt to record the index of the stable entries that have survived the longest. It's very dizzy, I'm also dizzy, maybe you will get better after reading the following paragraph!

If we haven't found a matching key so far, it's a big deal, and we need to return an empty key index for the find_entry caller. After the above paragraph, find_entry already knows the index of the first empty state entry, the index of the oldest pending state with data buffer, the index of the oldest pending state without data buffer, and the index of the oldest stable state entry. If the value of the parameter flags is set to ETHARP_TRY_HARD, then find_entry will choose a suitable index to return in the order of a-->d-->c-->b, why in that order? Obviously, no explanation. find_entry First determine if a is within the scope of the ARP entry, if so, select a, if not, determine if b is within the scope of the ARP entry, and so on. When an index is selected, the table entry corresponding to the index is set to the empty state, the IP address of the table entry is set to the value of ipaddr, the ctime value is set to 0, and finally the index is returned. At this point, find_entry is done!

Next, one function that I'm interested in is etharp_query, which functions to send a packet to a given IP address or send an ARP request, although it's far from simple. It is still necessary to translate the content of the function note in the source code: if the given IP address is not in the ARP table, a new ARP entry will be created, the key will be in the pending state, and an ARP request packet about the IP address will be broadcast, and the packet to be sent will be hooked to the data buffer pointer of the table entry; If the IP address has a corresponding entry in the ARP table, but the entry is in the pending state, the operation is the same as the former, that is, an ARP request is sent and a hook packet is sent; If the IP address has a corresponding entry in the ARP table and the key is in a stable state, it will determine whether the given packet is empty, if it is not empty, the packet will be sent out directly, and if it is empty, an ARP request will be sent to the IP address.

The etharp_query function prototype is shown below, the source code is about 150 lines, and the process is mainly explained here:

```
err_t etharp_query(struct netif *netif, struct ip_addr *ipaddr, struct pbuf *q) (1) First, determine whether the given ipaddr is legitimate, and do not handle empty IP addresses, broadcast IP addresses, and multicast IP addresses. (2) Call the function with ipaddr as a parameter find_entry, and the function returns an ARP key index, which may be original
```

, the table key should be in a pending or stable state; The item may also be obtained from a new application, at which point the item should be in empty status. (3) Find the ARP key based on the returned key index, determine whether the key is in the empty state, and if so, explain

If the key is newly requested, set the key status to the pending state. (4) Determine whether the packet to be sent is empty, or determine whether the ARP table entry is in a pending state

To have one to hold, send an ARP request out, and the function that sends the ARP request is etharp_request. (5) If the packet to be sent is not empty, it will be processed differently according to the state of the ARP entry: if the ARP entry is located

If it is in a stable state, the function etharp_send_ip is directly called to send the packet. If the ARP key is in the pending state, it needs to hook the packet to the linked list of the data to be sent to the entry, because the pending key must have sent an ARP request in step (4), when the kernel receives the ARP response, it will set the key to the stable state and send all the data on its linked list. Hooking packets to the sending linked list of table entries, which is another more complicated process: the most important point is to determine the type of pbuf of the packet, and for packets of type PBUF_REF, PBUF_POOL, and PBUF_RAM, it cannot be directly attached to the sending linked list, because these packets will not be sent immediately after being hooked, which may cause the internal data of the packet to be changed while waiting to be sent. For these types of packets to be sent, you need to copy the data into a new PBUF and then hook the new PBUF to the sending linked list. At this point, the merit of the etharp_query function is complete!

Time always runs by inadvertently... I didn't finish writing.

9 ARP layer, process layer, process layer, process layer

The previous section focused on ARP cache tables and how to manipulate them, and there are three functions about ARP, two of which have already been discussed.

The last function to talk about is `update_arp_entry`, which is used to update a key in the ARP cache table or insert a new one into the cache table. This function is called after receiving an IP packet or an ARP packet. The prototype of the function is as follows, `static err_t update_arp_entry(struct netif *netif, struct ip_addr *ipaddr, struct eth_addr *ethaddr, u8_t flags)` in which the two important parameters `ipaddr` and `ethaddr` correspond to the IP address and MAC address respectively, and the function uses these two addresses to update or insert ARP Table item. Note that this source code has been processed by me, and non-key parts such as compilation options, source code comments, and debugging output information have been removed. `static err_t update_arp_entry(struct netif *netif, struct ip_addr *ipaddr, struct eth_addr *ethaddr, u8_t flags){`

```
s8_t i;    Two variables, not explained
u8_t k; i =
find_entry(ipaddr, flags);    Find or create a new ARP table key that returns its index value
if (i < 0) return (err_t)i;    If it is an invalid index value, the cache table fails to update
arp_table[i].state = ETHARP_STATE_STABLE;    Otherwise, change the corresponding
entry state to stable
arp_table[i].netif = netif;    Record the network interface

k = ETHARP_HWADDR_LEN;    This section is the MAC address in the update cache entry
while (k > 0) {
k--;    arp_table[i].ethaddr.addr[k] = ethaddr->addr[k];
}    arp_table[i].ctime = 0;    The time to live value is set to 0
if
ARP_QUEUEING// If there are unsent queues on the ARP table entry,
send those queues

while (arp_table[i].q != NULL) {    As long as there is data in the buffer linked list, it loops
struct pbuf *p;    struct etharp_q_entry
*q = arp_table[i].q;    Record the buffer linked list header
arp_table[i].q = q->next;    The buffer linked list header points to the next node
p = q->p;    Retrieves the data packets pointed to by the buffer linked list header under the record
memp_free(MEMP_ARP_QUEUE, q);    Releases the buffer linked list header under the record
etharp_send_ip(netif, p, (struct eth_addr*)(netif->hwaddr), ethaddr);    // 发送
数据包
pbuf_free(p);    // 释放数据包缓存空间}
endif

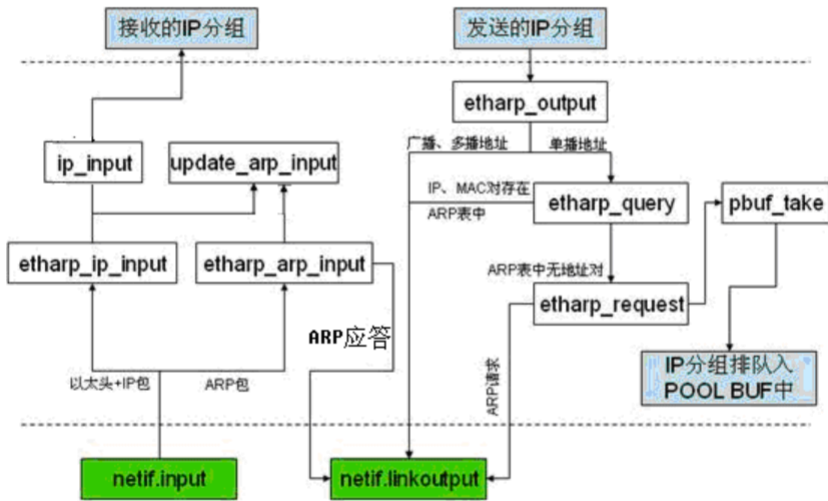
return ERR_OK;}
```

As can be seen from the source program, the process of `update_arp_entry` is as follows: first find the corresponding `ipaddr` table key by calling `find_entry`, and set the corresponding arp table entry members (mainly `state`, `netif`, `ethaddr`, `ctime`).

Finally, if a `ARP_QUEUEING` is defined and there are unsent packets in this ARP entry, send all of this data. Although it is a bit long-winded, let's take a look at several different situations in which `update_arp_entry` runs after `find_entry` execution according to different `ipaddrs`.

First of all, it is certain that the two parameters of `update_arp_entry`, `ipaddr` and `ethaddr`, must match each other, because they are parsed from the IP packet or ARP packet sent by the source host, representing the MAC address and IP address of the source host. `find_entry` Executed with `ipaddr` as a parameter, returns an ARP table key index. If the key is in the empty state, then the key must now be newly created, and it ends when the key is set to stable and the other field values of the item are set. If the key is in the pending state, the key is also set to stable and the other field values of the item, as a MAC address matching `ipaddr` has already been returned. If the table key is in a stable state, in fact, you only need to reset the value of `ctime`, but in order to save code volume, LWIP still chooses to do the same thing as in the above case, so that although some steps are redundant, it does not affect the function function. Finally, it checks whether the table entry still has data to send, and if so, sends all packets.

Now is the time to take a macro look at what ARP is and how important it plays in the overall LWIP stack. Regarding this, I have to borrow from a certain hero on the Internet, but I made some small changes to TA's picture (blushing, using the drawing tool to change it, Visio said it wouldn't work).



This diagram concisely explains the whole process of receiving and sending packets for basically all LWIPs. We can see several familiar figures: `etharp_query`, `etharp_request`, `update_arp_entry`. It has already been said before!

ARP can be simply divided into two parts in terms of function: when there is packet input, the ARP table is updated, and if it is an IP packet, it is submitted to the IP layer, and if it is an ARP packet, it responds accordingly to different ARP packet types; When sending a packet to the destination IP, it is necessary to realize the mapping of IP to MAC address through ARP, and if necessary, it is necessary to send broadcast packets to obtain the MAC address of the target machine.

LWIP receives Ethernet packets using a function pointed to by `netif.input`, which is usually `ethernet_input`. Note that this is not a direct interaction between `ethernet_input` and the underlying hardware to receive packets, but rather a lower-level function that receives the packet and delivers it to `ethernet_input`, `ethernet_input` processes it.

Ethernet frame types can be: IP, ARP, or even `pppoe`, `wlan`, etc. Here we mainly analyze IP and

ARP packets of both types. ethernet_input Judge the type of packet received based on the type field in the Ethernet header, and submit the packet to etharp_ip_input if it is an IP packet, or to the etharp_arp_input if it is an ARP packet.

For IP type packets, etharp_ip_input first checks whether the ETHARP_TRUST_IP_MAC option is enabled, and if it is enabled, it is necessary to update the ARP table with the information in the frame and the update_arp_entry function (using the source MAC address at the beginning of the frame and the source IP address in the IP packet in the frame data), and then discard the Ethernet frame header and pass the IP packet to the IP layer through the ip_input function.

For ARP packets, the etharp_arp_input function first uses the packet header information to update the contents of the ARP table, and then determines the type of the ARP packet. If it is not for yourself, just ignore it. In the case of ARP response packages, the main task is to update the ARP table, but this step has already been processed when the ARP package first came in, so there is no need to repeat it here, so that the ARP package is processed.

LWIP sends IP packets using a function pointed to by netif.output, usually etharp_output. Note that this does not mean that etharp_output directly interacts with the underlying hardware to send data packets, but processes the data packets accordingly, mainly packaging IP packets into Ethernet frame data, and finally submitting them to the netif.linkoutput function for sending.

The etharp_output function receives the packets to be sent by the IP layer and sends them out. Since it is sending IP packets, the function initially needs to increase the size of the buffer, which is the size of the data header of Ethernet. Then check the IP address, which can be divided into broadcast packets, multicast packets, and unicast packets (unicast packets are divided into whether they are inside the LAN or outside the LAN).

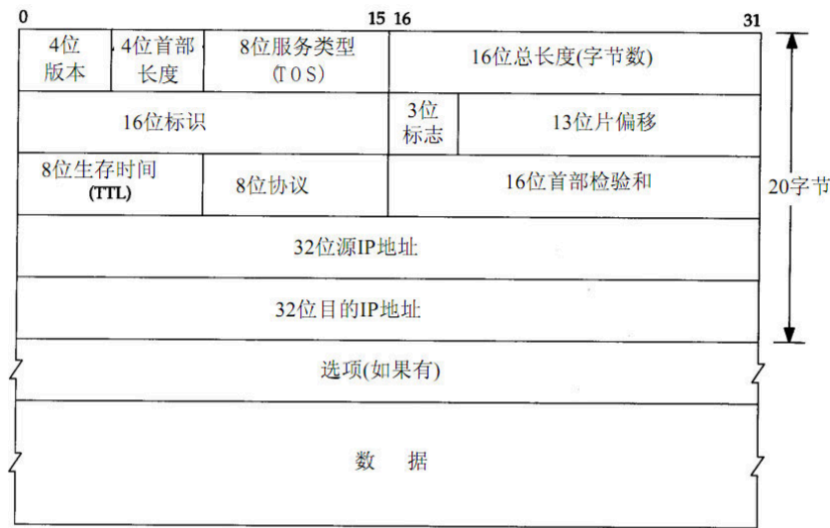
Broadcast packet: Determine whether the destination IP address is all 1 or all 0 (used in the old version), if it is a broadcast packet, the MAC address of the destination IP does not need to query the arp table, just set the MAC address to all 1 to send, that is, the MAC six byte value is 0xff, 0xff, 0xff, 0xff, 0xff, 0xff.

Multicast packet: Determine whether the target IP address is a Class D address, that is, 0xexxxxxxx, if it is multicast, the MAC address is also determined, that is, the lower 23 bits of MAC address 01-00-5e-00-00-00 are set to the lower 23 bits of the IP address. For the above two packets, etharp_output directly call the function etharp_send_ip to send the packets.

Unicast packets: To compare the destination IP and the local IP address to see if it is in the LAN or not in the LAN, set the destination IP address as the address of the default gateway, and then call the etharp_query function to send the packets out, note that these packets may be connected to the sending linked list of the relevant ARP table entries in this case, waiting to be sent.

10 IP layer by layer input input input

For the IP layer, it mainly discusses the reception of packets, the reinstallation of sharded packets, and the sending and forwarding of packets. The IP header structure is as follows, where the option field can be dispensed without, so the usual IP header length is 20 bytes.



The first field is the version number of 4bit, which is 4 for IPv4 and 6 for IPv6. The next 4-bit field is used to record the length of the head, in words. So for an IP header without any option fields, the length value is 5, and since the field has a maximum value of 15, the maximum IP header length that can be described is $15 \times 4 = 60$ bytes.

Next is an 8-bit service type field, which is mainly used to describe the type of service that is urgently needed for the IP packet, such as minimum latency, maximum throughput, highest reliability, minimum cost, etc. This field is of little use in LWIPs.

The 16-bit Total Length field describes the entire IP datagram, including the total number of bytes in the IP header. Theoretically, the total length of IP packets can be up to 65,535 bytes, but in practice, the underlying link does not allow such large packets to appear on the link, because this greatly increases the possibility of data errors, so large IP packets are often sharded at the link layer, of course, these are later.

The next 16-bit identification field is used to identify each IP datagram sent by the IP layer, with a value added by 1 for each packet sent. The 3-bit flag and 13-bit slice offset fields are then used when sharding IP packets, which are not discussed here. Only higher versions of LWIP support IP sharding.

The TTL field describes the maximum number of times the IP packet can be forwarded, with a value minus 1 for each forward, and an ICMP packet is returned to the source host when the value is 0.

The 8-bit protocol field is used to describe which protocol the IP packet is from, with a value of 1 for the ICMP protocol, a value of 2 for the IGMP protocol, a value of 6 for the TCP protocol, and a value of 17 for the UDP protocol.

The 16-bit first checksum only checks for the IP head, and it doesn't care if its internal data is wrong during transmission.

The upper-level protocol is responsible for the verification of data, such as ICMP, IGMP, TCP, and UDP protocols, which calculate the length of their headers and the entire data area. Here is another COPY of how this checksum is generated and how it is experimentally verified on the receiving end.

In order to calculate the IP check sum of a datagram on the sending end, first set the check sum field to 0. Then, each 16-bit in the header is summed in binary inverse code (the entire header is seen as a string of 16-bit words), and the result is in the test and field. When the receiver receives an IP datagram, it also sums each 16 bits in the first part with binary inverse code. Since the receiver includes the checks and fields that the sender saves in the header during the calculation, if there are no errors in the header during transmission, the result of the receiver's calculation should be all 1. If the result is not all 1 (i.e., check and error), then the IP discards the received datagram. However, error packets are not generated, and the upper layer discovers the lost datagrams and retransmits them.

Next up are two 32-bit IP addresses, no longer verbose. The last field is optional, which is optionally used by different protocols, and is not discussed here.

Now let's take a look at how this IP header is described in LWIP, using a struct called `ip_hdr`:

```
PACK_STRUCT_FIELD(u16_t _v_hl_tos);    The first three fields: version number, head length, and service type
PACK_STRUCT_FIELD(u16_t _len);         Overall length
PACK_STRUCT_FIELD(u16_t _id);          Identification field
PACK_STRUCT_FIELD(u16_t _offset);      3-bit flag and 13-bit offset field#define
IP_RF 0x8000// #define IP_DF 0x4000// Ungrouped identification bitmask#define
IP_MF 0x2000// Subsequent grouping arrives to identify bitmask#define
IP_OFFMASK 0x1fff// Get the mask for the 13-bit slice offset field

PACK_STRUCT_FIELD(u16_t _ttl_proto);    TTL field and agreement field
PACK_STRUCT_FIELD(u16_t _chksum);       First checksum field
PACK_STRUCT_FIELD(struct ip_addr src);   Source IP address
PACK_STRUCT_FIELD(struct ip_addr dest);  Destination IP address
} PACK_STRUCT_STRUCT; Note that several macro definitions are defined when declaring a struct:
IP_RF, IP_DF, IP_MF, and IP_OFFMASK, which are masks to be used when finding the two fields related
to the grouping, and can also be defined outside the struct without any effect.
```

As mentioned earlier, the packets coming in from the bottom layer of Ethernet are distributed to the IP module or ARP module through the `ethernet_input` function, and the distribution to the IP module is done by calling the `ip_input` function, of course, before submitting, the packet needs to be removed `ethernet_input` from the Ethernet header. Now let's take a look at what the function does when the packet is passed to `ip_input`. We will not touch on its internal handling of the DHCP protocol.

The first thing is to check the version number of the IP header and if the value is not 4, then immediately discard the packet. Later versions of the LWIP stack can support IPv6, but here we will only talk about IPv4. Next, the function checks whether the IP header is only stored in one pbuf, and if not, it also discards the IP packet directly, because LWIP does not allow IP packet headers to be separated into different pbufs. At the same time, the function checks whether the total length field in the IP header is greater than the total length of the submitted packets, and if so, it means that there is a transmission error and the packet is directly discarded.

Then the IP data header is checked, which is done by the function `inet_chksum`, and if the check fails, the packet is directly discarded. `inet_chksum` functions will be explained in detail if needed.

Next, we need to truncate the packet here, intercepting the packet according to the total length field of the IP packet header record, because the packet passed by `ethernet_input` only the Ethernet packet header is removed, and the Ethernet padding field and the Ethernet check field (last byte) that may exist are not processed, so we truncate them here to get a complete and unredundant IP packet.

The function then detects whether the destination IP address in the IP packet matches the local one, the local IP address is saved in the `netif_struct` variable, a system may have multiple NIC devices, which means that it has multiple `netif_struct` variables to describe these NIC devices, and it also means that the machine has multiple IP addresses that are connected to the `netif_list` linked list. The `ip_input` function traverses the `netif_struct` on the `netif_list` linked list to find a matching IP address and logs the `netif_struct` variable, that is, the NIC card. From this point of view, in the ARP section, it is better to iterate over an ARP request package received in this way before giving an ARP response, which the source code does not do, of course, this is just a personal opinion. When the traversal is completed, if there is still no matching `netif_struct` variable, it means that the packet is not for the machine, and the packet needs to be forwarded or discarded, which is done by defining the macro `IP_FORWARD`, and be careful not to forward the broadcast packet.

Next, the destination IP address determines whether the packet is a broadcast or multicast IP packet, and LWIP does not respond to these types of packets.

The next task can be said to be the most complex and difficult part of the `ip_input` function, which is the reinstallation of IP shard packets, `ip_input` function uses the packet's 3-digit flag and 13-bit slice offset field to determine whether the IP packet sent to itself is a shard packet, if so, it needs to stage the shard packet, and wait until all the shard packets are received, and then deliver the entire packet to the upper-level application. This is a process that is difficult to describe, stop here first, and we will discuss it in detail in the future content. If it is a sharded package and not the last piece, the function returns here.

Finally, the packets that get to this point must be unsharded or fully reinstalled by sharding. In this case, the `ip_input` function determines which parent protocol the packet should be submitted to based on the protocol field inside the IP packet header, and calls the corresponding function to submit the packet. If it is a UDP protocol, call the `udp_input` function; If it is a TCP protocol, call the `tcp_input` function; If it is an ICMP protocol, call the `icmp_input` function; If it is an IGMP protocol, call the `igmp_input` function; If neither is the case, call the function `icmp_dest_unreach` to return a protocol-unreachable ICMP packet to the source host while deleting the packet.

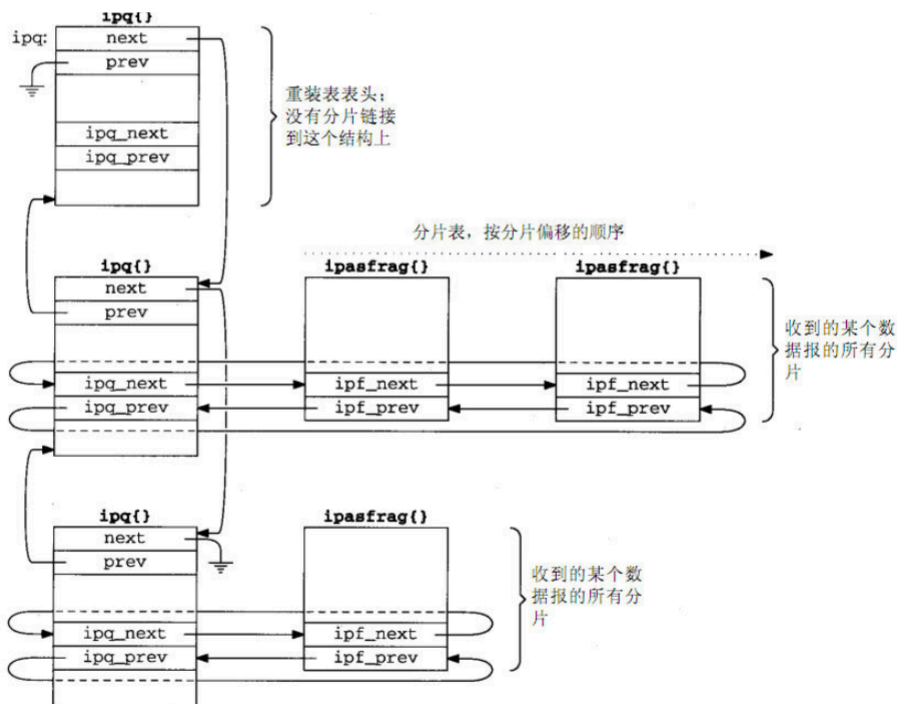
Finish writing!

11 IP ShardingReinstallPartition ReloadPartition Reload1

The lower version of the LWIP protocol does not support packet sharding and reinstallation of packets, while the higher version of the LWIP protocol adopts a different method from the standard protocol for the reinstallation of sharded packets. It uses a simpler data structure to assemble the sharded package, but the result is that the assembly process is obscure and the code execution is inefficient. Personally, I don't think this implementation mechanism of LWIP is very good, and if I have time, I would like to re-implement the entire packet sharding and reinstall mechanism myself, and it is better to specify it.

It should be noted that in general embedded products, the amount of data is relatively small, and there is basically no data fragmentation, and embedded products will not act as routers in the network to realize packet reorganization, forwarding and other functions. As a result, lower versions of LWIPs can still perform their functions in most applications.

First, let's take a look at how BSD in the standard protocol realizes packet reorganization, and then compare the LWIP reinstallation mechanism with this mechanism.



There are two main data structures used in the standard to implement packet reload, ipq and ipasfrag. The ipq is the node information data structure established for a packet to be reorganized, including the target IP address, packet number, etc., and the structure also has pointers ipf_next and ipf_prev to make the data of each shard of the packet form a bidirectional linked list, and ipasfrag is used to wrap each shard packet in the figure. At the same time, all IPQ structures are sufficient to form a bidirectional linked list to facilitate the search, insertion, and deletion operations of an IPQ structure. The linked list that the entire IPQ consists of has a fixed linked header that does not exist

Store any data packet data for identification only.

In LWIP, struct `ip_reassdata` plays a similar role to `ipq` above, but the structure members are very different, `ip_reassdata` is used to construct a one-way linked list to perform operations such as finding and deleting a `ip_reassdata` structure. All `ip_reassdata` structures make up one-way linked lists have a header node pointer, known as `reassdatagrams`, that point to the linked list's header. At the same time, LWIP does not have an `ipfrag`-like structure to encapsulate each received shard packet, but directly hooks the shard packet after the corresponding `ip_reassdata` structure. As for how to hook up, that's a later story.

Now let's take a look at the legendary `ip_reassdata` structure, source code:

```
struct ip_reassdata {  
    struct ip_reassdata *next;  A pointer used to build a one-way linked list  
    struct pbuf *p;             The datagram's data link list  
    struct ip_hdr iphdr;        The IP header of the datagram  
    u16_t datagram_len;         The length of the datasheet that has been received  
    u8_t flags;                 whether the last shard packet was  
    received u8_t timer;        Set the timeout interval};
```

`ip_reassdata` is mainly used to describe a datagram being assembled, and the function that completes the assembly of the packet is called `ip_reass`, which uses the IP sharded packet as the input parameter to output the assembled packet pointer or empty pointer, here we will take a look at its operation process.

First, determine the head size of the IP packet, currently LWIP does not support IP packets with IP options, so the IP packet header size can only be 20 bytes, if it is greater than this value, `ip_reass` directly discard the packet and return.

Then, since LWIP has an upper limit on the sum of the number of `pbufs` connected by all `ip_reassdata` structures, that is, `IP_REASS_MAX_PBUFS`, `ip_reass` checks the number of `pbufs` occupied by the current packet shard, assuming that if so many `pbufs` are connected to a certain `ip_reassdata`, whether the number of `pbufs` used is greater than the upper limit we just mentioned, and if so, LWIP It is not allowed to connect `pbufs` to `ip_reassdata` in such cases. At this point, the user has two options: either to delete the packet directly and return, or to delete the longest-lived `ip_reassdata` struct in the `ip_reassdata` linked list and its associated `pbufs` until there are enough `pbufs` available. This choice is achieved through macro definitions `IP_REASS_FREE_OLDEST`. Hey, this paragraph is too messy! Why define a maximum number of `pbufs` used, I don't understand!

At this point, you can insert the sharded datagram, the first thing to do is to find the corresponding `ip_reassdata` struct in the linked list `reassdatagrams`, at this time, there are two situations: no matching struct is found, that is, the shard is the first shard to arrive, at this time you need to create a new `ip_reassdata` struct and insert it into the linked list `reassdatagrams`; Find a matching struct, that is, a shard has arrived before the shard arrives, at this time to determine whether the shard is the first shard of the entire packet, and if so, record the IP packet header with the `iphdr` field of the corresponding `ip_reassdata` struct.

At this point, we must find the `ip_reassdata` corresponding to the sharded packet, and then we can determine whether the shard is the last shard packet of the entire packet, because the `IP_MF` bit of the last shard packet is 0. If it is the last sharded package, set the flag `flag` and `datagram_len` value for the corresponding `ip_reassdata` to indicate that the last shard was received and set the length value of the entire packet. PS: The first shard package to arrive is not necessarily the first shard package, and the last packet to arrive is not necessarily the last shard package.

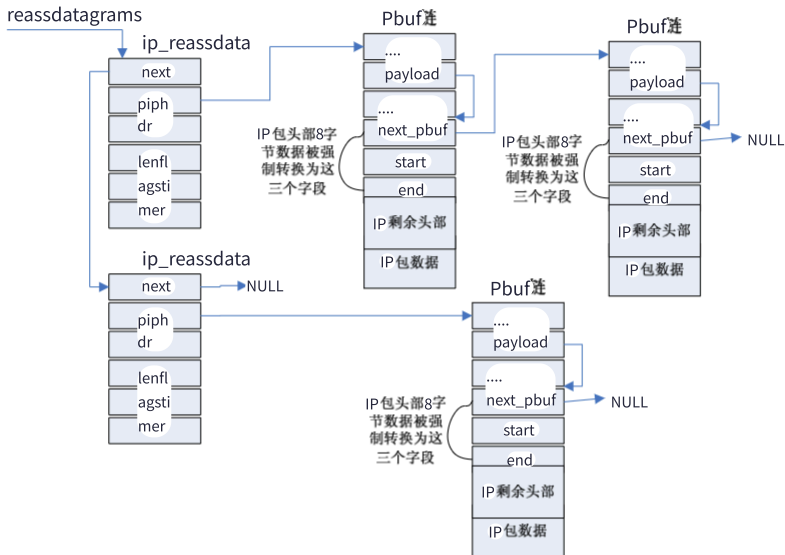
Finally, call the function `ip_reass_chain_frag_into_datagram_and_validate` to insert the sharded packet, which is a function that we will focus on in the future. The above function also detects whether a packet is assembled, and if a packet is assembled, then `ip_reass` also does the following:

Find the first shard packet according to the `ip_reassdata` structure, copy the IP header field `iphdr` in the `ip_reassdata` structure to the first shard packet header (the header data of the first shard packet header has been overwritten, in the discussion function).

`ip_reass_chain_frag_into_datagram_and_validate` we will talk about it later), and recalculate the checksum. At the same time, the header information is removed from each shard packet after the first shard, and the entire packet is restored, and then the `ip_reassdata` structure is removed from the linked list, and the reorganized packet pointer is returned as a return parameter.

12 IP ShardingReinstallPartitioningReinstallPartitioningReinstall2

There is also a question left over from the previous section about how IP sharding packages are inserted into the corresponding assembly linked list. Here's an intuitive graph to explain the process.



The diagram shows the process of two packets being assembled, and the two `ip_reassdata` structures are used for the assembly of the two packets. The first packet already has two shards assembled, and the second has one sharded data assembled. LWIP does not assign a separate data structure to describe each shard data as described in the standard protocol, but directly uses each shard data to store the first eight bytes of the IP header in the data to store information about the assembly process, so that each incoming IP shard packet header is changed, so the `iphdr` field in the `ip_reassdata` structure is used to temporarily save the full header of the IP packet.

What value is these eight bytes turned into, this is the content of struct `ip_reass_helper`. This structure is the most important structure in the assembly process. Look, this is probably the simplest structure.

```
struct ip_reass_helper
{
    struct pbuf *next_pbuf;

    u16_t start;
    u16_t end;
};
```

`next_pbuf` fields are used to connect individual sharded packets during assembly; The `start` field is used to record the starting position of the sharded packet data in the entire IP packet. Similarly, the `end` field indicates the end position in the IP packet.

The function `ip_reass_chain_frag_into_datagram_and_validate` (PS: God, this function name is too long) is used to insert an IP sharded packet `pbuf` into a `ip_reassdata` structure, and after insertion, it checks whether the IP packet is assembled, if it is not assembled, it returns 0, otherwise it returns a non-0 value. Obviously, this function has two input parameters, `ip_reassdata`

Structure and sharding package pbuf. Let's take a closer look at what the function with the longest name does.

First, it extracts information from the header of the IP shard packet, including the start offset address of the sharded data offset and the length len of the shard data, and then it casts the first eight bytes of the head of the IP shard packet into a `ip_reass_helper` structure and sets the value of the start field to offset. end field is set to offset+len; next_pbuf field has a value set to NULL. At this point, the incoming shard package has been reshaped, and its IP header has been destroyed, so the remodeled shard package will be inserted into the operation. The insertion operation is mainly to compare the values of the start and end fields of each `ip_reass_helper` to determine where the shard package is inserted in the linked list, the search for the insertion position is the most difficult part of the whole assembly process, but in principle it is a very simple process, here we will no longer parse the source code of the insertion location and insertion operation.

At this point, the sharded packet has been inserted into the linked list of the corresponding `ip_reassdata` structure, and the function with the longest name needs to check whether the packet corresponding to the `ip_reassdata` has been assembled. As mentioned in the previous section, when the received IP shard packet is the last shard of an IP packet, the function `ip_reass` will set the flags field of the corresponding `ip_reassdata` structure to 1, so if the flags field of a `ip_reassdata` structure is still 0, it means that the last shard has not been received, IP The packet assembly is definitely not complete yet, and the function with the longest function name can return 0 directly. In the second step, if the flags field is set to 1, it means that the last shard packet has been received, but it is still unknown whether the entire IP has been assembled, because on the network, the shard packets do not arrive in order every time, so the last shard packet received is not necessarily the last shard packet. You need to iterate through the linked list of individual shard packets behind the `ip_reassdata` structure to detect if there are any shard packets that have not been received.

At this point, the `ip_reass_chain_frag_into_datagram_and_validate` function has done its job, it inserts the sharded data into a `ip_reassdata`-structured data shard linked list, and detects whether the IP packet is assembled, returns a non-0 value when assembled, otherwise returns a 0 value.

Here we go back to the `ip_reass` function, `ip_reass` insert a sharded packet into the corresponding `ip_reassdata` structure through a function call, and then use the return value of the function to determine what to do next. If the packet assembly is not complete, the `ip_reass` function needs to return an empty pointer to the function calling it, and if the packet assembly is complete, it needs to return the assembled packet to the function that called it. As you can see from the diagram above, the assembled packets are all shards mounted on a `ip_reassdata` structure, and the `ip_reass` function needs to remove the corresponding shard data from this `ip_reassdata` structure, remove unnecessary information from each shard, and then return the entire packet to the caller. To accomplish this task, the `ip_reass` function does the following.

First, the values of the `iphdr` field in the `ip_reassdata` structure are corrected accordingly, such as correcting the total packet length and checksum, and then copying all the `iphdr` fields into the IP header field of the first shard packet, so that the head of the entire IP packet appears in the first shard packet, and then starting from the second shard packet, delete their IP header information, so that a complete IP packet is reassembled. Finally, call the `ip_reass_dequeue_datagram` function to delete the `ip_reassdata` struct used in the packet assembly process, i.e., to remove the node corresponding to the `reassdata` from the `reassdatagrams` linked list described in the figure above. Okay, the merit is complete!

This circle is a bit big, originally talking about `ip_input` functions, but it turned out to talk about `ip_reass` functions, and later `ip_reass_chain_frag_into_datagram_and_validate` functions. No way, who made the last two functions serve the `ip_input` functions? `ip_input` functions use `ip_reass` assembled packets to be delivered to upper-layer TCP or UDP applications, as mentioned earlier.

Now we still have to go back to the original road and take the path of `ip_input`. The basic process of `ip_input` has been clearly explained earlier, and there is a function to serve it, that is, the `ip_forward` function, which is mainly to complete the forwarding of data packets, when the data received by the device is not for itself, it can choose to forward the data packet, originally there is no need to talk about the `ip_forward` function here, because in general applications, this function will be prohibited, when the device receives a packet that is not for itself, it will be in the `ip_input` function is discarded at the beginning. But so far, we haven't touched on anything

Regarding the content of IP packet transmission, I thought that the `ip_forward` function should be explained, because the forwarding of data packets and the sending of packets are exactly the same principle, and the exact same interface function is used, so explaining the `ip_forward` function is equivalent to explaining all the details of the IP layer packet transmission. In the TCP layer or UDP layer, the transmission of data packets is inevitably involved, here we will use the `ip_forward` function to explain the whole process of sending IP layer packets clearly, so that the logic is clear and easy to understand!

When an IP packet is received, LWIP will traverse the IP addresses of all network interfaces to determine whether the packet is for itself, and if not, it will call the network interface that received the packet to forward the packet. But don't panic, before forwarding, you have to check whether this package is a broadcast package, if so, just discard it and don't deal with it. Now let's take a look at what the packet forwarding function `ip_forward` does, this function has three input parameters: the packet pointer to be forwarded, the IP header pointer of the packet to be forwarded, and the netif pointer to the network interface data structure that receives the packet.

First, call the `ip_route` function to find the network interface that should be used to forward the packet, and the `ip_route` function to find the relevant structure that should be used based on the destination address in the packet IP header. If you cannot find an interface that meets the requirements, select the default network interface. The `ip_route` function is now stopped here, and after talking about the `ip_forward` function, I will explain it in detail.

`ip_forward` Check whether the network interface found by the `ip_route` function is valid, that is, it cannot be empty or the interface that received the IP packet. When the network interface is determined to be invalid, the packet is not forwarded. When a packet can be forwarded using a network interface, `ip_forward` first subtract the value of the TTL field in the IP header by 1, and if the TTL becomes 0, it needs to send a timeout ICMP message to the source host, indicating that the lifetime of the current packet has expired, and the packet is discarded here and will not be forwarded. As for how to send this timed out ICMP packet, this involves the send function of the IP layer packet `ip_output`, which we will talk about later.

The function then recalculates the header checksum because the value of the header TTL field has been modified, and finally calls the output function registered with the netif structure, which assembles the packets into an Ethernet data frame and sends them out. As mentioned earlier, this function is `etharp_output`.

At this point, the work of the `ip_forward` function is complete, and there are two questions left: the `ip_route` function and how to send a timeout ICMP message packet. The first question is explained here, and the second question is placed in the ICMP section. The `ip_route` function takes the target IP address as the input parameter, and then finds the network interface on the same subnet as the IP address on the network interface structure link list `netif_list`. It's so simple, not much to say.

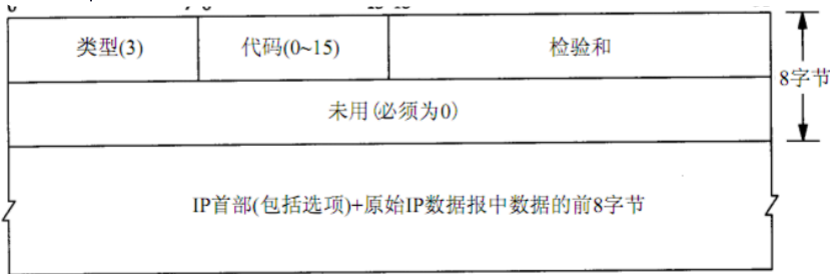
13 ICMP processing processing

At present, the IP layer is basically explained, and the sending or sharding of data packets is not specifically involved. The transmission of data packets is closely related to the upper layer protocol, that is, the transport layer, and the rest of the content is to discuss the transport layer. Let's first explain the simpler ICMP protocol among the transport layer protocols. ICMP (Internet Control Message Protocol) is an Internet control message protocol used to transmit control messages between IP hosts and routers. Control messages refer to messages about the network itself, such as whether the network is unreachable, whether the host is reachable, and whether the route is available. Although these control messages do not transmit user data, they play an important role in the transmission of user data.

The first time in the packet forwarding process, the TTL value of the packet needs to be subtracted by 1, and if the TTL value becomes 0 at this time, the icmp_time_exceeded function is used to return a timeout ICMP message to the source host.ip_input The ip_input function also determines which parent protocol the packet is handed over to through the protocol field value in the IP packet header, and calls the icmp_input function if it is an ICMP protocol. If no protocol can accept this packet, call the icmp_dest_unreach function to return a protocol-unreachable ICMP error control packet to the source host. Let's first explain how the icmp_time_exceeded and icmp_dest_unreach functions send ICMP packets.

Let's take a look at the format of ICMP packets, the first four bytes of all ICMP packets are the same, which are 1 byte of type field, 1 byte of code field, and 2 bytes of checksum field. The calculation of the checksum field covers the entire ICMP packet. The type and code fields uniquely determine which type the ICMP packet belongs to: echoes, timeouts, timeout requests, etc., although the structure of ICMP messages is usually not the same, they start with the same four bytes. ICMP packets can be broadly divided into ICMP query packets and ICMP error packets. ICMP query packets include ICMP echo reply, echo request, time kill request, address mask request, etc., and LWIP only implements ICMP echo reply. ICMP error messages are of the following types: unreachable, timeout, redirect, etc., but LWIP only realizes two ICMP processing functions: unreachable and timeout. Here we will first explain the two types of ICMP processing, namely unattainable and timeout.

Both ICMP messages are ICMP error packets, namely unreachable and timeout, and the protocol stipulates that ICMP error packets should always contain the IP header and the first 8 bytes of data of the IP datagram that generated the ICMP error packet. Therefore, both ICMP packets have the following message structure, which is exactly the same as the previous description and will not be explained.



Let's talk about the purpose of the icmp_dest_unreach function and the non-error message. The type field value of the packet should be 3, and the code field value should be (0~15), and LWIP only supports (0~5), which means that the network is unreachable, the host is unreachable, the protocol is unreachable, the port is unreachable, the sharding is required but the shardless slot is set, and the origin server selection fails. Obviously, when the ip_input function cannot find which parent protocol to send the packet to, it should generate an error report that the protocol is unreachable

text, that is, the code field value is 2. The function prototype is as follows, with the input parameters of the received destination unreachable IP packets and the values that should be used to populate the ICMP header code fields.

```
void icmp_dest_unreach(struct pbuf *p, enum icmp_dur_type t)
```

Now let's take a look at what this function does, first of all, apply for a pbuf buffer for the ICMP packets to be sent, the length of this buffer is the sum of the length of the structure described in the above figure and the size of an IP data header, the reason why you need to apply for more space for the IP data header size is that when the ICMP packet is submitted to the IP layer for sending, the IP layer does not need to apply for a data header to encapsulate the ICMP packet, but directly fills in the IP header that has already been requested Header data, note that the payload pointer of the pbuf applied here points to the ICMP data header. Next, the function fills in the relevant fields of the ICMP packet, fills in the type segment as 3, and the code segment as the input parameter t, copies the IP header and the first 8 bytes of the data of the unreachable IP packet to the corresponding field of the ICMP packet, and finally calculates the value of the checksum field, and then calls the ip_output function to send the assembled ICMP packet out.

The ip_output function completes the sending of the packet by calling the ip_output_if function, and before calling the ip_output_if function, it must first find the corresponding sending network interface structure according to the destination IP address of the sending packet, and use this structure as a parameter for calling the ip_output_if function. The ip_output_if function mainly fills in the values of each field in the IP header, and then calls the netif->output function to send the encapsulated IP packets.

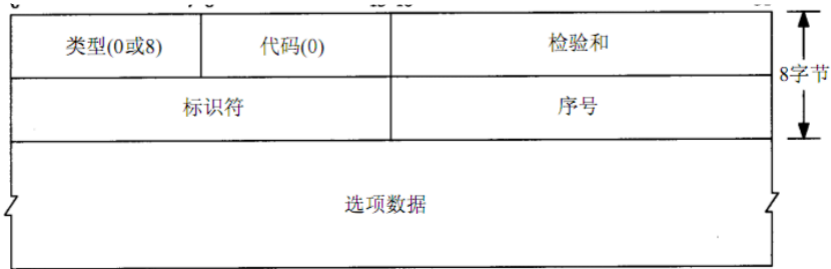
The icmp_time_exceeded function is used to generate an ICMP packet of a timeout type. This type of packet has the exact same message structure as the one described above, but the type field value should be 11 and the code field should be 0 or 1, representing the timeout of timeout during transmission and timeout during data assembly, respectively. For the latter, when discussing packet reorganization, we know that there is a time field in every ip_reassdata struct, and when the specified time arrives and the packet has not yet been assembled, the kernel deletes all sharded data related to that ip_reassdata structure and sends a timeout ICMP packet with a code field of 1 to the source host.

From the perspective of code flow and content, the icmp_time_exceeded function is exactly the same as the icmp_dest_unreach function, except that different values are used in the type and code fields of the ICMP packet, which will not be repeated here.

Now it's time to talk about ICMP query packets, which play a very important role in the overall product design and debugging process. That is, the ping command, which is closely related to ICMP echo answers and request packets.

This small paragraph comes from the protocol: the name "Ping" comes from the sonar positioning operation. The purpose is to test whether another host is reachable. The program sends an ICMP echo request message to the host and waits for an ICMP echo reply to be returned. In general, if you can't ping a host, you can't Telnet or FTP to that host. Conversely, if you can't telnet to a host, you can usually use a ping program to determine what the problem is. The ping program can also measure the round-trip time to the host to indicate how far it is from us. At the same time, the Ping program can also record the route path during the routing process of the packet.

The format of the ICMP echo request and echo reply packet is as shown in the figure below, the type value of the echo reply is 0, the type value of the echo request is 8, and the value of both code fields is 0. This allows the ping program to recognize the returned information even if multiple ping instances are running on the same host at the same time. The serial number starts at 0 and adds 1 for each new echo request sent.



`icmp_input` function processes the received ICMP packets and processes them accordingly according to the packet type. Currently, LWIP can only handle ICMP echo request packets and does not respond to other types of ICMP packets, which is sufficient in embedded products. For ICMP echo requests, `icmp_input` needs to generate an echo reply packet to return to the source host.

Let's take a look at what the `icmp_input` function does. First, adjust the payload pointer of the incoming packet `pbuf` to point to the ICMP header, and determine whether the ICMP header length is less than 4 bytes. For the correct ICMP package, the function will determine what to do based on the value of its header type field. The current version only implements response operations related to ICMP echo requests.

If the current packet is an ICMP echo request, the function continues to determine whether the packet is broadcast or multicast, and does not process these two packets. Next, determine whether the packet size is less than the length of the ICMP echo request header (as shown in the figure above), and then discard the packet. Next, the function applies for the IP header and Ethernet header memory space for this packet, and after success, the ICMP packet type field is changed to 0, the checksum is calculated from a new one, and the source IP address and the destination IP address of the IP header are exchanged, and finally the entire packet is sent out using the `ip_output_if` function. The ICMP echo response returns the data in the echo request to the source host as it is, and after receiving the echo response, the source host can obtain relevant information by processing the data in the echo response, such as calculating the round-trip time.

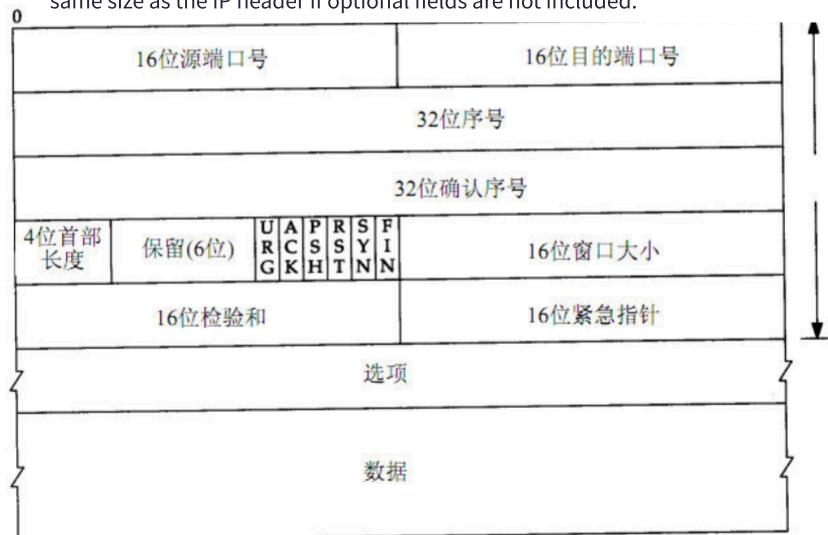
That's all ICMP is about.

14 TCP Establish and Disconnect, Establish and Disconnect, Establish and Disconnect, Establish and Disconnect

The TCP part is the largest and most difficult part of the entire LWIP to understand, with code accounting for nearly half of the entire stack. Seeing such a big project, I really don't know where to start to explain it clearly. Depressed, write what you see first, and then slowly sort it out after writing. But the basic thread I want to refer to is still the TCP part of the standard protocol.

TCP is called Transmission Control Protocol, which provides a connection-oriented, reliable byte streaming service for the upper layer. TCP provides reliability through a series of mechanisms: application data is split into chunks of data that TCP deems best for sending; When TCP sends a segment, it starts a timer and waits for the destination to confirm the receipt of this packet segment. When TCP receives data from the other end of the TCP connection, it will send an acknowledgment that is not sent immediately and will usually be delayed by a fraction of a second; TCP will keep its first part and the data check sum, if there is an error in the received segment, TCP will discard the packet segment and not send an acknowledgment to make the sender time out and resend it; If necessary, TCP will reorder the received data and give the received data to the application layer in the correct order; The IP datagram will be duplicated, and the receiver of TCP must discard the duplicate data. The TCP also provides flow control.

The following figure shows the TCP header structure, which is 20 bytes in size, which is the same size as the IP header if optional fields are not included.



Source and destination port numbers are used to identify the application processes on the sender and receiver sides. These two values, together with the source IP address and destination IP address in the IP header, uniquely determine a TCP connection. An IP address and a port number are also called sockets. The 32-bit sequence number field identifies the flow of data bytes from the TCP sender to the TCP receiver, and is used to identify the sequence number of the first data byte in the message segment. When a new connection is established, the SYN flag is set to 1, and the sequence number field contains the initial sequence number ISN(Initial Sequence Number) on the connection selected by this sending host. The first byte serial number for which the host wants to send data is ISN+1.

The 32-digit confirmation number is only valid if the ACK flag is 1, and it contains the next sequence number that the sending acknowledgment is expected to receive. Therefore, the confirmation sequence number should be the last time the data byte sequence number was successfully received plus 1. When a TCP connection is established correctly, the ACK field is always set to 1.

The 4 heads give the number of 32-bit words in the first part. This value is required because the length of the optional field is variable. Since this field is 4 bits, TCP has a maximum of 6 0-byte heads. However, without optional fields, the normal length is 20 bytes.

There are 6 logo bits in the TCP head. Multiple of them can be set to 1 at the same time. Here is a brief introduction to their use, which will be explained in detail when used in the future: URG urgent pointer effective identification; ACK confirms the valid identification of the serial number; The PSH receiver should hand over this packet segment to the application layer as soon as possible; RST rebuilds connections; SYN synchronization serial number, which is used to initiate a connection; The FIN originator completes the sending task.

The 16-bit window size field enables flow control by declaring its own window size, which represents the number of bytes it can still receive.

16-bit checks and covers the entire TCP message segment: TCP header and TCP data. This is a mandatory field that must be calculated and stored by the sender and verified by the receiver. The TCP header used by TCP inspection does not include all the fields in the actual header, but rather a pseudo-header, which can be found in the UDP section or in the protocol.

The 16-bit emergency pointer is a positive offset that is added to the value in the sequence number field to represent the sequence number of the last byte of the emergency data, and is only valid if the URG flag is set to 1. The emergency method of TCP is a way for the sender to send urgent data to the other end.

The most common optional field is the longest packet size, also known as MSS (Maximum Segment Size). Each connector typically indicates this option in the first message segment of the communication (the segment where the SYN flag is placed to establish the connection). It indicates the maximum length of message segments that can be received by the local end.

The portion of the data in a TCP segment is sometimes empty. For example, when a connection is established and a connection is terminated, the message segments exchanged between the two parties are only the TCP head; If one party has no data to send, it needs to use the first part without any data to confirm the data received; In many cases where the timeout is handled, a packet segment without any data is also sent.

Let's take a look at how LWIP describes such a TCP header, the data structure `tcp_hdr` is as follows, which is exactly the same as described in the figure above, and will not be explained.

```
PACK_STRUCT_BEGIN
struct tcp_hdr {
    PACK_STRUCT_FIELD(u16_t src);    Source port
    PACK_STRUCT_FIELD(u16_t hand);   Destination port
    PACK_STRUCT_FIELD(u32_t seqno);  Serial number
    PACK_STRUCT_FIELD(u32_t ackno);  Confirm serial number
    PACK_STRUCT_FIELD(u16_t _hdrlen_rsvd_flags); Head length + reserved position + sign bit
    PACK_STRUCT_FIELD(u16_t wnd);    window size
    PACK_STRUCT_FIELD(u16_t chksum);  checksum
    PACK_STRUCT_FIELD(u16_t urgptr); Emergency pointer
} PACK_STRUCT_STRUCT;
PACK_STRUCT_END
```

Hey, I'm at a loss again, I don't know what to write! TCP is really hard to chew. Let's put aside the content of the LWIP and carefully explain the content of the detailed explanation paper. Let's start with a TCP connection establishment process. TCP establishing a connection requires three message segments to interact, so it is also called the triple handshake process.

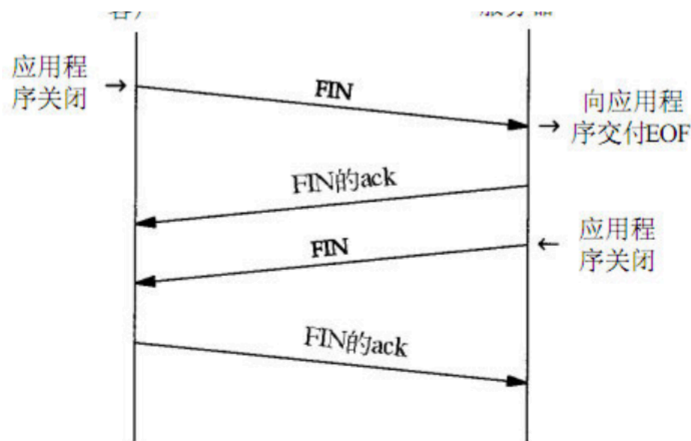
First, the requester (usually called the customer) sends a TCP datagram with the SYN flag set to 1, which indicates its port number and the port number of the server to be connected, and announces its initial serial number ISN.

When the server receives the packet and parses it, it also sends back a SYN packet segment as an answer. The response packet contains the initial serial number ISN selected by the server itself, and at the same time, the ACK is set to 1, and the confirmation serial number is set to the ISN of the requesting side plus 1 to confirm the customer's SYN packet segment. The ISN here also indicates the sequence number of the next byte that the server wants to receive. This shows that a SYN will take up a sequence number.

Finally, when the requester receives the server's SYN reply packet, it will generate a handshake packet again, in which the ACK flag is placed, and the confirmation serial number is set to the ISN sent by the server plus 1, so as to confirm the server's SYN packet segment.

Usually there is a situation where both ends initiate a connection at the same time, i.e. send the first SYN packet at the same time, and both ends are actively open, which we will cover in a later discussion.

The disconnection of the TCP connection requires four handshake processes. A TCP connection is full-duplex (i.e., data can be passed in both directions at the same time), so each direction must be closed separately. When the sending party completes its data sending task, it can send a handshake packet with a FIN flag set to 1 to terminate the connection in this direction. When the other end receives this FIN packet, it must notify the other end of the application layer that it has terminated data delivery in that direction. Sending a FIN is usually the result of the application layer shutting down, and receiving a FIN means that there is no data flow in this direction. A TCP connection can still send data after receiving a FIN, at which point the connection is semi-closed. Normally the party that closes first (i.e., sends the first FIN) performs an active closing, while the other party (which receives this FIN) performs a passive closing. Usually one party completes the active closure and the other party completes the passive closure, but there are also cases where both parties are active closed, which will be discussed later. The four-time handshake process to disconnect a connection is shown in the following figure, where the client sends a FIN handshake package to the server to shut down the data transfer from the client to the server when the client application actively performs a shutdown operation. When the server receives this FIN, it sends back an ACK confirming that the sequence number is the sequence number received plus 1. As with SYN, a FIN will occupy a sequence number. At the same time, the server also notifies its parent application that it has received the end action, and then the server program will close its connection, causing its TCP side to send a FIN handshake packet, and the customer must send back a confirmation and set the confirmation serial number to receive the serial number plus 1.

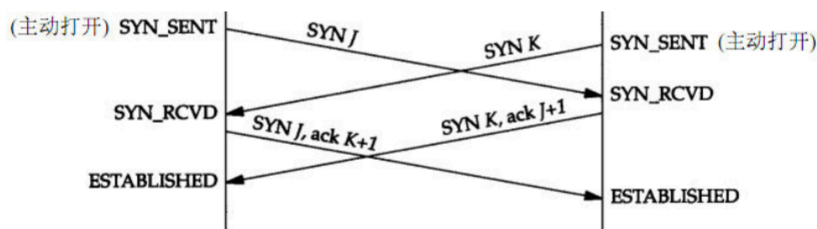


In this diagram, sending FINs will cause applications to close their connections, and the ACKs for these FINs are automatically generated by the TCP software. The active initiator of the connection is usually the client, but if the server initiates the FIN package first, the handshake process in the diagram above is also fully established.

When the FIN packet is sent, the client enters the `FIN_WAIT_1` state and waits for the server to return the confirmation packet, when this confirmation packet is received, it indicates that the connection from the client to the server direction is successfully disconnected, at this time the client enters the `FIN_WAIT_2` state and waits for the connection from the server to the client direction to be disconnected, at this time, when the client receives the FIN packet from the server, it returns an ACK packet to the server, indicating that the connection from the server to the client direction is successfully disconnected. The client then enters the `TIME_WAIT` state, where it waits for 2MSL before entering the initial `CLOSED` state. While the connection is waiting for 2MSL, any late packet segments are dropped. This process is exactly the same as the disconnected four-handshake process.

Another typical state transition path describes the server, as shown by the dotted line in the image. The server establishes a connection as a passive process that first opens a port and enters the `LISTEN` state to listen for the client's connection request. When the server receives a SYN connection request from the client, it enters the `SYN_RECV` state and returns an ACK and its own SYN packet to the client. It can be seen that the process of connection establishment is exactly the same as the triple handshake process described earlier. When the server receives a disconnected packet FIN sent by the client, it enters the `CLOSE_WAIT` state, announces this message to the layer-level application, and returns an ACK packet to the client, at which point the connection from the client to the server is successfully disconnected. After that, when the server upper-layer application processes the relevant information, it will send a FIN packet to the client and enter the `LACK_ACK` state, waiting for the client to return an ACK packet. This process is exactly the same as the disconnected four-handshake process.

There are also two more special transition routes in the above diagram that are used to handle the simultaneous initiation or disconnection of both ends of TCP. The handshake package interaction between two hosts performing the opening operation at the same time is shown in the following figure, and the two hosts are simultaneously



After sending its own SYN request packet, each enters the `SYN_SENT` state, waiting for the other party's ACK packet to return, but after a certain period of time, each host receives the other party's SYN packet instead of the ACK packet, at this time, both ends determine that they have encountered a simultaneous opening situation, both ends enter the `SYN_RECV` state, and confirm the other party's SYN and send their own SYN packet again, when receiving the other party's ACK+SYN, both sides enter the `ESTABLISHED` state. From this point of view, there should be two conditions for the transition from `SYN_RECV` to `ESTABLISHED` in the state transition diagram, and only one is marked in the diagram. A simultaneously open connection requires 4 message segments to be exchanged, one more than a normal triple handshake. Note that during such a connection, neither end is referred to as a client or server, as each end is both a client and a server.

The handshake package interaction in which two hosts initiate an active shutdown operation at the same time is as follows, when the user program of both hosts is executed at the same time

16 TCP Control Block Control Block Control Block Control Block

This section officially steps through the TCP portion of the LWIP protocol. Let's take a look at how it describes a TCP connection. This structure is very complex, and the simple description here is not comprehensive, and it does not clearly explain the role of each field. The source code for struct tcp_pcb is as follows: struct tcp_pcb {

```
    IP_PCB;      This is a macro that describes information about the IP of the connection, including the IP addresses of both parties, TTL, etc
    struct tcp_pcb *next; //用于连接各个 TCP 控制块的链表指针
    enum tcp_state state; //TCP 连接的状态, 即为状态图中描述的那些状态
    u8_t prio; //该控制块的优先级
    void *callback_arg; //u16_t local_port; //本地端口
    u16_t remote_port; //远程端口
    u8_t flags; // 附加状态信息, 如连接是快速恢复、一个被延迟的 ACK 是否被发送等
#define TF_ACK_DELAY(u8_t)0x01U /* Delayed ACK. */
#define TF_ACK_NOW(u8_t)0x02U /* Immediate ACK. */
#define TF_INFR(u8_t)0x04U /* In fast recovery. */
#define TF_RESET(u8_t)0x08U /* Connection was reset. */
#define TF_CLOSED(u8_t)0x10U /* Connection was successfully closed. */
#define TF_GOT_FIN(u8_t)0x20U /* Connection was closed by the remote end. */
#define TF_NODELAY(u8_t)0x40U /* Disable Nagle algorithm */
    /// 接收相关字段
```

```
    u32_t rcv_nxt;      The next byte that is expected to be received, which is the sequence number of its ACK to the sender
    u16_t rcv_wnd;      Receive window
    u16_t rcv_ann_wnd;  Announcement window size, this field is not available in the lower version
```

```
    u32_t tmr;          This field records the moment the PCB was created
    u8_t polltmr;       pollinterval; Three timers, follow-up explanation
```

```
    u16_t rtime;       The value increases over time and occurs when it is greater than the value of RTO
```

```
    u16_t mss;         Maximum data segment size
```

RTT estimates relevant parameters

```
    u32_t rttest;      Estimated number of 500ms ticks
    u32_t rtseq;       The serial number of the package used to test RTT
    s16_t with, St.    The mean and its time difference estimated by RTT
    u16_t rto;         The resend timeout is calculated using the previous values
    u8_t nrtx;         The number of times the packet is retransmitted, which is used when the packet times out multiple times, is related to the value of the set RTO
```

Parameters related to fast retransmission/recovery u32_t lastack; The largest confirmation serial number, this field does not understand u8_t dupacks; The number of times the serial number above has been retransmitted

Blocking control related parameters u16_t cwnd; The current blocking window of the connection u16_t ssthresh; Slow start threshold

Send the relevant

field u32_t snd_nxt, The next byte sequence number to be sent

snd_max, The highest sent byte sequence number

snd_wnd, Send window

snd_wl1, snd_wl2, // The data serial number and confirmation serial number snd_lbb the last window update; The sequence number of the last byte in the sending queue u16_t acked; //

u16_t snd_buf; The number of buffer bytes available

u8_t snd_queuelen; The number of packets that are available

struct tcp_seg *unsent; A queue of unsent data segments

struct tcp_seg *unacked; A data queue that has not received an acknowledgment has been sent

struct tcp_seg Received packet queues outside of the sequence

*ooseq;

#if LWIP_CALLBACK_API Callback functions, some of which are not defined in later versions

err_t (* sent)(void *arg, struct tcp_pcb *pcb, u16_t space); err_t (* recv)(void *arg, struct tcp_pcb *pcb, struct pbuf *p, err_t err); // 数据包接收回调

函数err_t (* connected)(void *arg, struct tcp_pcb *pcb, err_t err); err_t (* accept)(void *arg, struct tcp_pcb *newpcb, err_t err); err_t (* poll)(void *arg, struct tcp_pcb *pcb); void (* errf)(void *arg, err_t err);#endif /* LWIP_CALLBACK_API */

All remaining fields are not defined in later versions, and will be explained when needed

u32_t keep_idle;#if

LWIP_TCP_KEEPALIVE

u32_t keep_intvl; A keep-alive timer to detect if the other end of an idle connection is crashed

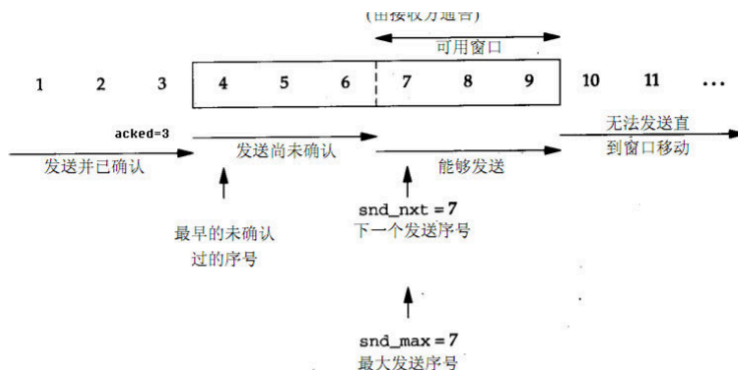
u32_t keep_cnt;#endif /*

LWIP_TCP_KEEPALIVE */

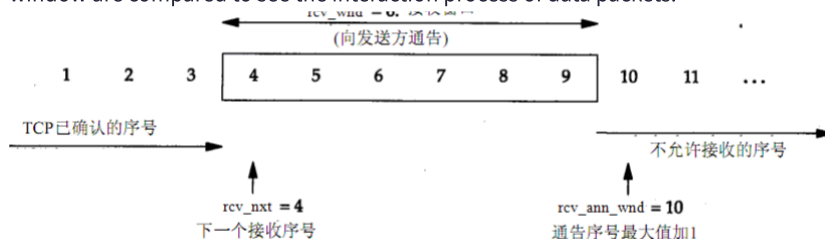
u32_t persist_cnt; These two fields keep window size information flowing

u8_t persist_backoff; u8_t keep_cnt_sent; }; Let's start with the fields related to receiving data rcv_nxt, rcv_wnd, rcv_ann_wnd and the fields related to data sending snd_nxt, snd_max, snd_wnd, and acked. These fields are closely related to the famous sliding window protocol in TCP

System. As shown in the figure below, both sides of the connection maintain a window for data transmission. The sliding window divides the entire sequence into three parts: the left is the group that has been sent and confirmed, the right side of the window is the group that has not yet been sent, the inside of the window is the group to be confirmed, and the inside of the window is divided into those that have been sent to be confirmed, and those that have not been sent but will be sent immediately. TCP ensures reliability through front-facing acknowledgment and retransmission technology, and the sliding window allows the sender to send the next packet before receiving the confirmation message of the previous packet, which improves the bandwidth utilization of the network.



In addition to the send window, both sides of the TCP connection also maintain a receive window, as shown in the figure below, the receiver's receiving window and the sender's sending window are compared to see the interaction process of data packets.



On the receiver, `rev_wnd` represents the size of its receiving window, and it can advertise its window size value in the ACK package to the sender, and after the sender receives the value, it sets its own sending window size value `snd_wnd`. The data sending sequence contained in the sender's send window is closely related to the ACK sequence, that is, it includes the `snd_wnd` bytes after the ACK sequence number in the window. The sender's `acked` field represents the highest ACK serial number received, `snd_nxt` represents the serial number of the data that the sender is about to send, the data between `acked` and `snd_nxt` indicates that the ACK has been sent but has not yet been received, and the sender must also include them in the slider window to facilitate timeout retransmission, and `snd_nxt` to the end of the send window indicates the data that has not yet been sent. `snd_max` expressed confusion, MARKKKKK for a moment, and I'll take a look later. On the receiver, the receiver receives the numbered data in the sliding window, and when all the serial numbers before a serial number have been received, the receiver can ACK the serial number and slide the receiving window back. When the sender receives the ACK, it also slides its sending window backwards. On the receiving side, `re_nxt` indicates the next byte sequence number that you want to receive, and `rev_ann_wnd` indicates the window size of the other party's announcement, which also indicates puzzlement. messy, messy... Also related to sending is the `cwnd` field, which touches on the concept of slow start. When the sender receives the receiver's window notification, it will not send all the data allowed in the window at once, because this may cause unstable network throughput and low bandwidth utilization due to intermediate router forwarding congestion and other reasons. The sender's approach is to save a congestion window with the `cwnd` field, and the sender takes the minimum value of the congestion window and the advertisement window as the send

The initial value of the congestion window is generally 1, and a value is added after each ACK received from the receiver. Slow start will be explained in the follow-up content.

17 TCP Creation Process Creation Process Establishment Process Creation Process Creation Process

There are a lot of ethereal things mentioned earlier, and most of them are borrowed from the content of the standard protocol, which makes people a little dizzy! In this section, we'll look at how to implement an http server on our LWIP, combined with the connection establishment process to understand the significance of the TCP state transition diagram and the fields in the TCP control block. Here we will explain some of the most basic functions related to TCP, but how to organize these functions reasonably and efficiently to facilitate practical applications will not be covered here.

The first function is the `tcp_new` function, which simply calls the `tcp_alloc` function to assign a TCP control block `tcp_pcb` to a connection. The `tcp_alloc` function first allocates memory space to the new `tcp_pcb`, and if there is not enough memory space, the function frees up TCP in the TIME-WAIT state or a lower priority PCB (in the `prio` field of the PCB control block) to allocate space to the new PCB. When the memory space is successfully allocated, the function initializes the contents of the new `tcp_pcb`, and the source code is as follows: `if (pcb != NULL) {`

```
memset(pcb, 0, sizeof(struct tcp_pcb));    Clear the value of 0 for all fields
pcb->prio = TCP_PRIO_NORMAL; Set the priority of PCB to 64, and the priority is
between 1~127
pcb->snd_buf = TCP_SND_BUF; TCP send data buffer remaining
size
pcb->snd_queuelen = 0; Number of packets in the send buffer PBUFs:
pcb->rcv_wnd = TCP_WND; Receive window size
pcb->rcv_ann_wnd = TCP_WND; Announcement window size
pcb->tos = 0; TOS field
pcb->ttl = TCP_TTL in the header of the IP report; IP header TTL field
pcb->mss = (TCP_MSS > 536) ? 536 : TCP_MSS; Set the maximum segment size, which cannot exceed 536 bytes
PCB->RTO = 3000 / TCP_SLOW_INTERVAL; The initial timeout time value is 6s
pcb->sa = 0; Estimated RTT average??
pcb->sv = 3000 / TCP_SLOW_INTERVAL; Estimated RTT variance??
pcb->rtime = -1; Retransmission timer, when the value is greater than RTO, the retransmission occurs
PCB->CWND = 1; blocking window iss = tcp_next_iss(); ISS is a temporary variable that holds the initial data sequence number of the connection
pcb->snd_wl2 = iss; ACK number
pcb->snd_nxt received when the previous window was updated = iss; The next data number to be sent
pcb->snd_max = iss; Maximum data number sent
pcb->lastack = iss; Previous ACK number
```

```
pcb->snd_lbb = iss; The next data number to be buffered
```

```
pcb->tmr = tcp_ticks; tcp_ticks is a global variable that records the current protocol clock tick
pcb->polltmr = 0; I don't understand???
```

```
#if LWIP_CALLBACK_API
pcb->recv = tcp_recv_null; Register the default receive callback function
#endif /* LWIP_CALLBACK_API */
```

```
pcb->keep_idle = TCP_KEEPIDL_DEFAULT;
#if LWIP_TCP_KEEPALIVE    Keeping the timer related settings. Unsolved?
```

```
PCB->keep_intvl = TCP_KEEPINTVL_DEFAULT; PCB->keep_cnt= TCP_KEEPCNT_DEFAULT;#endif
```

pcb->keep_cnt_sent = 0; There are many halo spots on it, which will be explained one by one later. There are also some function segments in the PCB such as send and receive functions that are initialized in specific applications.

When a new PCB is initialized, the tcp_bind function will be called to bind the IP address and port number to the TCP control block. The function has obviously three input parameters, namely the TCP control block, the IP address, and the port number. The tcp_bind function also works simply, assigning the values of two parameters to the fields of local_ip and local_port in the TCP control block. But there is a premise here, that is, this < IP address, port, > pair is not used. Therefore, the function needs to traverse each PCB linked list first to ensure that the < IP address, port, > pair are not used by other PCBs, there are several types of PCB linked lists: linked lists in listening state, linked lists in stable state tcp_listen_pcb, tcp_bound_pcb, tcp_active_pcb, PCB linked lists that have been bound, and PCB linked lists tcp_tw_pcb in TIME-WAIT state. If after traversing these linked lists, no corresponding < IP address, port > pair is found, it means that the < IP address, port, > pair are available, then the above assignment operation can be performed, and finally, the function adds this PCB to the bound PCB linked list tcp_bound_pcb.

There are four types of PCB link lists mentioned above, and now let's take a look at which state each of them is used to link the PCB control blocks in which state. tcp_bound_pcb Linked Lists are used to connect newly created blocks, and the newly created blocks can be considered closed. tcp_listen_pcb Linked lists are used to connect blocks in the LISTEN state, tcp_tw_pcb linked lists are used to connect blocks in the TIME_WAIT state, and tcp_active_pcb linked lists are used to connect all other controls in the TCP state transition diagram.

From the state transition diagram, it can be seen that the server needs to enter the LISTEN state and wait for the client to connect. Therefore, the server needs to call the function tcp_listen to bring the corresponding TCP control block into the LISTEN state. You can directly imagine that it is very simple to put a control block in the LISTEN state, first remove it from the tcp_bound_pcb linked list, set its state field to LISTEN, and finally hook the PCB to the linked list tcp_listen_pcb. But in fact, there is a certain difference in the implementation of LWIP, it introduces a structure called tcp_pcb_listen, which is similar to the tcp_pcb structure, but removes the transmission control fields that are not used in the LISTEN phase, so that the structure of tcp_pcb_listen is smaller and can save memory space. So, in fact, tcp_listen does this, first applying for a tcp_pcb_listen structure, then copying the useful fields in the tcp_pcb parameters, and then hooking this tcp_pcb_listen structure to the linked list tcp_listen_pcb.

At this point, the server waits for the SYN packet sent by the client to connect, and waits for the outside packet, which is related to the ip_input ip_input function discussed before, which will judge the protocol field in the head of the IP packet and pass the TCP number packet to the TCP layer through the tcp_input function. SYN packets are of course TCP layer packets, but they also need to be processed by the tcp_input function and handed over to the upper layer, so let's take a look at the tcp_input function now.

tcp_input functions start to perform basic operations on packets submitted by the IP layer, such as moving the payload pointer of packets, dropping broadcast or multicast packets, data and checksums, extracting the values of various fields in the TCP header, and so on. Next, the function traverses the tcp_active_pcb linked list based on the < IP address, port, > of the received TCP packet for a matching PCB control block, and if found, it calls the tcp_process function to process the packet. If it is not found, it will look for it in the tcp_tw_pcb linked list and tcp_listen_pcb respectively, and if it is found, it will call the respective packet processing functions tcp_timewait_input and tcp_listen_input to process the packet, and if the matching TCP control block has not been found here, the tcp_input function will call the function tcp_rst to send a TCP reset packet to the source host.

Here our TCP control block is in the LISTEN state, connected on tcp_listen_pcb, waiting for a SYN packet. Therefore, when that packet is waited, the function tcp_listen_input should be called. From the state transition diagram

As can be seen above, the TCP control block in the LISTEN state can only respond to the SYN handshake packet, so the `tcp_listen_input` function returns a TCP reset packet for a non-SYN handshake packet, and if a packet is not a SYN packet, the ACK field in its TCP packet header is usually set to 1, so the `tcp_listen_input` function is implemented by checking the bit. Next, the function confirms whether the package is a SYN handshake package by verifying the SYN bit. If so, you need to create a new `tcp_pcb` structure, because the control block structure on the `tcp_listen_pcb` is the `tcp_pcb_listen` structure, and the control block structure on other linked lists is `tcp_pcb` structure, so create a new `tcp_pcb` structure here, copy the corresponding `tcp_pcb_listen` structure to it, and add this new `tcp_` to the `tcp_active_pcb` linked list. PCB structure. In this way, the new TCP control block is in `tcp_active_pcb`, and note that the state field of this `tcp_pcb` structure should be set to `SYN_RCVD`, indicating that it has entered the SYN state. Note that this `tcp_pcb_listen` structure in the `tcp_listen_pcb` linked list still exists, and it will not be deleted to wait for other clients to connect, which is exactly what the server needs.

At this point, the function `tcp_listen_input` is not finished. It should extract the value of the options field in the TCP header from the received SYN datagram and set its own TCP control block. The function to be called here is called `tcp_parseopt`, which currently only can do MSS (Longest Packet Size) field in the extraction options, which will be extended to support more TCP options in later versions of LWIP. After that, the function can also call `tcp_eff_send_mss` to set the value of the mss field in the control block, which can be literally translated as "valid longest packet size", which means that the MSS value in the received SYN packet cannot be greater than the maximum sent packet length supported by my hardware, that is, the MTU of the hardware. Therefore, when the received MSS value is larger, the mss field value in the settings control block is set to MTU instead of MSS.

Finally, the function needs to return a handshake packet with a SYN and ACK flag to the source, and can advertise its MSS size to the source. Sending packets is done through both the `tcp_enqueue` and `tcp_output` functions. The sending of packets will be introduced later.

Finally, let's take a look at the key source code parts inside the function `tcp_listen_input`, these lines of code involve setting the values of various fields inside the TCP control block, the most important of which is the sliding window-related fields.

```
ip_addr_set(&(npcb->local_ip), &(iphdr->dest)); 复制本地 IP 地址npcb->local_port= pcb->local_port;//复制本地端口ip_addr_set(&(npcb->remote_ip), &(iphdr->src)); 复制源 IP 地址npcb->remote_port = tcphdr->src;//复制源端口npcb->state = SYN_RCVD;// 设置 TCP 状态npcb->rcv_nxt = seqno + 1;// 期望接收到的下一个字节序号npcb->snd_wnd = tcphdr->wnd; 设置发送窗口大小nPCB->SSTHRESH = nPCB->snd_wnd;//快速启动阈值设为和发送窗口大小相同??npcb->snd_wl1 = seqno - 1;//该字段??NPCB->callback_arg = PCB->callback_arg; 该字段??#if LWIP_CALLBACK_API
```

```
npcb->accept = pcb->accept;  Receive callback functions
#endif /* LWIP_CALLBACK_API */ where npcb represents the newly created tcp_pcb structure, there are still many things I don't understand, why only copy and keep these fields, and the other fields are directly ignored?
```

18 TCP state machine, state machine, state machine, state machine, state machine

The server receives the SYN handshake packet, returns the handshake packet with SYN and ACK to the client, and puts the corresponding TCP control block in the SYN_RCVD state and hangs it on the tcp_active_pcb's linked list. Later, continue to wait for the handshake packet sent by the client, and this time, the server expects to receive an ACK packet to complete the three-way handshake operation required to establish the connection.

Again, as in the previous times, packets come in and pass through ip_input to tcp_input, which looks for a matching connection control block in three linked lists. This time it comes in an ACK handshake package sent by the client, and the corresponding tcp control block on the server side must be on the tcp_active_pcb's linked list. Next, use the found tcp_pcb structure as a parameter to call the TCP state machine function tcp_process to process the input data segments. Before explaining the tcp_process function, let's take a look at some of the important global variables used in this state machine function, which are set in the tcp_input function by receiving the values of each field of the data segment. The global variable tcphdr points to the TCP header of the received data segment; The global variable seqno records the data sequence number field in the TCP header; The global variable ackno stores the confirmation serial number field; The global variable flags represent the individual flag fields in the TCP header; The global variable tcplen represents the length of the data in the datagram, which is 1 for SYN and FIN packets; The global variable inseq is used to describe the data content received, which is of type tcp_seg, tcp_seg will be discussed later in this structure; The global variable rcv_flags is used to identify the result of the tcp_process function's processing of the data segment, initialized to 0.

tcp_process The function first determines whether the data segment is a reset data segment, and if so, it will be processed accordingly, skipping this small part for now. Coming directly to the tcp_process function state machine section, it is a simple code interpretation of the TCP state transition diagram. A large piece of code must be posted, which explains the problem better than any language, and note that the following code has been removed by me to explain and compile the output. switch (pcb->state) {

```
case SYN_SENT:           The client sends the SYN to the server and waits for the handshake packet to return
if ((flags & TCP_ACK) && (flags & TCP_SYN) // Actual ACK and SYN packets received
    && ackno == ntohs(pcb->unacked->tcphdr->seqno) + 1) {pcb-
    >snd_buf++; pcb->rcv_nxt = seqno + 1; pcb->lastack = ackno;
    pcb->snd_wnd = tcphdr->wnd; pcb->snd_wll = seqno - 1; /*
    initialise to seqno - 1 to force window update */pcb->state =
    ESTABLISHED;

    tcp_parseopt(pcb);    Process options fields
    #if TCP_CALCULATE_EFF_SEND_MSS // 根据需要设置有效发送 mss 字段
    pcb->mss = tcp_eff_send_mss(pcb->mss, &(pcb-
    >remote_ip));#endif
    pcb->sssthresh = pcb->mss * 10;//由于重新设置了 mss 字段值，所以要重设 sssthresh 值

    pcb->cwnd = ((pcb->cwnd == 1) ? (PCB->MSS*2) : PCB->MSS); Set the blocking window
    --pcb->snd_queueelen; The number of data segments to be sent is reduced by 1??
    rseq = pcb->unacked; Remove the field to be acknowledged
```

```

pcb->unacked = rseg->next;

if(pcb->unacked == NULL) // If there are no fields to be acknowledged, stop the timer
pcb->rtime =
-1; else {           Otherwise, restart the timer
pcb->rtime = 0; pcb->nrtx = 0; }tcp_seg_free(rseg);
Release of confirmed segment TCP_EVENT_CONNECTED
(pcb, ERR_OK, err); tcp_ack_now(pcb); Return ACK
Handshake Pack}else if (flags & TCP_ACK) { //Only ACK
without the SYN flag

    tcp_rst(ACKNO, SEQNO + TCplen, &(IPHDR->DEST), &(IPHDR->SRC),
tcphdr->dest, tcphdr->src); Half-open state is not supported, so return
a reset package}break; case SYN_RCVD:// is in this state after the
server sends SYN+ACK

if (flags & TCP_ACK && // receives the ACK return packet in the SYN_RCVD state
! (flags & TCP_RST)) { Determine whether the ACK serial number is legitimate
if (TCP_SEQ_BETWEEN(ackno, pcb->lastack+1, pcb->snd_nxt)) {
    ul6_t old_cwnd; pcb->state = ESTABLISHED; Enter ESTABLISHED state
    old_cwnd = pcb->cwnd; Save the old blocking window accepted_inseq
    = tcp_receive (pcb); If data is included, then receive data
    segment pcb->cwnd = ((old_cwnd == 1) ? (pcb->mss * 2) : pcb->mss);
    Reset the blocking window

    if ((flags & TCP_FIN) && accepted_inseq) { // If the ACK package contains both FIN bits and
        tcp_ack_now(pcb);           If the last data has been received, the FIN is answered
        pcb->state = CLOSE_WAIT; Enter CLOSE_WAIT
        state}}else { //Invalid ACK serial number returns a
        reset package

        tcp_rst(ACKNO, SEQNO + TCplen, &(IPHDR->DEST), &(IPHDR->SRC),
            tcphdr->dest, tcphdr->src);
    }}break; case CLOSE_WAIT: //server, TCP is in a semi-open state and no more
    packets will be received in that direction // The server will wait for the
    parent application to execute the shutdown command tcp_close in this state and
    change the state to LAST_ACK

case ESTABLISHED:// steady-state, and the client remains in steady-state until the parent application calls
the tcp_close function to close the connection, changing the state to FIN_WAIT_1
    accepted_inseq = tcp_receive(pcb); Receive data directly

```

```

if ((flags & TCP_FIN) && accepted_inseq) { //At the same time, the packet has a FIN flag and is received at the end
    tcp_ack_now(pcb);    , then respond to FIN
pcb->state = CLOSE_WAIT; enter CLOSE_WAIT
state}break; case FIN_WAIT_1:// Client-specific status

```

tcp_receive(pcb); You can also receive data from the server if (flags & TCP_FIN) { //if a FIN packet is received

```

if (flags & TCP_ACK && ackno == pcb->snd_nxt) { //且还有 ACK, 则进入 TIME_WAIT
    tcp_ack_now(pcb); Issue an ACK
    tcp_pcb_purge(pcb); Clear all existing data from that connection
    TCP_RMV (&tcp_active_pcb, pcb); Removed from the tcp_active_pcb linked list
    pcb->state = TIME_WAIT; TIME_WAIT state
    TCP_REG (&tcp_tw_pcb, pcb); Add tcp_tw_pcb linked lists
} else { //No ACK, it means that both ends are closed at the same time
    tcp_ack_now(pcb); Send an ACK
    pcb->state = CLOSING; // 进入 CLOSING 状态} else if (flags & TCP_ACK && ackno
== pcb->snd_nxt) { //不是 FIN 包, 而是有效 ACK

```

pcb->state = FIN_WAIT_2; then enter the FIN_WAIT_2
state}break; case FIN_WAIT_2:// client waits for the FIN
returned by the server in that state

tcp_receive(pcb); You can also receive data from the server if (flags & TCP_FIN) { //if a FIN packet is received

```

tcp_ack_now(pcb); ACKtcp_pcb_purge (PCB); Clear all existing
data TCP_RMV (&tcp_active_pcb, pcb) in that connection;
Remove pcb->state = TIME_WAIT from the tcp_active_pcb linked
list; set to TIME_WAIT state TCP_REG (&tcp_tw_pcb, pcb);
join tcp_tw_pcb linked list}break; case CLOSING: // enters a
simultaneous shutdown state, which is rare

```

tcp_receive(pcb); You can also receive data from the other party if (flags & TCP_ACK && ackno
== pcb->snd_nxt) { //If it is a valid ACK

```

    tcp_ack_now(pcb);    Similar operations to the above, will not be explained
tcp_pcb_purge(pcb); TCP_RMV (&tcp_active_pcb,
pcb); pcb->state = TIME_WAIT;
TCP_REG (&tcp_tw_pcb, pcb);}break; case
LAST_ACK://服务器端(被动关闭端)能出现该状态

```

tcp_receive(pcb); It can also receive data from the other party

```
if (flags & TCP_ACK && ackno == pcb->snd_nxt) { //接收到有效 ACK
recv_flags = TF_CLOSED; The global variable recv_flags is used to identify what
processing has been done for the data segment} // for subsequent processing of the
tcp_input, at this time the state of pcb->state is not set to CLOSED state break; default:
```

break; This is the general flow diagram of the TCP state machine, and if you compare it with the TCP state transition diagram, you will find it so simple. However, there are still many things that are not clear, such as the values of the fields in the control block, especially the blocking window and the send-receive window, what they represent, why they are set this way, etc.

Note that both sides in the TCP state transition diagram are turned on at the same time, that is, from the LISTEN state to the SYN_SENT state, and from the SYN_SENT state to the SYN_RCVD state, which is not implemented. Many other TCP/IP implementations, such as those in BSD, do not implement this part of the functionality. Because this is almost invisible in practice, the implementation is not strictly in accordance with the protocol.

19 TCP Input Output Function Input Output Function Input Output Function Input Output Function 1

This section starts with the `tcp_receive` function and gradually gains an in-depth understanding of the meaning of each field in the control block and the operation mechanism of the entire TCP layer, with a full 600 lines. The function of the source code annotation is: check whether the received data segment is a confirmation of the sent data segment, and if so, release the data in the corresponding send buffer; Next, if there is data in that data segment, the data should be hooked to the receive queue of the control block (`pcb->oosseq`). If the data segment is also an acknowledgment of the data segment for which the RTT estimation is being made, the RTT calculation is also performed in this function. I fainted and fell into a vicious circle. The more you look at it, the more difficult it becomes, the more you can't explain it, there are too many things in TCP. To explain `tcp_receive` clearly, you have to make it clear `tcp_enqueue`, no matter what, just bite the bullet and write it down first! The source code comments describe the function as simple: place a packet or connected control handshake packet on the transmit queue of the TCP control block. This function is based on `err_tcp_enqueue ( struct tcp_pcb *pcb, void *arg, u16_t len, u8_t flags,`

`u8_t apiflags, u8_t *optdata, u8_t optlen )`

There are several important input parameters: the PCB is the TCP control block to which it is connected; `arg` is a pointer to the data to be sent; `len` is the length of the data to be sent, measured in bytes; `flags` is an identification field in the header of a TCP data segment that is primarily used to connect established or disconnected handshakes; `apiflags` indicate the operations to be performed on the data segment, including whether to copy the data and whether to set the PUSH flag; `optdata` represents the value of the option field in the TCP header, and `optlen` represents the length of the option field.

`tcp_enqueue` First, check whether the length of the data to be sent is less than the size of the data transmission buffer that can be used by the current connection, that is, the `PCB->snd_buf`, if the buffer is not enough, the data will not be processed in any way (in fact, this buffer does not exist, just use `snd_buf` to indicate the amount of data that can still be cached by the connection). Next, set the sequence number field of the data segment to be sent to `PCB->snd_lbb`, and then determine if the `PCB->snd_queueulen` value exceeds the upper limit of the packet allowed to be mounted `TCP_SND_QUEUEULEN`, and if it exceeds this upper limit, the function will not process the data segment to be sent. Next, the `tcp_enqueue` function will assemble the data into `tcp_seg` type data segments, depending on the size of the data length, it may take several `tcp_seg` type structures to describe all the data, and the TCP header part field values in each data segment must be set here, including the data serial number and flag fields. Finally, all created `tcp_seg` type structures are attached to the queue `queue`, which is a temporary variable of the function. Next, the function `tcp_enqueue` needs to hook the data segment on the queue `queue` to the unsent queue of the TCP control block, and there are several situations, i.e., whether the unsent queue is empty, if it is, hook it directly, if not, it needs to hook the queue after the last `tcp_seg` of the unsent queue, if the two adjacent `tcp_seg` at the hook point contain data size less than the maximum send segment size `pcb->mss`, and the two adjacent segments are not FIN or SYN packets, you need to merge the two segments into one segment. Finally, the function needs to adjust the values of the relevant fields in the TCP control block, which is also my main concern.

```
if ((flags & TCP_SYN) || (flags & TCP_FIN)) {    Sending a SYN or FIN packet is considered to have
++len;} if (flags &                             a data length of 1
TCP_FIN) {
```

For FIN packages, set the flags field to the corresponding value

```
pcb->flags |= TF_FIN; }pcb->snd_lbb += len; The next serial number of the
data to be buffered, note that it is different from snd_next pcb->snd_buf
-= len; Reduce the number of free send buffers, noting that this buffer
does not really exist pcb->snd_queueulen = queueulen; The number of PBUs in
the unsent queue
```

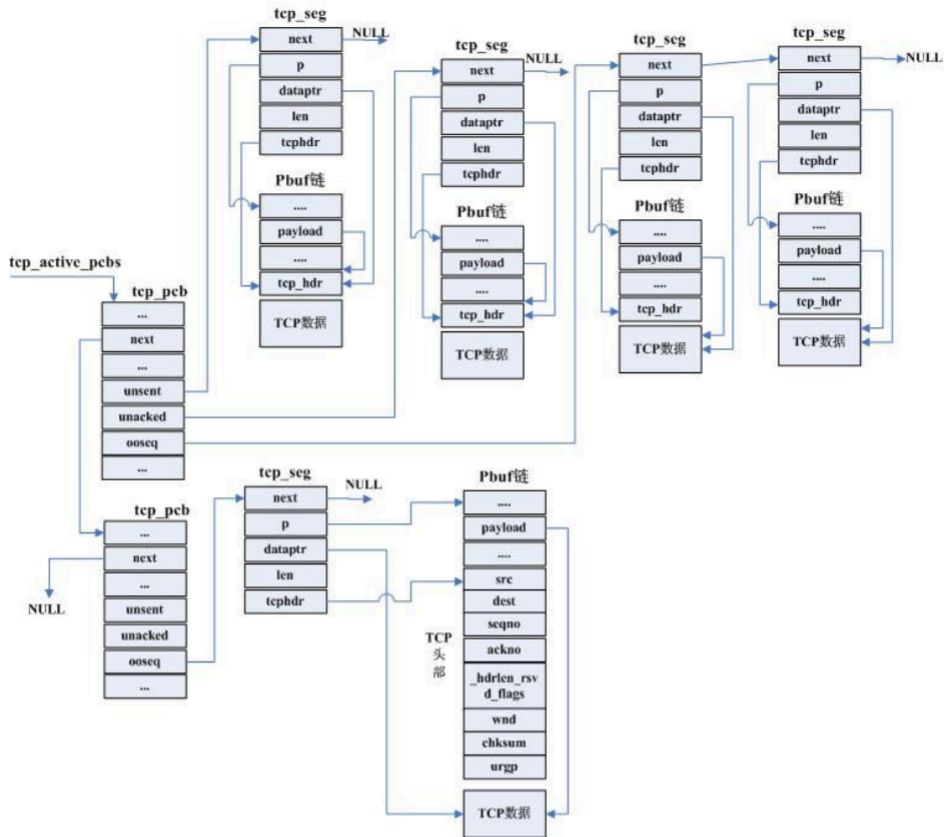
Because these fields are very critical when looking at how they are implemented when sliding the window. Messy and messy, talking about tcp_enqueue functions, and having to talk about tcp_output functions. The tcp_output function has a unique parameter, that is, a linked BTCP control block pointer PCB, which sends a data block on the unsent queue of the control block or sends an ACK data segment directly. If the flags field of the control block has a TF_ACK_NOW flag set when the function is called, the function must immediately emit an ACK flag. Therefore, if there is no data sent in the unsent queue or the send window does not allow data to be sent at this time, the function needs to send an ACK datagram without any data. When there is no TF_ACK_NOW placement, or TF_ACK_NOW placement but the ACK can be sent with the data segment, the function will remove the data segment on the unsent queue and send it out (not considering the nagle algorithm for now). Sending a specific data segment is achieved by calling the function tcp_output_segment, which mainly fills in the TCP header of the data segment to be sent with a confirmation sequence number of pcb->rcv_nxt, a notification window size of pcb->rcv_ann_wnd, a checksum field, and finally tcp_output_segment delivers the packet to the IP layer for sending. Of course, there are many other operations in tcp_output_segment that we won't care about here for now.

Okay, let's go back to the tcp_output path, after the data segment is sent, this function also needs to set the value of the relevant fields of the control block. What I am most concerned about here is the field that is closely related to the sliding window, pcb->snd_nxt = ntohl(seg->tcphdr->seqno) + TCP_TCLEN(seg); Next byte sequence to send if (TCP_SEQ_LT(pcb->snd_max, pcb->snd_nxt)) { pcb->snd_max = pcb->snd_nxt; Maximum send sequence} Next, the function hooks the sent segment on the control block unacked linked list for subsequent retransmission and other operations. At this point, the first data segment on the unsent queue is processed, and the tcp_output function will process the remaining data segments on the unsent queue in turn until all the data is sent out or the send window is filled.

Now it's time to take a look at the behemoth tcp_receive. This function simply operates the unsent, unacked, and ooseq fields in the TCP control block, which are used to connect various data segments of TCP. unsent is used to connect segments that have not yet been sent, unacked is used to connect segments that have been sent but have not yet been acknowledged, and ooseq is used to connect received unordered segments. These three fields are pointers of type tcp_seg, struct tcp_seg used to describe a TCP data segment, the source code is as follows:

```
struct tcp_seg {
    struct tcp_seg *next;  A pointer used to create a linked list
    struct pbuf *p;        Data segment PBUF pointer
    void *dataptr;         Data area pointing to the TCP segment
    u16_t len;             The length of the data for the TCP segment
    struct tcp_hdr *tcphdr; pointing to TCP headers};
```

Mastering this struct is important, which is key to understanding tcp_receive functions. As you can see from the diagram below, how a TCP data segment is described when tcp_seg structure. TCP control blocks that can send and receive data segments are connected to the linked list tcp_active_pcb, and the three pointers of each control block, unsent, unacked, and ooseq, connect the data related to the connection. The tcp_seg structure on the unsent, unacked linked lists and ooseq linked lists describe the data segments in different ways, and it can be seen from the graph that the dataptr and tcphdr fields of the tcp_seg structures of unsent, unacked linked lists point to the data start position of pbufs, that is, the TCP header position. The tcp_seg structure dataptr on the ooseq linked list points to the start of the TCP data segment, and the tcphdr field points to the TCP header. And for packet pbuf on the linked list ooseq, its payload pointer also points to the start of the TCP data segment, not to the data start position of the pbuf. This is because the TCP data segments on the linked table ooseq are all submitted from the IP layer, and the TCP layer has called the tcp_input function to point the payload pointer of the packet to the start of the TCP data field.



TCP Input Output Function Input Output Function Input Output Function Input Output Function 2

Continue with the `tcp_receive` function. As mentioned earlier, when the `tcp_input` function receives a packet from the IP layer, it will save the relevant TCP header fields in some global variables, and the global variable `seqno` records the data sequence number field in the TCP header. The global variable `ackno` stores the confirmation serial number field; The global variable `flags` represent the individual flag fields in the TCP header; The global variable `tcplen` represents the length of the data in the datagram, and for packets set by SYN or FIN, the length is added by 1; the global variable `inseq` is used to describe the content of the data received, which is of type `tcp_seg`. In the implementation function `tcp_process` of the TCP state machine, function `tcp_receive` is called in different places to process the input TCP data segments.

`tcp_receive` First, determine whether the size of the new local sending window can be communicated according to the received data segment, because the data segment contains the other party's window size announcement. The fields in the new TCP control block with the window are: `snd_wnd`, `snd_wl1`, and `snd_wl2`, which represent the current window size, the data sequence number received when the window was updated last time (`seqno`), and the confirmation sequence number received when the last window was updated (`ackno`). There are three situations that can cause a local send window to be new: a `seqno` greater than `snd_wl1` in the received data segment (this is used during the connection establishment handshake), a new `seqno` equal to `snd_wl1` and a new acknowledgment number `ackno` greater than `snd_wl2`, a new acknowledgment number `ackno` equal to `snd_wl2`, and a window advertisement larger than `snd_wnd` in the data segment. Why are these three situations, it means that I don't understand, and I still need to work hard. When any of the above three situations are met, the window needs to be updated, that is, the three values of `snd_wnd`, `snd_wl1`, and `snd_wl2` need to be reset.

Next, the `tcp_receive` function performs the relevant operation based on the acknowledgment sequence number `ackno` value of the received data segment. This value is the data confirmation returned by the other party. The `lastack` field in the TCP control block records the value of the last confirmation sequence received by the connection, and the function compares the `lastack` and `ackno` values to determine whether the ACK is a duplicate ACK.

Normally, `ackno` should be between `lastack+1` and `snd_max`, in this case, the function first sets the values of the TCP control block-related fields, the most important of which are `snd_buf`, `lastack` fields, and of course, fields related to timeout retransmission, slow start, etc., which will not be discussed here. The function then processes the unacked queue of the control block based on this `ackno`, because the unacked queue connects to the data segments that have not been confirmed after being sent, so when receiving the `ackno`, it should traverse the unacked queue and remove all data segments in the data segment with a data number less than or equal to `ackno`. When all the required data segments are successfully removed, the function should check whether the unacked queue is empty, if so, stop the retransmission timer, otherwise reset the retransmission timer.

Next, the function will also use this `ackno` to handle the unsent queue of the control block, which may seem strange (this is said in the source code comments)! Because for TCP segments that need to be retransmitted, LWIP directly hangs them on the unsent queue, so after receiving the confirmation, it may be an acknowledgment of the data segment that has timed out, so the function will traverse the unsent queue and remove all data segments in the data segment whose data number is less than or equal to `ackno`, of course, the legitimacy of `ackno` is also a consideration in this case, that is, `ackno` cannot be greater than `snd_max` value, That is, the maximum data serial number that has been sent.

Next, if the `ackno` confirms the data segment for which the RTT estimation is being made, the estimation of the RTT takes place here. The `RTSEQ` field of the control block records the data sequence number of the data segment for which the RTT estimation is being made. About RTT, that's for later discussion.

As mentioned earlier, `seqno` records the starting sequence number of the received data, `tcplen` represents the length of the data, and for SYN and FIN packets, the length is added by 1, these two global variables play a very important role in this part. The receiving window is closely related to the receiving data, and the three important fields in the control block `rcv_nxt`, `rcv_wnd`, and `rcv_ann_wnd` represent the next byte number to be received, the size of the receiving window, and the size of the receiving window to be communicated to the other party. The following three things are mainly done for data processing: if the received data segment number contains `rcv_nxt`, it means that the data segment is a sequential segment, and this segment can be directly submitted to the upper level;

Otherwise, this data segment needs to be connected to the ooseq linked list according to its data number, and during the process of inserting data segments on the linked list, there may be data overlap, and the duplicate data needs to be removed. Finally, it checks if the first data on the ooseq linked list is the segment that arrived sequentially, and if so, the segment is submitted to the upper level.

First, if the data starting number seqno is not exactly rcv_nxt, but larger than rcv_nxt, and there is data in the data segment with a data number of rcv_nxt, then the data segment needs to be truncated so that the starting number is exactly rcv_nxt. If the seqno is less than rcv_nxt and all the data in the data segment is numbered less than rcv_nxt, it means that this is a duplicate data segment, and it only responds to an ACK packet to the source side and does no other processing.

Next, determine if the data segment is in the receive window (the value of the seqno may have been adjusted in the previous step), and the starting byte sequence numbers of the receive window are rcv_nxt and rcv_nxt + rcv_ann_wnd - 1, respectively. If it is in a window, the data can be further processed according to the position of the data segment in the window.

If the data segment is at the starting position of the window, i.e. seqno equals rcv_nxt, it means that the data segment is a continuous data segment and can be submitted directly to the upper level. However, LWIP does not directly hand over this data to the upper layer, but hooks the data segment to the first position of the ooseq linked list, because other hooked data segments on the ooseq linked list may become ordered due to the arrival of this data segment (it can be thought that if the data is smaller than the starting number of a certain data segment, the data segment is ordered and can be submitted to the upper layer), so that the protocol can submit as much data as possible to the upper layer. After the data segment is inserted into the first position of the ooseq linked list, the data of the first and second data segments on the linked list may overlap, and the tail of the first data needs to be truncated to avoid data duplication. The global variable rcv_data pointer is used to point to the data package that can be submitted to the upper level, so here the tcp_receive function will point rcv_data to the data in the first data segment, and set the values of rcv_nxt, rcv_wnd, and rcv_ann_wnd, and then, iterate through the subsequent data segments of the ooseq linked list, hook all the ordered data to the rcv_data pointer, and after the hooking is completed, Return an acknowledgment packet to the source. The actual source code implementation is exactly the same as the above process, but there is a slight difference in the implementation from the above, as can be seen from the above description, the data segment inserted to the first position will be immediately removed and hung on the rcv_data pointer, so this insertion operation is not necessary, and it is true.

If the data segment is not at the starting position of the window, an immediate confirmation data segment is returned to the source end, and the data segment is hooked to the ooseq linked list, and the hooked data segment is mainly inserted in the linked list according to the data number, and if the insertion operation may have data overlap, the data needs to be truncated. Remembering the word pseudocode, let's describe the process in pseudocode: ZSL = 1; for (take the ZSL data segment from ooseq; The data segment is not empty; ZSL++){if the segment start number == The segment start number to be inserted

if the data segment to be inserted is longer

Replace the ZSL data segment with the data segment to be inserted; Delete the ZSL data segment; If the inserted data segment has data overlap with the next data segment

truncates the tail of the inserted data segment; end jumps out of the loop; else jumps out of the loop; end else if ZSL == 1 // is the first data segment

if the segment start number > the segment start number to be inserted

Insert a new data segment into the first position; If there is a data overlap with the latter data segment, the end of the data segment is truncated; Jumping out of the loop;

Extension

If the data segment to be inserted is not the first data segment starting number, the starting number is between the ZSL-1 and the ZSL data segment starting number
Insert the data segment onto the ZSL location; If there is a data overlap with the latter data segment, the end of the data segment is truncated; If there is a data overlap with the previous data segment, the tail of the previous data segment is truncated; Jumping out of the loop; endend

if the ZSL data segment is the last data segment on the linked list
insert the new data segment on the linked list tail; If there is a data overlap with the previous data segment, the tail of the previous data segment is truncated; Jumping out of the loop; endend}

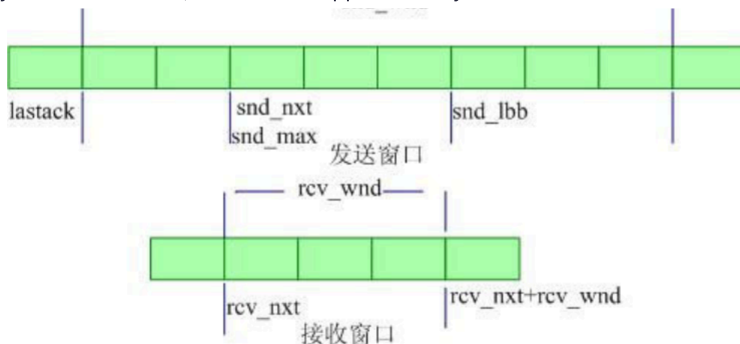
In retrospect, if the data segment is outside the window, an immediate confirmation data segment is returned directly to the source. At this point the function tcp_receive is done, if it hooks new data to the data pointer recv_data, the function returns a 1, otherwise it returns 0.

21 TCP Sliding Windows

Earlier, all the basic things of the TCP layer were discussed, and now we will talk about the various TCP mechanisms that support TCP functions, including the sliding window mechanism, TCP timer, RTT estimation and timeout retransmission mechanism, slow start and congestion avoidance algorithm, fast recovery and retransmission, Nagle algorithm, etc. Let's talk about how the sliding window is implemented first, in the TCP control block, we have already touched on the fields related to the send and receive windows, let's take a look at it again. The fields related to the sending window are: `snd_wl1`, `snd_wl2`, `snd_next`, `snd_max`, `lastack`, `snd_lbb`, `snd_wnd`. That's too many!!

The `snd_wl1` and `snd_wl2` fields are closely related to the update of the window, and they represent the `seqno` and `ackno` in the data segment received at the last window update, respectively. When a new data segment is received, whether it is necessary to extract the window notification in the data segment and update the window depends on the size relationship between the values of the `snd_wl1` and `snd_wl2` fields and the `seqno` and `ackno` in the data segment. Field `snd_wnd` saves the window size. The `lastack` field represents the last valid confirmed `ackno`; `snd_next` indicates the start number of the data that will be sent next; `snd_max` represents the maximum data sequence number sent +1 (which seems to be the same as the value of `snd_next`, but why use the `snd_max` field to indicate confusion), and `snd_lbb` indicates the data number that will be cached in the next send window. The last three fields are too tangled, and it may be easier to understand by looking at the diagram below.

`snd_max` and `snd_next` have the same value, the data from `lastack` to `snd_next` is sent but not acknowledged, they are hooked to the unacked linked list of the control block, and the data from `snd_next` to `snd_lbb` is the data that has been submitted to the stack by the application but not sent by the stack, and these data are hooked to the unsent linked list of the control block, from `snd_lbb` To the end of the window indicates that the send buffer is available, but the application layer has not yet submitted data, and the next application layer commits data from `snd_lbb` onwards.



The receiving window has three fields `rcv_next`, `rcv_wnd`, `rcv_ann_wnd`, where `rcv_next` represents the serial number of the next data to be received, `rcv_wnd` represents the size of the receiving window, and `rcv_ann_wnd` represents the size of the window announced to the sender. The receive window is simpler than the send window, which we also describe in the image above. So how does the sliding window on both ends of the TCP connection work? Now let's take a look at how the slider works by establishing a connection between the client and the server.

(1) Server-side: First, call the function `tcp_alloc` allocate a new TCP control block, in the function `tcp_alloc`, the `rcv_wnd` and `rcv_ann_wnd` of the control block are initialized as the default size `TCP_WND`, and the five fields of `snd_wl2`, `snd_next`, `snd_max`, `lastack`, and `snd_lbb` are initialized as `ISS` (`ISS` can be regarded as a system global variable, and its value changes with time) This assumes that all five fields are initialized as `ZSL1`, which means that they were received the last time the window was updated

The last valid confirmed data number is ZSL1, the next data number to be sent is ZSL1, and the next data number to be cached is ZSL1. Then call the function `tcp_bind` to bind the port, which does not involve the operation of window-related fields; Next, the server calls the function `tcp_listen_with_backlog` to enter the listening state, and a new `tcp_pcb_listen` structure will be created to save the relevant fields of the TCP control block, and the `tcp_pcb_listen` structure will be placed `tcp_listen_pcb`s the linked list, at which point the TCP control block will enter the LISTEN state and wait for the client to connect.

(2) Client: The first few steps are the same as the server, first call the function `tcp_alloc` to allocate a new TCP control block, the `rcv_wnd` and `rcv_ann_wnd` of the control block are initialized to the default size `TCP_WND`, and the five fields of `snd_wl2`, `snd_nxt`, `snd_max`, `lastack`, and `snd_lbb` are initialized as their own ISS (initial sequence number), here it is assumed that these five fields are all initialized ZSL2, which means that the confirmation number received during the last window update is ZSL2, the last valid confirmed data number is ZSL2, the next data number to be sent is ZSL2, and the next data number that will be cached is ZSL2; Then call the function `tcp_bind` to bind the port, which does not involve the operation of window-related fields; Next, the client calls function `tcp_connect` to initiate a connection to the server side, in the `tcp_connect` function, the `rcv_nxt` field is set to 0, `snd_nxt` is ZSL2, `lastack` is ZSL2-1, `snd_lbb` is ZSL2-1, `rcv_wnd` and `rcv_ann_wnd` are set to the default size `TCP_WND`, and the receive window `snd_wnd` is set to the default size `TCP_WND`. Next, the `tcp_enqueue` function is called to send a SYN packet, which is what was talked about earlier, in the function `tcp_enqueue`, the packet is assembled and hung on the unsent queue, the start number of the packet is set to the value of `snd_lbb` ZSL2-1(*), and then adjust the relevant fields of the send window, as in the code below,

```
if ((flags & TCP_SYN) || (flags & TCP_FIN)) {    Sending a SYN or FIN packet is considered to have
++len;} PCB-                                   a data length of 1
>snd_lbb += Len;
```

The sequence number of the next data to be buffered is different from `snd_nxt`. So, after the `tcp_enqueue` function, the `snd_lbb` value becomes ZSL2, and the other field values remain unchanged.

The `tcp_connect` function then calls `tcp_output` to send the packet, which sends a specific data segment by calling the function `tcp_output_segment`, which mainly fills the TCP header of the data segment to be sent with a value of 0 (*) with a confirmation sequence number of `rcv_nxt` and a value `TCP_WND` (*) with a notification window size of `rcv_ann_wnd`. Finally, `tcp_output` update the window-related fields with the following code: `pcb->snd_nxt = ntohl(seg->tcphdr->seqno) + TCP_TCLEN(seg)`; Next byte sequence to send if `(TCP_SEQ_LT(pcb->snd_max, pcb->snd_nxt))` { `pcb->snd_max = pcb->snd_nxt`; The values of both `snd_nxt` and `snd_max` become ZSL2. In summary, after the client sends a SYN packet, it enters the `SYN_SENT` state, and at this time, the fields related to the window in the control block are `rcv_nxt` 0, `snd_nxt` is ZSL2, `lastack` is ZSL2-1, `snd_lbb` is ZSL2, `rcv_wnd` and `rcv_ann_wnd` are the default size `TCP_WND`, and the sending window `snd_wnd` is the default size `TCP_WND`; The `seqno` in the outgoing SYN package is ZSL2-1, the `ackno` is 0, and the announcement window is `TCP_WND`.

(3) Server-side: After the control block in the LISTEN state receives the SYN packet, it calls the function `tcp_listen_input` process the packet, because the packet is a SYN package, it needs to re-establish a control block, copy the content in the `tcp_pcb_listen` structure, and add the control block to the `tcp_active_pcb`s linked list. Here, the window-related fields of the control block are populated: `rcv_nxt = seqno + 1= ZSL2-1+1= ZSL2`, `snd_wl1 = seqno - 1= ZSL2-2` (set to minus 2 to ensure that the window can be updated the next time a data segment is received). `rcv_wnd` and `rcv_ann_wnd` are the default size `TCP_WND`, and `snd_wl2`, `snd_nxt`, `snd_max`, `lastack`, and `snd_lbb` remain the same as ZSL1. Then the server calls the function `tcp_enqueue` assembles the packet to hang on the unsent queue, the start number of the packet is set to the value of `snd_lbb` ZSL1(*), and then adjusts the relevant fields of the send window, as shown in the code below,

```
if ((flags & TCP_SYN) || (flags &    Sending a SYN or FIN packet is considered to have
TCP_FIN)) {                          a data length of 1
```

```

++len;} PCB-
>snd_lbb += Len;

```

The sequence number of the next data to be buffered is different from `snd_nxt`

Therefore, after the `tcp_enqueue` function, the `snd_lbb` value becomes `ZSL1+1`, and the other field values remain unchanged.

Next, call `tcp_output` to send the packet out, similar to the client, fill the TCP header of the data segment to be sent with a value of `rcv_nxt` with a value of `ZSL2(*)` and a value of `TCP_WND(*)` with a window size of `rcv_ann_wnd`. Finally, `tcp_output` also update the window-related fields: `pcb->snd_nxt = ntohl(seg->tcphdr->seqno) + TCP_TCPLen(seg);` Next byte sequence to send if `(TCP_SEQ_LT(pcb->snd_max, pcb->snd_nxt))` {

`pcb->snd_max = pcb->snd_nxt;` The values of both `snd_nxt` and `snd_max` become `ZSL1+1`. In summary, the server enters the `SYN_RCVD` state after sending a `SYN+ACK`, and the fields related to the window in the control block `rcv_nxt` `ZSL2`, `snd_nxt` `ZSL1+1`, `lastack` `ZSL1`, `snd_lbb` `ZSL1+1`, `rcv_wnd` and `rcv_ann_wnd` are the default size `TCP_WND`, and sending window `snd_wnd` is the default size `TCP_WND`. The `seqno` in the `SYN+ACK` package sent out is `ZSL1`, the `ackno` is `ZSL2`, and the announcement window is `TCP_WND`.

(4) Client: The client in the `SYN_SENT` state calls the function `tcp_process` processes the `SYN+ACK` packet returned by the server: `rcv_nxt = seqno + 1 = ZSL1+1`; `lastack = ackno = ZSL2`; `snd_wl1 = seqno - 1 = ZSL1-1` (this sets `snd_wl1` so that the window can be updated the next time a segment is received). At this point, the client calls the function `tcp_ack_now` to return an immediate confirmation packet to the server, and the `tcp_ack_now` also sends the packet by calling the function `tcp_output`, here we do not call the function `tcp_enqueue` to assemble a packet, but directly assemble and send it in `tcp_output`, here the packet sent `seqno = (pcb->snd_nxt) = ZSL2`, `ackno = (pcb->rcv_nxt) = ZSL1+1`, note that the packet does not contain any data and does not take up any data length, so the `snd_nxt` field value remains the same. After sending the `ACK` package, the client enters the `ESTABLISHED` state.

(5) Server side: After the server in the `SYN_RCVD` state receives the confirmation returned by the client, it calls the function `tcp_process` to process the packet. The process is simple, there is no operation involving the window-related fields, just putting the control block in the `ESTABLISHED` state.

At this point, a connection is established, let's take a look at the windows at both ends. Server-side: `ZSL2` for `rcv_nxt`, `ZSL1+1` for `snd_nxt`, `ZSL1` for `lastack`, `ZSL1+1` for `snd_lbb`, and the window size is default. Client: `ZSL1+1` for `rcv_nxt`, `ZSL2` for `snd_nxt`, `ZSL2` for `lastack`, `ZSL2` for `snd_lbb`, and the window size is default.

Comparing the above values, it can be seen that the sliding windows at both ends are established correctly. Judging from the whole process, it was chaotic and irregular. When the connection between the two ends enters the `ESTABLISHED`, the window adjustment is quite regular, and the functions `tcp_receive` and `rcv_nxt` are closely related, because the received data segment will be processed by this function; `tcp_enqueue` functions and `snd_lbb` fields are closely related to cache packets; The `tcp_output` function is closely related to `snd_nxt`, which is used to send packets into the network; The `tcp_output_segment` function primarily populates the `ackno` field of the sending package.

During the interaction between the two sides of TCP, both data and acknowledgments can be lost. TCP solves this problem by setting a timer when sending. If no acknowledgment has been received when the timer overflows, it retransmits that data. For any TCP protocol implementation, how to determine the timeout interval and how to determine the frequency of retransmission is key to improving TCP performance.

This section explains the TCP timeout retransmission mechanism, the relevant fields inside the TCP control block `tcp_pcb` are `rtime`, `rttest`, `rtseq`, `sa`, `sv`, `rto`, `nrtx`, too much, don't faint!

Closely related to the timeout interval is the estimate of round-trip time (RTT). RTT is the time interval between the time that a byte of data is emitted to that byte acknowledging the return. Since both router and network traffic change, RTT can change frequently, and TCP should track these changes and change its timeout accordingly.

The sender may send multiple packets in a row over a period of time, but the sender can only select one packet start timer to estimate its RTT value, and the estimator should not be updated until a packet segment is resent and the packet is acknowledged. Some optimization algorithms are used in the protocol to smooth the value of RTT, and the value of RTO is set according to the RTT value, which is the retransmission timeout time for the next packet.

Let's first look at how the timeout retransmission mechanism is implemented, and then focus on the parts that are closely related to RTT estimation. As mentioned earlier, `tcp_output` takes the first data segment from the unsent queue and calls the function `tcp_output_segment` to send the data segment, and after sending it, `tcp_output` hooks the data segment to the unacked queue. `tcp_output_segment` is responsible for sending out the data segments, and what it does after sending them out is shown in the following code:

```
if(pcb->rtime == -1)
pcb->rtime = 0; if
(pcb->rttest == 0) {

pcb->rttest = tcp_ticks; pcb->rtseq = ntohl(seg->tcphdr->seqno); }rtime is used to retransmit the timer's count, and when
its value is -1, it means that the counter is not enabled; When the value is non-0, it indicates counter enable, in which
case the value of rtime is added by 1 every 500ms by the kernel, and when rtime exceeds the value of rto, all data
segments on the unacked queue will be retransmitted. RTOs have been mentioned many times as the timeout retransmission
time we set for packets. The next rttest field is closely related to RTT estimation, when the rttest value is 0, it means
that the RTT estimation is not started, otherwise to start the RTT estimation, the value of the rttest should be set to
tcp_ticks (global variable, the current number of ticks in the system) after sending the packet out, and the starting data
number of the data segment for which the RTT estimation is to be made is recorded with the rtseq field. After receiving
the ACK number returned by the other party, RTT can be calculated based on the values of rttest and rtseq, and the
calculation of fields sa and sv and rto values is closely related, which will be discussed later. That's all there is to
send data segments, mainly to start the retransmission timer for the sent data segments. Of course, if the data segment is
sent out, the retransmission timer is started, that is, the rtime is not equal to -1, and no operation is done on the
retransmission timer at this moment. Similarly, if the rttest is not equal to 0, the RTT is in progress and no action is
taken on the individual fields of the RTT.
```

The TCP slow timer generates interrupt processing every 500ms, in which the `rtime` value is added by 1 if the TCP control block's retransmission counter is started (i.e., `rtime` is not 0). At the same time, when the `rtime` value is greater than `rto`, call the retransmission function to retransmit the unconfirmed data, the code is as follows (only the part related to retransmission is listed): `if (pcb->unacked != NULL && pcb->rtime >= pcb->rto) {`

.....

pcb->rtime = 0; Reset counter

tcp_rexmit_rto(pcb); The function call to retransmit the}tcp_rexmit_rto The mechanism of the function to implement retransmission is simple, it inserts all the data segments on the unacked linked list into the front end of the unsent queue, and retransmits the control block to the sub-digit segment nrtx plus 1, and finally calls tcp_output to resend the packet.

Here you may find a problem, let's say we just took a data segment from the unsent queue and sent it out, hooked it to the unacked linked list and waited for confirmation, and then the retransmission timer set on a previous data segment times out, so that the entire unacked linked list is put on the unsent queue for retransmission. Inevitably, the data segment we just sent back to the unsent queue is not a tragedy, and it has to be sent again. While this can happen, LWIP minimizes this possibility by controlling it through the window and traversing the unsent queue after receiving an acknowledgment number (described below). Note here that in older versions of the LWIP stack, there is an rtime field in each segment structure tcp_seg to record the timeout of a certain data segment, which avoids putting the entire unacked linked list back to the unsent queue when retransmitting. In the new version, the entire TCP control block uses an rtime field.

In terms of data reception, the tcp_receive function extracts the ackno in the received data segment and uses the ackno to process the unacked queue, that is, when the ackno confirms all the data in a data segment, it removes the data segment from the unacked queue and frees up the space occupied by the data segment. At the same time, the function needs to check the unacked queue, if there are no data segments in the unacked queue that need to be confirmed, the retransmission timer needs to be stopped, otherwise the retransmission timer needs to be reset. It's simple, with the following code:

```
if(pcb->unacked == NULL)
    pcb->rtime = -1;
else
```

```
pcb->rtime = 0; Next, the tcp_receive function also iterates through the unsent queue based on the acknowledgment number received to process the segments that are waiting to be retransmitted. Which data segments on the unsent queue are retransmitted? Obviously, the data segments within the data segment have a data number less than the value of the control block snd_max field. The data segment that can be confirmed by the confirmation number is removed from the unsent linked list, and the space occupied by the data segment is freed.
```

That's all there is to the timeout setting and retransmission of data segments, and here is the very important content, that is, RTT estimation. The code here is a bit difficult! Let's take a look at how RTT is described in TCP/IP Explained 1.

The most important part of TCP timeout and retransmission is the measurement of the round-trip time (RTT) of a given connection. Since both router and network traffic change, we believe this time can change frequently, and TCP should keep track of these changes and change its timeout accordingly. TCP must measure the RTT between sending a byte with a special sequence number and receiving an acknowledgment containing that byte, noting that there is no one-to-one correspondence between the outgoing data segment and the returned acknowledgment.

The following algorithm was proposed by Jacobson and is widely used in the implementation of the TCP protocol, and LWIP is no exception.

$$\begin{aligned}Err &= M - A \\ A &\leftarrow A + gErr \\ D &\leftarrow D + h(|Err| - D) \\ RTO &= A + 4D\end{aligned}$$

Where M represents the value of a measured RTT, A represents the average of the measured RTT, the update of the A value is shown in the second equation, and the D value is the estimated variance of the RTT, and its update is shown in the third equation. In the second and third equations, g and h are constants, generally g is 1/8, and h is 1/4. The value is taken in this way to facilitate the calculation, and it can be seen from the back that the above calculation can be completed by a simple shift operation. As shown in the fourth formula for the calculation of RTO, at the beginning, the RTO is taken as a value of 6, that is, 3s, the A value is 0, and the D value is 6. Now let's make a simple change to the above four expressions to get the expression for calculating RTT in LWIP:

$$\begin{aligned} \text{Err} &= M - A = A + \\ \text{Err}/8 &= A + (M - A)/8 \longrightarrow 8A = 8A + M - A \\ D &= D + (|\text{Err}| - D)/4 = D + (|M - A| - D)/4 \longrightarrow 4D = 4D + (|M - A| - D) \\ \text{RTO} &= A + 4D \end{aligned}$$

Let $sa = 8A$, $sv = 4D$, these are the two fields in the TCP control block. Bring in the above transformed expression to get:

$sa = sa + M - sa \gg 3$
 $sv = sv + (|M - sa \gg 3| - sv \gg 2)$
 $\text{RTO} = sa \gg 3 + sv$
 So we get the RTO value that we care about the most, and there is another question: how to get the M value? M represents the value of the RTT measured at one time, and in LWIP it is the current `tcp_ticks` value of the system minus the `tcp_ticks` value when the packet is sent. `tcp_ticks` is also added by 1 to the kernel's 500ms periodic interrupt handling.

If you're still awake at this point, then there's nothing wrong with understanding RTT. Let's take a look at how the source code does RTT estimation, which is also done in function `tcp_receive`. if (`pcb->rttest && TCP_SEQ_LT(pcb->rtseq, ackno)`) { // There is RTT in progress and the data segment is acknowledged

```
m = (s16_t)(tcp_ticks - pcb->rttest);    Calculate the M-value

m = m - (pcb->sa >> 3); M -
sa >> 3; pcb->sa += m; //更新 saif
(m < 0) {

    m = -m;    // |M - with >> 3 |
} m = m - (pcb->sv >> 2); // (|M - sa >> 3| -
sv >> 2); pcb->sv += m; // 更新 sv; pcb->rto =
(pcb->sa >> 3) + pcb->sv; 计算 rto; pcb-
>rttest = 0; // 停止 RTT 估计
```

This code is basically a translation of the formula mentioned earlier, no explanation! There is also such a problem, when a packet is sent with a certain RTO as the timeout value, and the data segment is not acknowledged after the RTO time, the packet is retransmitted, and if the acknowledgment of the packet is not received after retransmission, in this case, what should the protocol stack do, is it to resend the packet according to the original RTO every time? The answer is no, because when multiple retransmissions fail, it is likely that the network is blocked or blocked, and if a large number of retransmission packets are put into the network at this time, this will inevitably make the problem more and more serious, and the data packets may be blocked in the network forever and cannot reach the destination. As described in the standard, LWIP does this: if the resent packet times out, the next resent packet must be exponentially avoided by 2, that is, the RTO value is set to 2 times the previous one, and the packet is no longer resent after a certain number of retransmissions. This is done in the 500ms timing function `tcp_slowtmr`, look at the source code. if (`pcb->rtime >= 0`) // If the retransmission timer is on

```
++PCB->rtime;    then increase the value of the timer
if (pcb->unacked != NULL && pcb->rtime >= pcb->rto) { //If the timer times out and there is data not confirmed
    if (pcb->state != SYN_SENT) { // No avoidance treatment is given to data in the SYN_SENT state
```

```

        PCB->RTO=((PCB->with >> 3) + PCB->SV) << tcp_backoff[PCB->NRTX];    RTO value concessions
    }pcb->rttime
    = 0;                          Reset the timer
    .....
    tcp_rexmit_rto(pcb Retransmit data
    );
}

```

There are three things I want to say in this small piece of code: First, the control block in the SYN_SENT state does not avoid the timeout, which may be due to the fact that the SYN_SENT state generally sends out a SYN handshake package, and it is resent according to a fixed RTO value every time. I don't understand, it seems that the standard is also avoided! The second point is that avoidance is implemented by shifting an array tcp_backoff, tcp_backoff defined as follows: const u8_t tcp_backoff[13] = { 1, 2, 3, 4, 5, 6, 7, 7, 7, 7, 7, 7, 7}; Here, when there are more than 6 repasses, the RTO value will no longer be avoided. The last point is the function tcp_rexmit_rto, which really does the retransmission of packets: void tcp_rexmit_rto(struct tcp_pcb *pcb) {

```

    struct tcp_seg *seg; if
    (pcb->unacked == NULL) {

    return;}for(seg = pcb->unacked; seg->next != NULL;
    sig = seg->next);

```

Put all the unacked queues

arrive

```

    seg->next = pcb->uncent;                          unsent queue frontend
    pcb->unsent = pcb->unacked; pcb->unacked = NULL; pcb->snd_nxt = ntohl(pcb-
    >unsent->tcphdr->seqno); The next data number to be sent points to the queue

```

```

// unsent data segment at the beginning
++pcb->nrtx;    The number of repasses is increased by 1
pcb->rttest = 0; No RTT estimation is performed during retransmission of packets
tcp_output(pcb); Send a packet
}

```

From this code and the avoidance algorithm above, you can clearly see the role of the field nrtx, which is an important variable for setting the RTO value when retransmitting multiple times. Also note that RTT estimation should not be performed during retransmission, as estimates in this case are often inaccurate. This is the legendary Karn algorithm, which believes that because a packet is about to be retransmitted, the timing of the message is also meaningless. Even if an ACK is received, it is not possible to distinguish whether it is an acknowledgment of the first or second packet. Therefore, TCP only times unretransmitted packets. Another thing to note is that after the above code, the value of snd_nxt is likely to be less than the value of snd_max, I believe you can vaguely feel the effect of snd_max, haha, it's over!

This section discusses slow start algorithms versus congestion avoidance algorithms. Copyright Notice: Most of the content of the following paragraphs is from "TCP/IP Detailed Explanation, Volume 1: Protocol", if there is any similarity, it is purely no coincidence!

If all of the previous discussions have been discussed, the sender can initially send multiple message segments to the network until the size of the receiver's announcement window is reached. This is possible when the sender and receiver are on the same LAN. However, if there are multiple routers and slower links between the sender and receiver, some problems can arise. Intermediate router cache packets can consume significant time and memory space overhead, which can significantly reduce the throughput of TCP connections.

To solve this problem, TCP needs to use a slow start algorithm. The algorithm works by observing that the rate at which new packets enter the network should be the same as the rate at which the other end returns acknowledgments. Slow start adds another window to the sender's TCP: the congestion window, which is noted as *cwnd*. When a TCP connection is established with a host on another network, the congestion window is initialized to 1 packet segment (i.e., the size of the packet segment advertised on the other end). For each ACK received, the congestion window adds a packet segment. The sender takes the minimum value in the congestion window and the advertisement window as the maximum sending limit.

The congestion window is the flow control used by the sender, while the advertisement window is the flow control used by the receiver. The sender starts by sending a packet segment and then waits for the ACK. When the ACK is received, the congestion window increases from 1 to 2, which means that two packet segments can be sent. When ACKs are received for both segments, the congestion window increases to 4. It is an exponential incremental relationship.

When it comes to slow start algorithms, we have to talk about congestion avoidance algorithms, which are usually implemented together in practice. Usually slow start algorithms are the way the sender controls the data flow, but sometimes the sender's data flow reaches the limits of the intermediate router, at which point the packet is discarded. A congestion avoidance algorithm is a method of handling lost packets. The algorithm assumes that the loss caused by packet corruption is very small (much less than 1%), so in the sender's eye, packet loss means congestion on the network somewhere between the source and destination hosts. The sender can determine packet loss in two ways: packet transmission timeout and receipt of duplicate acknowledgments.

When congestion occurs, we want to reduce the transmission rate of packets entering the network, and the sender actively adjusts the rate of packets entering the network through slow start and congestion avoidance algorithms, and the sender also passively adjusts the packet transmission rate through the window size announced by the receiver.

The congestion avoidance algorithm and the slow start algorithm need to maintain two variables for each connection: a congestion window *cwnd* and a slow start threshold *ssthresh*. The algorithm is generally carried out as follows: 1) For a given connection, the initialization *cwnd* is 1 packet segment, and the *ssthresh* is 65535 bytes. 2) The output of the TCP output routine cannot exceed the size of the *cwnd* and receiver advertisement window. Congestion avoidance is the flow control used by the sender, while the advertisement window is the flow control performed by the receiver. The former is an estimate of the network congestion felt by the sender, while the latter is related to the amount of cache available to the receiver on that connection. 3) When congestion occurs (timeout or duplicate acknowledgment is received), *SSTHRESH* is set to half the size of the current window (minimum value of *CWND* and receiver advertisement window size, but minimum of 2 message segments). Also, if the timeout is causing congestion, the *cwnd* is set to 1 packet segment (this is slow start). 4) *Cwnd* is increased when new data is confirmed by the other party, but the method of increasing depends on whether we are doing slow start or congestion avoidance. If *cwnd* is less than or equal to *ssthresh*, slow start is in progress, otherwise congestion avoidance is in progress. The slow start lasts until we are halfway back to where we were when the congestion occurred (because we recorded half the size of the window that caused us trouble in step 2) and then switch to performing congestion avoidance.

The slow start algorithm initially sets the *cwnd* to 1 packet segment, and then adds 1 for each confirmation received, which causes the window to press

Exponential growth: 1 message segment is sent, then 2, then 4 The congestion avoidance algorithm requires that the cwnd be incremented by 1/cwnd each time an acknowledgment is received. This is an additive increase compared to the exponential increase of slow start. We want to add up to 1 message segment to the cwnd in a round-trip time (regardless of how many ACKs are received in this RTT), but slow start will increase the cwnd according to the number of acknowledgments received during this round-trip time.

After talking for a long time, I summarized the slow start algorithm and the congestion avoidance algorithm in one sentence: each TCP control block has two fields, cwnd and ssthresh, and when the data is sent, the sender can only take the smaller of the cwnd and receiver announcement window size as the upper limit of the send. When a confirmation is returned, if the cwnd value is less than or equal to ssthresh, the slow start algorithm is performed, that is, every confirmation received, cwnd is added by 1; if the cwnd value is greater than ssthresh, the congestion avoidance algorithm is performed, that is, every confirmation received, cwnd is added by 1/cwnd, which ensures that the cwnd increase value does not exceed 1 within an RTT estimate. When congestion occurs when cwnd is increased to a certain value, update the values of cwnd and ssthresh as shown in 3). Halo, MS is not a sentence, it is a paragraph!

Here we follow the steps described in the algorithm above to see how LWIP describes the whole process. 1) Given a connection, initialize its cwnd with ssthresh. cwnd is both initialized to 1, while the ssthresh field is in the guest

The client side is slightly different from the server side, performing an active connection on the client side, calling the function tcp_connect, where the relevant field values are set:

```
PCB->CWND = 1; PCB->
>SSTHRESH = PCB->MSS * 10;
PCB->state = SYN_SENT;
```

After sending out the SYN packet and receiving the return SYN+ACK from the server, the client makes further settings for the relevant fields in function tcp_process:

```
pcb->ssthresh = pcb->mss * 10; pcb->cwnd = ((pcb->cwnd == 1) ? (pcb->mss
* 2) : pcb->mss); On the server side, make a passive connection and set
the values of the relevant fields in the function tcp_listen_input:
```

```
npcb->snd_wnd = tcp_hdr->wnd;
npcb->ssthresh = npcb->snd_wnd;
```

You can see that on the server side, ssthresh is set to the size of the receiving window for the other party's announcement. 2) The output of the TCP output routine cannot exceed the size of the cwnd and receiver advertisement window. In the tcp_output function is

Use the following code to send the data segment:

wnd = LWIP_MIN(pcb->snd_wnd, pcb->cwnd); Takes the Valid Send Window size value

sig = PCB->uncent; Obtain the first send data segment

```
..... while (seg != NULL && ntohl(seg->tcp_hdr->seqno) - pcb->lastack +
seg->len <= wnd)
```

```
{ The data segment is within the valid send window
```

```
pcb->uncent = seg->next;
```

```
tcp_output_segment(say, PCB); Data segments are sent
```

```
..... sig = PCB->
uncent;
```

Gets the next send data segment

}3) When congestion occurs, you need to update the values of cwnd and ssthresh. If the congestion is caused by a timeout, the corresponding update code is implemented in tcp_slowtmr, as shown in the following familiar code, which we also see in the timeout retransmission:

```
if(pcb->rtime >= 0)
```

```
++pcb->rtime; if (pcb->unacked != NULL && pcb->
>rtime >= pcb->rto) {
```

Timeout occurs

```
..... Sections related to timeout retransmission
```

`eff_wnd = LWIP_MIN(pcb->cwnd, pcb->snd_wnd);` Get the effective send size of the timeout `pcb->ssthresh = eff_wnd >> 1;` ssthresh set to half of the valid size if `(pcb->ssthresh < pcb->mss){` // Fix ssthresh to have at least 2 message segments

`pcb->ssthresh = pcb->mss * 2;` } `pcb->cwnd = pcb->mss;` Set the cwnd size to 1 packet segment size} If the congestion is caused by repeated acknowledgments, the corresponding update code is implemented in the `tcp_receive` function, which involves the knowledge of the fast recovery and retransmission part, which we will focus on later.

```
if (pcb->lastack == ackno) {
    ..... IF (PCB->CWND >
    PCB->snd_wnd)
                                ssthresh is set to half of the valid send window

    PCB->SSTHRESH = PCB->snd_wnd/2;
    else
        pcb->ssthresh = pcb->cond/2;
        IF (PCB->SSTHRESH < 2*PCB->mss) { Fix ssthresh to at least 2 message segments

    pcb->ssthresh = 2*pcb->mss;} pcb->cwnd = pcb-
    >ssthresh + 3 * pcb->mss; // 这里不解? .....
```

4) Update the values of cwnd and ssthresh when the new data is confirmed by the other party. if `(TCP_SEQ_BETWEEN(ackno, pcb->lastack+1, pcb->snd_max))`{.....

```
if (pcb->state >= ESTABLISHED) {
    if (pcb->cwnd < pcb->ssthresh) { //Slow start is performed when it is less than, is it less than or equal to in the standard?
        if ((u16_t)(pcb->cwnd + pcb->mss) > pcb->cwnd) { // 能否增加
            PCB->CWND+= PCB->MSS; Increase the segment size
        }
    }
    else { The following is done to avoid congestion
        u16_t new_cwnd = (pcb->conduit + pcb->mss * pcb->mss/pcb-
        >cond); IF (new_cwnd > PCB->cwnd) { // 能否增加
    pcb->cwnd = new_cwnd;}} .....}
```

This code is basically a direct translation of the fourth step mentioned above. It should be noted that why is `new_cwnd` calculated using the above equation? That's because the size of cwnd increases or decreases in terms of the size of the mss, and nothing else is explained.

TCP Fast Recovery Retransmission and Fast Recovery Retransmission and Fast Recovery Retransmission and Nagle Algorithm Algorithm Algorithm Algorithm

As mentioned earlier, when an out-of-order packet segment is received, the packet segment will be hooked to the ooseg queue and an ACK (expected next byte) will be returned to the sender, and it is obvious that this ACK must be a duplicate ACK, and there will be no delay when this duplicate ACK is sent. The receiver uses this duplicate ACK to let the other party know that it has received an out-of-order message segment and to tell the other party the serial number they want to receive.

However, it is impossible for the sender to know whether a duplicate ACK is caused by a missing message segment or by a reordering of only a few message segments. So we need to wait for a small number of duplicate ACKs to arrive. If this is just a reordering of some message segments, only 1 ~ 2 duplicate ACKs can be generated before the reordered segments are processed and a new ACK is generated. If you receive a series of 3 or more duplicate ACKs, it is very likely that one of the packet segments is missing. So we retransmit the lost packet segments without waiting for the timeout timer to overflow. This is the fast retransmission algorithm. In the previous section, when talking about timeout retransmission, it was mentioned that when a timeout occurs, ssthresh is set to half of the valid send window, and cwnd is set to a packet block size, that is, the slow start algorithm is executed. Here, when the fast retransmission is executed, the next execution is not the slow start algorithm but the congestion avoidance algorithm, which is the so-called fast recovery algorithm.

The reason for not performing a slow start after a fast retransmission is that receiving a duplicate ACK doesn't just tell us that a packet is missing. And since the receiver only generates duplicate ACKs after receiving another segment and hooking that segment to the ooseg queue, this means that there is still data flowing between the sender and receiver ends, and we don't want to perform a slow start to suddenly reduce the flow of data.

The steps of this algorithm described in Volume 1 are as follows: 1) When the 3rd duplicate ACK is received, set the ssthresh to half of the current congested window cwnd. Retransmit lost message segments. Set the cwnd to ssthresh plus 3 times the packet segment size. 2) Each time another duplicate ACK is received, cwnd increases the segment size by 1 and sends 1 packet (if the new cwnd allows sending). 3) When the next ACK confirming the new data arrives, set the cwnd to ssthresh (the value set in step 1). This ACK should be a confirmation of the repass in step 1 within one round-trip time after the repass is made. In addition, this ACK should also be an acknowledgment of all intermediate message segments between the missing packet and the 1st duplicate ACK received.

LWIP also implements fast recovery and retransmission in function tcp_receive, as shown below, and the whole process is basically the same as described in the above algorithm.

```
if (pcb->lastack == ackno) {  If the ACK is a duplicate ACK
    pcb->acked = 0;  The number of data confirmed by the ACK is 0
    if (pcb->snd_wl1 + pcb->snd_wnd == right_wnd_edge){ // if no window update is made
        ++pcb->dupacks;  The number of duplicate confirmations received adds 1
        if (pcb->dupacks >= 3 && pcb->unacked != NULL) { //如 1) 所述, 三个以上充复 ACK
            if (!(pcb->flags & TF_INFR)) { // Fast retransmission is not turned on at this time, i.e. 3 times for dupacks
                The call function retransmits the missing packet segment if (pcb->cwnd > pcb->snd_wnd) // ssthresh set to half of the valid send window
                PCB->SSTHRESH = PCB->snd_wnd/2;
                else
                    pcb->ssthresh = pcb->cond/2;
                IF (PCB->Ssthresh < 2*PCB->mss) {  Corrected the ssthresh value to a minimum of 2 message segments
                    pcb->ssthresh = 2*pcb->mss;}
            }
```

```
pcb->cwnd = pcb->ssthresh + 3 * pcb->mss; cwnd is ssthresh+3*packet
segment size pcb->flags |= TF_INFR; Set fast retransmission flag}else// Fast
retransmission has started, i.e. dupacks greater than 3 times {
```

```
    if ((u16_t)(pcb->cwnd + pcb->mss) > pcb->cwnd) // Fast retransmission has begun, as in 2)
pcb->cwnd += pcb->mss; For each duplicate ACK received, cwnd adds 1 packet
segment large;// This is different from what is described in 2), where no 1 packet is
sent after receiving a duplicate ACK} // if dupacks are greater than 3} //if if no window
update is made} // if if duplicate ACKelse if (TCP_SEQ_BETWEEN(ackno, pcb-
>lastack+1, pcb->snd_max)){ // New data confirmed
```

```
    if (pcb->flags & TF_INFR) { is in a state of fast retransmission,
        pcb->flags &= ~TF_INFR;    Clear the fast retransmission flag
        pcb->cwnd = pcb->ssthresh;    Set the value of cwnd as shown in 3).
    }....., pcb-
    >dupacks = 0;
```

Clear duplicate confirmation flags

```
pcb->lastack = ackno; Record ackno.....} There are two places to explain in the
above code: one is to call the function tcp_rexmit to retransmit the missing packet
segment, which is similar to the function tcp_rexmit_rto mentioned in the previous
section, all of which are retransmission packets. This function functions by placing
the first data segment on the unacked queue to the top of the unsent queue and
calling the function tcp_output output packets. The second thing to note is that
after receiving more than three duplicate ACKs, the code simply increases the value
of cwnd by one packet size, and does not send a packet as described in 2) above.
```

That's all for fast retransmission and recovery, and here is the Nagle algorithm part. Window-based flow control schemes can lead to a condition known as "Silly Window Syndrome." SWS occurs when a TCP receiver advertises a small window and the TCP sender immediately sends data to fill the window, when a small packet segment is acknowledged, the window is opened again with a smaller unit and the sender sends a small packet segment to fill the window again. This results in the TCP stream containing very small segments instead of full-length segments. Confused window syndrome is a TCP phenomenon that can cause significant degradation of network performance, as the fields of IP header and TCP header take up most of the space in small data segments, while there is very little real TCP data.

This phenomenon can be caused by either end of the TCP connection. This is because the receiver can advertise a small window (instead of waiting until a large window is available), and the sender can send a small amount of data (instead of waiting for other data to send a large message segment).

To avoid SWS, it is necessary to find a way to eliminate this situation on both the sender and receiver. The receiver does not have to advertise the small window update, and the sender does not have to send a small packet segment when only the small window is available. Measures can be taken at either end to avoid the phenomenon of confused window syndrome.

The receiver's solution to SWS is that the receiver does not advertise a small window. The usual algorithm is that the receiver does not advertise a window that is larger than the current window (which can be 0) unless the window can increase the size of the packet segment (i.e., the MSS to be received) or can increase the receiver's cache space by half, regardless of how much it actually has.

The sender's way to avoid confusion window syndrome is to send data only when one of the following conditions is met:

(a) A full-length message segment can be sent; (b) can send a message segment that is at least half the size of the receiver's notification window; (c) Any data can be sent and you don't want to receive an ACK (i.e., we don't have data that has not yet been acknowledged) or the Nagle algorithm cannot be used on that connection.

Condition (b) deals primarily with hosts that always advertise a small window (perhaps smaller than 1 message segment) and requires the sender to always monitor the maximum window size advertised by the other party, an attempt by the sender to guess the size of the cache received by the other party. While the size of the received cache may decrease when the connection is established, this is rare in practice.

Condition (c) allows us to avoid sending small packet segments when there is data that has not yet been acknowledged (waiting to be acknowledged) and when the Nagle algorithm cannot be used. If the application process is writing small data (e.g., smaller than the packet segment), condition (c) can avoid the confusion window syndrome.

These three conditions also allow us to answer the question: If the Nagle algorithm prevents us from sending small packet segments in the absence of unconfirmed data, how small is it considered small? From condition (a), it can be seen that the so-called "small" refers to the number of bytes less than the size of the message segment MSS.

In LWIP, SWS is naturally avoided on the sender side, because TCP packet segments are not known to the advertised receiver window when they are established and queued. In large data volume transmission, the output queue will include the largest packet segment. This means that if the TCP receiver advertises a small window, the sender will not send the first message segment in the queue because it is larger than the advertised window. Instead, it will wait until the window has enough space for it. When being a TCP receiver, LWIP will not advertise a receiver window that is less than the maximum packet segment size allowed by the connection (this is not understood).

Let's take a look at how LWIP makes way to avoid confusing the window when sending data segments, and the following code has been cut down to a certain extent, leaving only the parts related to the algorithm.

```
sig = PCB->unsent; 取得第一个数据段while (seg != NULL && ntohl(seg->tcphdr->seqno) - pcb->lastack + seg->len <= wnd) { //整个
```

Whether the data segment is within the valid send window

```
    if((tcp_do_output_nagle(pcb) == 0) && // The nagle algorithm prevents the packet from being sent
((pcb->flags & (TF_NAGLEMEMERR | TF_FIN)) == 0)){ // There are memory error flags and
breaks; FIN package flag, the nagle algorithm fails}..... pcb->unsent = seg->next; Record
the next data segment tcp_output_segment (SEG, PCB); Sending Data Segments..... seg =
pcb->unsent; Get the next data segment} in just a few sentences to implement the nagle
algorithm? There's so much to explain! The first is the while loop condition, which requires
that all data sequence numbers in the data segment to be sent must be in the valid send
window. In this case, if the other party announces a small window and the data segment
sent is large, the data segment will not be sent, which can be regarded as a deformation of
conditions (a) and condition (b) above. For other cases, the Nagle algorithm can be used to
determine whether a packet is output. At the same time, if the flags are placed with the
TF_NAGLEMEMERR and TF_FIN flags, the Nagle algorithm is invalid, that is, the packet will
not be blocked by the Nagle algorithm. TF_NAGLEMEMERR can be understood as
indicating a memory error, which is placed when the sending cache space is insufficient
when the tcp_enqueue function assembles the packet, and when this happens, the
assembled packet needs to be sent out as soon as possible, so the Nagle algorithm is
invalid when the flag is placed; TF_FIN flag is placed to indicate that the upper application
has issued a command to close the connection, so the data segment on the connection
should also be sent as soon as possible, and the Nagle algorithm will also be invalid in this
case. The Nagle algorithm is implemented through a macro tcp_do_output_nagle, as
follows: #define tcp_do_output_nagle(tpcb) (((tpcb)->unacked == NULL) || \
```

```
((tpcb)->flags & TF_NODELAY) || \(((tpcb)-
>unsent != NULL) && ((tpcb)->unsent->next != \
NULL))) ? 1 : 0 )
```

Don't be intimidated by so many parentheses, we only care about the case when the above equation equals 0, i.e. when the Nagle algorithm prevents packets from being sent. Specifically, when the unacked queue is not empty, and the Nagle algorithm is enabled (TF_NODELAY is not placed), and there are less than two data segments to be sent in the unsent queue, the tcp_do_output_nagle value is 0, and the tcp_output jumps out of the while loop and does not send any data segments.

The above paragraph can also be described as (c), when there is no data that has not yet been confirmed (the unacked queue is empty), or when the Nagle algorithm is not enabled, or when there are two or more data segments on the unsent queue. There is an additional limit on the number of packets on the unsent queue than in (c), and we can think of this extra limit as another interpretation of condition (a) and condition (b).

In many cases, the Nagle algorithm needs to be disabled, which is achieved by setting the TF_NODELAY flag in the flags field of the control block.

As mentioned earlier, the receiver's solution to SWS is not to advertise a small window. However, if LwIP is used as the receiver, it does not seem to have implemented this function yet, that is, it may advertise small window information to the sender. Haven't I understood it yet...

This section explains TCP's persistence timer and keep-alive timer, starting with the persistence timer. The receiver of TCP tells the sender the number of bytes of data it can receive by announcing the size of the window, and the receiver uses this method for traffic control. What happens if the receiver advertises a window size of 0? This effectively prevents the sender from delivering data until the advertisement window becomes non-zero.

When the sender receives a 0 window advertisement, the data segment is stopped until the receiver passes through a non-0 window. It is important that TCP can handle packet loss with a new non-0 window advertisement, which is usually sent in an ACK packet with no data. ACK transmission is unreliable, that is, TCP does not acknowledge ACK fragments (obviously, there will be no retransmission of the ACK segment), and TCP only acknowledges those ACK segments that contain data.

If an acknowledgment is lost, both parties risk terminating the connection by waiting for each other: the receiver is waiting to receive the data (because it has already announced a non-zero window to the sender), and the sender is waiting for the non-0 window to update that allows it to continue sending data. To prevent this deadlock from occurring, the sender uses a persist timer to periodically query the receiver to find out if the window has increased. These segments of packets sent from the sender are called window probes.

There are two fields in the control block related to the sticking timer: `persist_cnt` and `persist_backoff`. `persist_cnt` Used to persist the timer count, when the count value exceeds a certain value, a window probe packet is issued. `persist_backoff` indicates that the insistence is whether the timer has been started (whether it > 0) and that several probe packets have been sent (`persist_backoff` when it is an integer greater than 0). If the insistence that the timer has been started, the `tcp_slowtmr` of the interrupt handler will do the following at the kernel 500ms:

```
if (pcb->persist_backoff > 0) { // if the insist timer is on
    pcb->persist_cnt++;    Increase the count value
    if (pcb->persist_cnt >= tcp_persist_backoff[pcb->persist_backoff-1]) { // The count value exceeds
        When a certain number is at the upper limit, a window probe is performed

        pcb->persist_cnt = 0; Reset the Count Value if (pcb->persist_backoff <
        sizeof(tcp_persist_backoff)) { // Increase the upper limit of the Count Value
```

```
pcb->persist_backoff++; } tcp_zero_window_probe(pcb); Sending a window probe
pack }}
```

There are two points to mention. One is the array `tcp_persist_backoff`, which holds the upper limit of the count value of a series of stick timers, and `persist_backoff` is the index of that array. That is, after sending the first probe packet for 3 interrupts of 500ms (1.5s), and after sending the second probe packet for 6 times (3s), the time interval becomes 120 interrupts (60s) when the probe packet is sent for the sixth time and above.

```
const u8_t tcp_persist_backoff[7] = { 3, 6, 12, 24, 48, 96, 120 };
```

Let's take a look at how the function `tcp_zero_window_probe` does window probing. The `tcp_zero_window_probe` function is simple, assemble a piece of TCP packet containing one byte of data and send it out, which is obtained from the unacked or unsent queue, and the data sequence number field of the window probe packet is filled with the data sequence number of this byte. So when both queues are empty, it means that there is no data to be processed, and of course there is no need to perform window probing; while

When this byte of data is obtained from the unacked queue, since the queue has already been sent, when the corresponding window probe reaches the receiver, it will be regarded as a duplicate packet without relevant data processing, and only an ACK packet will be returned to the sender. When this byte of data is obtained from the unsent queue, the byte data will be hooked to the ooseq queue or directly submitted to the parent application when it reaches the receiver, and when the sender window allows, the first byte of the first data segment in the unsent queue will be sent and will appear to be duplicate in the eyes of the receiver, and the receiver will be able to detect this duplicate byte and delete the byte data directly. It can be seen from the whole process that the 1 byte of data in the window probe package does not affect the entire data transmission process.

When to start a window probe? This is done at the end of the function `tcp_output`. When the unsent queue is not empty after the data segments that can be sent are sent, and the window probe is not started, and the current window is too small to send the next data segment, the window probe must be started.

```
if (sig != NULL && pcb->persist_backoff == 0 &&
    ntohl(seg->tcphdr->seqno) - pcb->lastack + seg->len > pcb-
    >snd_wnd) {pcb->persist_cnt = 0; // 复位计数pcb-
    >persist_backoff = 1; // 开始窗口探查}
```

When to stop a window of probing? As you have already known, at the beginning of function `tcp_receive`, the send window will be updated according to the packet received, that is, if a non-0 window is detected, the window probe will be stopped, as shown below.

```
IF (TCP_SEQ_LT(PCB->snd_wl1, Seqno) ||
    (pcb->snd_wl1 == seqno & TCP_SEQ_LT(pcb->snd_wl2, ackno)) || (PCB-
    >snd_wl2 == ACKNO & TCPHDR->WND > PCB->snd_wnd)) { // 若满足窗口跟新条件
    pcb->snd_wnd = tcphdr->wnd; 窗口更新pcb-
    >snd_wl1 = seqno; pcb->snd_wl2 = ackno; if
    (PCB->snd_wnd > 0 & PCB->persist_backoff > 0) {
```

A non-0 window is detected and probe is turned
on

```
    pcb->persist_backoff = 0; Stop window probing
```

```
}} Let's look at the timer again. If neither side of a TCP connection that is
already in a steady state is sending data to the other, no information is exchanged
between the two TCP modules. However, many times, both parties want to know if the
other person's is inactive. A common situation is that a server wants to know if
the client host crashes and shuts down or crashes and restarts, and the keep-alive
timers provided in many TCP/IP implementations can provide this detection.
```

The keep-alive function is mainly provided for server applications. The server application wants to know if the client host is crashing, so that it can allocate resources to the customer. If a given connection does not take any action within two hours, the server sends a probe segment to the customer. The client host must be in one of four states: 1) The client host is still up and running and reachable from the server. The customer's TCP response is fine, and the server knows that the other party is working. The server resets the keep-alive timer after two hours and sends a probe packet. If application traffic passes through this connection before the two-hour timer arrives, the timer resets to send a probe message in the next 2 hours after the data is exchanged. 2) The client host has crashed and is shut down or restarting, in which case the client's TCP will not respond. The server will not be able to receive a response to the probe and will wait 75 seconds before it times out, and the server will send 9 such probe packets at 75 second intervals in the future. If the server does not receive a response, it assumes that the client host has gone down and terminates the connection. 3) The client host crashes and has been restarted. The server will receive a response to its keep-alive probe, but this

The response is a reset that causes the server to terminate the connection. 4) The client host is functioning normally, but it is not reachable from the server. This is the same as state 2, because TCP cannot tell the difference between state 4 and state 2, and all it can find is that it has not received a response from the probe.

In case 1, the server's application does not sense the keep-alive probe to occur. The TCP layer takes care of everything, and this process is invisible to the application. When scenario 2, 3, or 4 occurs, the server application receives an error report from its TCP layer (typically the server application makes a read request to the network and then waits for data from the customer. If the keep-alive function returns an error, the error is returned to the application as the return value of the read operation). In the second case, the error is a message such as "connection timed out", and in the third case, "the connection was reset by the other party". The fourth scenario looks like a connection timeout, and it can also be determined by whether it is caused by an unattainable objective based on whether or not an ICMP error message related to the connection is received.

This brings in to four fields in the TCP control block: keep_idle, keep_intvl, keep_cnt, and keep_cnt_sent. Where keep_intvl and keep_cnt are related to the compile option LWIP_TCP_KEEPALIVE, and when the compile option is 1, the keep_intvl and keep_cnt fields are used to hold the user-defined lifetime option values, respectively, which are discussed later. In practice, we can use the system's default retention time option values, so if we set the LWIP_TCP_KEEPALIVE to 0, the keep_intvl and keep_cnt fields will not be compiled, and naturally they are not within our scope of discussion.

The keep_idle field records how long it took to perform the keep-alive probe, typically 2 hours, and the keep_cnt_sent field indicates the number of keep-alive packets that have been sent. In addition to these two fields, there are several macro definitions with the default revival time option values: #defineTCP_KEEPIDLE_DEFAULT720000UL // Hold-up time milliseconds (2 hours) #defineTCP_KEEPINTVL_DEFAULT75000UL // Number of milliseconds between time intervals for continuous revival packs (75s) #defineTCP_KEEPCNT_DEFAULT9U // The number of times the keepal package is sent repeatedly #defineTCP_MAXIDLETCP_KEEPCNT_DEFAULT * TCP_KEEPINTVL_DEFAULT

The time consumed to perform a keep-alive probe

Users can allow the keep_intvl and keep_cnt fields through macro LWIP_TCP_KEEPALIVE, which are used to record user-defined Savings intervals and number of Savings Packs, respectively. When the custom values are not used, the two DEFAULT values above are used as the keep-alive option values.

Here, we analyze the entire process of keeping alive using the system's default hold-alive option value, where the field keep_idle is set to a value of TCP_KEEPIDLE_DEFAULT. Keeping alive processing is also done in the kernel 500ms interrupt handling function tcp_slowtmr. There is also a field in the TCP control block to mention again, namely the tmr records the system time tcp_ticks value when the most recent data segment on that TCP connection arrived. if((pcb->so_options &SOF_KEEPALIVE) & // If the keep-alive function is enabled, the data interaction status is stable

```

        ((pcb->state == ESTABLISHED) || (pcb->state == CLOSE_WAIT))) {
if((u32_t)(tcp_ticks - pcb->tmr) >    Disconnect after 2 hours + 9*75 seconds
    (PCB->keep_idle + TCP_MAXIDLE) / TCP_SLOW_INTERVAL) {

    tcp_abort(pcb);    Disconnect
} else if((u32_t)(tcp_ticks - pcb->tmr) > // Send a survival
pack within 2 hours + 9*75 seconds

    (PCB->keep_idle + PCB->keep_cnt_sent * TCP_KEEPINTVL_DEFAULT) /
    TCP_SLOW_INTERVAL) {

    tcp_keepalive(pcb);    Send a survival pack
    pcb->keep_cnt_sent++; Number of Survival
    Packs plus 1}}

```

The `tcp_abort` function is used to free up a connection, and its main work consists of removing the control block from the corresponding TCP linked list, freeing up the memory space occupied by the data if there is still data on the connection, and finally sending an RST packet to the other party. The `tcp_keepalive` function is used to send a live-saving package, which is only a TCP header and does not contain any data, so it will not affect the other party's data reception.

The two hours of idle time for the keep-alive option can be changed. Users can define the values of `keep_idle` themselves, but it is generally not recommended to modify these values. Don't change this unless you know what you're doing! Ha ha...

26 TCP Timer

This section discusses TCP's timing functions. In the previous discussion, we saw various timers with TCP, including retransmission timers, sustained timers, and keep-alive timers, in addition to several timers in TCP that we have not covered yet. Let's take a look at the various timers in TCP in general. TCP establishes a total of seven timers for each connection, in that order: 1) The connection establishment timer starts when a new connection is established in the sending SYN packet segment. If no response is received within 75 seconds, the connection establishment is aborted. 2) The "retransmission" timer is set when TCP sends a certain data segment. If the timer times out and the acknowledgment from the peer has not yet arrived, TCP retransmits the data segment. The value of the retransmission timer (i.e., the time TCP waits for the peer to acknowledge) is calculated dynamically and is closely related to the RTT estimate, and also depends on the number of times the segment has been retransmitted. 3) The "delayed ACK" timer is set when TCP receives data that must be acknowledged but does not need to be acknowledged immediately. If within 200ms, data is sent over the connection, the delayed ACK response can be sent back to the peer with the data, known as piggyback acknowledgment. If after 200ms, the confirmation fails to be carried out, the timer times out and an immediate confirmation needs to be sent. 4) The "persist" timer is set when the connection peer notification receiver window is 0 to prevent TCP from continuing to send data. Because the window advertisement sent by the connection peer is unreliable (only the data is acknowledged, the ACK is not), subsequent window updates that allow TCP to continue sending data may be lost. Therefore, if TCP has data to send, but the peer advertises that the receive window is 0, the continuous timer starts, and after a timeout, it sends 1 byte of data to the peer to determine whether the peer receive window is open. 5) The "keep alive" timer takes effect when the SOF_KEEPAVIVE option is set in the so_options field of the TCP control block. If the connection is idle for more than 2 hours, the keep-alive timer times out, and the connection probe packet segment should be sent to the peer to force the peer to respond. If an expected response is received, TCP can determine that the peer host is working properly and TCP will no longer perform a keep-alive test until the connection is idle for more than 2 hours again. If an RST reset response is received, TCP determines that the peer host has been restarted. If several consecutive live-on tests do not receive a response, TCP assumes that the peer host has crashed, but it cannot tell whether it is a host failure or a connection failure. 6) FIN_WAIT_2 timer, when a connection transitions from FIN_WAIT_1 state to FIN_WAIT_2 state and can no longer receive any new data, FIN_WAIT_2 timer starts, set to 10 minutes. After the timer times out, it is set back to 75 seconds and the connection is turned off after the second timeout. The purpose of this timer is to avoid that if the peer does not send a FIN, a connection will be stuck in the FIN_WAIT_2 state forever (assuming TCP does not use the half-open feature). 7) TIME_WAIT Timer, also commonly known as 2MSL Timer. 2MSL refers to twice the MSL, which is the maximum packet segment lifetime. The timer starts when the connection moves to the TIME_WAIT state, i.e., the connection is actively closed. The reason why a 2MSL wait state is needed is detailed in that section of the state transition diagram. When the connection enters the TIME_WAIT state, the timer is set to 1 minute, after the timeout, the TCP control block is deleted and the port number can be reused.

Of the previous 7 timers, the retransmission timer uses the rtime field count, the sustained timer uses the persist_cnt field count, and the other five timers use the rtime field count except the delayed ACK timer, as you can see from the description above, these four timers are used when TCP is in four different states, so the four timers use the rtime field completely independently without affecting each other. The delayed ACK timer is done using the system's 250ms periodic timing.

Two timer functions are included in the LWIP: one called every 250 ms (fast timer); Another function is called every 500ms (slow timer). The delayed ACK timer is different from the other 6 timers: if a delayed ACK timer is set on a connection, the delayed ACK must be sent after the next 250ms timer timeout (actual ACK delay time between 0~250ms). The other 6 timers increase by 1 every 500 ms when the value is counted

When certain thresholds are exceeded, the corresponding action is triggered.

Let's look at the simple fast timer

handler first: void tcp_fasttmr(void) {

```
struct tcp_pcb *pcb; for(pcb = tcp_active_pcbs; pcb != NULL; pcb =
pcb->next) { //遍历整个 active 链表
    if (pcb->refused_data != NULL) { // If a control block has not yet received data
        err_t err; TCP_EVENT_RECV(pcb, pcb->refused_data, ERR_OK, err); Call
        the parent function to receive data if (err == ERR_OK) {

        pcb->refused_data = NULL; Reset the pointer if it is successfully
        received}}if (pcb->flags & TF_ACK_DELAY) { // If the control block
        has a delayed ACK timer turned on
```

```
        tcp_ack_now(pcb);    Send an immediate confirmation
pcb->flags &= ~(TF_ACK_DELAY | TF_ACK_NOW); //清除标志位}}//
if} // for
```

As can be seen from the above, the fast timer processing function mainly does two things, one is to submit the data that the upper layer has not received to the upper layer, and the other is to send the immediate confirmation data segment on the connection. Compared with the fast timer, the slow timer processing function is quite large, here we only list the parts related to each timer, and describe the other parts in pseudo-code, of course, the so-called other parts here are the parts we have already explained before. void tcp_slowtmr(void){

```
++tcp_ticks; pcb =
tcp_active_pcbs;
while (pcb != NULL) {

    pcb_remove = 0; zero-window exploration and processing
    of the control block; the timeout retransmission
    processing of the control block; if (pcb->state ==
    FIN_WAIT_2) { // FIN_WAIT_2 Timer timed out

        if ((u32_t)(tcp_ticks - pcb->tmr) >
        TCP_FIN_WAIT_TIMEOUT / TCP_SLOW_INTERVAL)
        {++pcb_remove;}} 该控制块的超时保活处理; #if
        TCP_QUEUE_OOSEQ
```

```
if (pcb->ooseq != NULL && Discard data that has not been processed in the ooseq queue for a long time
(u32_t)tcp_ticks - pcb->tmr >= pcb->rto * TCP_OOSEQ_TIMEOUT)
{tcp_segs_free(pcb->ooseq); pcb->ooseq = NULL;}
```

```
#endif /* TCP_QUEUE_OOSEQ */
    if (pcb->state == SYN_RCVD) { // SYN_RCVD 状态超时
        if ((u32_t)(tcp_ticks - pcb->tmr) >
            TCP_SYN_RCVD_TIMEOUT / TCP_SLOW_INTERVAL) {++pcb_remove; }
        if (pcb->state == LAST_ACK) { // LAST_ACK Timer timeout

            if ((u32_t)(tcp_ticks - pcb->tmr) > 2 * TCP_MSL / TCP_SLOW_INTERVAL) {
                ++pcb_remove;
            }
        }
        // If there is a timeout, delete the
        // control block; If there is no timeout,
        // the outgoing packets (poll) will be
        // periodically sent out; Retrieves the
        // next control block on the active
        // linked list; }
    } while (pcb = tcp_tw_pcb);
    // handle TIME-WAIT linked
    // list while (pcb != NULL) {
```

```
        pcb_remove = 0;
        // TIME-WAIT timer timeout
        if ((u32_t)(tcp_ticks - pcb->tmr) > 2 * TCP_MSL / TCP_SLOW_INTERVAL) {
            ++pcb_remove;
        }
    }
    // If it times out, delete the control block; Get the next control block on
    // the TIME-WAIT linked list; }
} // while } // function tail
It can be seen that the implementation
of each timer is implemented using the difference between the global variable tcp_ticks and
the tmr field. When TCP enters a certain state, the corresponding tmr field will be set to
the value of the current global clock tcp_ticks, so the above difference can effectively
represent the time TCP is in a certain state. The processing of each timer timeout is also
similar, that is, the variable pcb_remove plus 1, the pcb_remove variable is the core
variable in the timeout processing, when the timeout judgment is made for a PCB control
block, the function processes the TCP control block by judging the value of pcb_remove,
when the pcb_remove value is greater than 1, it means that there is a timeout event on the
control block, and the control block is either deleted or suspended. Note that the
retransmission timer timeout in the pseudocode does not affect the value of pcb_remove. If
you are careful, you can also see that the above code has two additional timeout events,
namely SYN_RCVD state timeout and ooseq queue data timeout, of course, these two timeout
events do not affect the implementation of the stack function.
```

Finally, let's take a look at the timeouts set by the system for each timeout, as you can see from the code above, they are implemented in each macro definition.

```
#define TCP_TMR_INTERVAL 250 // 250ms
#define TCP_FAST_INTERVAL TCP_TMR_INTERVAL // Fast timer
#define TCP_SLOW_INTERVAL (2 * TCP_TMR_INTERVAL) // Slow timer
```

```
#define TCP_FIN_WAIT_TIMEOUT 20000 // FIN_WAIT_2 Status timeout
#define TCP_SYN_RCVD_TIMEOUT 20000 // SYN_RCVD State timeout
```

```
#define TCP_OOSEQ_TIMEOUT      6U      The number of RTO cycles that data is waiting for in the OOSEQ queue
#define TCP_MSL 60000U        // MSL
```

At this point, the fields in our PCB control block are basically covered, except for polltmr and pollinterval. These two fields are used to periodically call the function tcp_output to send unsent data segments that remain on the control block.

TCP Termination vs. Nitpoint Termination and Summary Termination vs. Summary Termination vs. Summary Termination vs. Summary of Summary

TCP has one last thing to sweep. Now let's jump out of the `tcp_process` function and go back to the `tcp_input`, as mentioned earlier, `tcp_input` receives the packets submitted by the IP layer, finds the corresponding TCP control block based on the packet, and calls the function `tcp_timewait_input`, `tcp_listen_input`, or `tcp_process` according to the state of the relevant control block. If it is the first two functions called, `tcp_input` ends after the two functions return, but if the `tcp_process` function is called, the function returns and `tcp_input` has to do a lot of corresponding processing.

To continue down, we need to look at a very important global variable `recv_flags`, which I briefly mentioned earlier when I talked about TCP global variables. This variable is similar to the `flags` field in the control block and is used to describe the current state of the TCP control block. `flags` field can be set and their meanings are shown in the following macro definitions: `#define TF_ACK_DELAY(u8_t)0x01U` // Delay reply ACK packet `#define TF_ACK_NOW(u8_t)0x02U` // Send ACK packet immediately `#define TF_INFR(u8_t)0x04U` // In fast retransmission state `#define TF_FIN(u8_t)0x20U` // Local upper-level app closes connection `#define TF_NODELAY(u8_t)0x40U` // Disable Nagle algorithm prohibits `#define TF_NAGLEMEMERR(u8_t)0x80U` // Insufficient sending cache space

Now that the above fields have been basically covered, let's take a look at the various flag bits that can be set by the global variable `recv_flags` and their meanings, as shown in the following macro definition: `#define TF_RESET(u8_t)0x08U` // Received a RESET package `#define TF_CLOSED(u8_t)0x10U` // Received an ACK package in the LAST_ACK state, and the connection was successfully closed `#define TF_GOT_FIN(u8_t)0x20U` // Received FIN packets

Why use two fields to describe the state of the corresponding TCP control block? There are two reasons for my personal understanding: first, because the `flags` field itself in the control block itself is 8 bits, if each state is described, it is not enough to describe the above 9 states; Second, the above 9 described states can be clearly divided into two categories, the first 6 are closely related to TCP packet processing, and the second 3 are closely related to TCP state transition.

Before each call of `tcp_process` by `tcp_input`, `recv_flags` will be initialized to 0, and in the processing of `tcp_process`, after the relevant control block completes the state transition, the bits related to the state transition of the global variable will be placed, and after the function returns to `tcp_input`, the `tcp_input` will also subsequently process the control block according to the corresponding set `recv_flags` value.

```
if (recv_flags & TF_RESET) {
```

```
    The TF_RESET flag indicates that the peer RESET packet TCP_EVENT_ERR (pcb->errf, pcb->callback_arg, ERR_RST); If a callback function is registered
```

```
        then call the function to notify the upper level
```

```
tcp_pcb_remove(&tcp_active_pcb, pcb); Remove the control block from the linked list
memp_free (MEMTCP_PCB, pcb); Free control block memory space} else if (recv_flags &
TF_CLOSED) {TF_CLOSED indicates that the server successfully closes the connection
tcp_pcb_remove(&tcp_active_pcb, pcb); Remove the control block from the linked
list memp_free (MEMTCP_PCB, pcb); free control block memory space} else {
```

```
err = ERR_OK; if (pcb->acked > 0) { // If the packet received
confirms the data in the unacked queue
```

```
    TCP_EVENT_SENT(pcb, pcb->acked, err); You can call a custom function to send the packet
```

```

} if (recv_data != NULL) { // If the data in the
packet is successfully received
    if(flags & TCP_PSH) { //The global variable flags holds the flag field in the TCP header
recv_data->flags |= PBUF_FLAG_PUSH; Set the packet pbuf field} // to the PUSH
flag TCP_EVENT_RECV(pcb, recv_data, ERR_OK, err); Call a custom function to
receive data if (err != ERR_OK) { // If the upper layer fails to receive data

        pcb->refused_data = recv_data;    The data is staged with the Control Block refused_data field
    } } // if (recv_data !=
NULL)

    if (recv_flags & TF_GOT_FIN) {    If a FIN packet is received
TCP_EVENT_RECV(pcb, NULL, ERR_OK, err); Call a custom function to
receive data} if (err == ERR_OK) { //If processing is normal

        tcp_output(pcb); Attempts to send outbound packets
    } } //else There are two places to say, the first is the function
tcp_pcb_remove, the source code is shown below, the two more important
functions in the code are TCP_RMV and tcp_pcb_purge, I will not explain it in
detail here.

void tcp_pcb_remove(struct tcp_pcb **pcblist, struct
tcp_pcb *pcb) {
    TCP_RMV(pcblist, pcb); Remove a PCB control block from a linked list
    tcp_pcb_purge(pcb);    Empty the data buffer queue of the control block to free up memory space

    if (pcb->state != TIME_WAIT &&    If there is a delayed ACK on that control block, it is sent immediately
pcb->state != LISTEN &&pcb->flags & TF_ACK_DELAY) {pcb->flags |= TF_ACK_NOW;
tcp_output(pcb); } pcb->state = CLOSED; Another thing to note is the call to the
callback function, as shown in TCP_EVENT_XXX above. In actual applications, we
can interact with the LWIP kernel through callback functions, and when
initializing a PCB control block, we can set the initial values of the
corresponding function pointer fields in the control block, including sent,
recv, connected, accept, poll, errf, etc. In kernel processing, our pre-
registered function is called at TCP_EVENT_XXX to complete the interaction
between the application and the stack. The issue of the interface between the
application and the protocol stack is another huge project and will continue to
be the focus of discussion in the future.

```

At this point, the TCP part is basically over, of course, there are still some things in the TCP layer that have not been discussed, such as functions such as `tcp_write`. The key to TCP layer learning is to understand the mechanism of the entire TCP layer, and to read the source code on this basis, there should be no problems. From Essay 14 to Essay 27, it can be seen that TCP is too much space, and the same is true for the actual source code, which accounts for about half of the code volume of the entire protocol stack. Note that UDP is generally mentioned when talking about the TCP protocol, but I don't want to get involved in the UDP protocol here for one reason

Embedded products need to provide effective and reliable network services, and the inherent nature of UDP does not meet this requirement. Therefore, if we really want to use LWIP in our products, we basically use the TCP protocol, and in the subsequent explanation, we will see how the application uses LWIP to set up a web server, which allows us to remotely access our devices through http. The next thing to talk about is the interface between the protocol stack and the application.

Now that the internal mechanism of LwIP is explained, the next step is the structural problem between the application and LwIP, that is, how the application uses the various services provided within the stack. Applications can interact with the stack in two ways: by directly calling the module functions of the stack, or by leveraging the API functions provided by LwIP.

Directly call the functions of each module in the stack, that is, when you want to create a TCP control block, you call the function `tcp_new`, when you make a TCP connection, you call `tcp_connect`, and if you want to receive data, you need to register the relevant callback function in the control block, and so on. The biggest disadvantage of interaction in this way is that it is difficult to write code, and the code is often obscure, and the code debugging will be extremely difficult.

The one that I pay more attention to is the second way: LwIP's API function. User programs can enjoy the services provided by the protocol stack with simple API function calls, which makes it easier to write applications. LwIP's API has a distinct feature compared to the BSD API, that is, data does not need to be copied when interacting with the protocol stack and the application, which can save us a lot of time and memory overhead, and also shows that LwIP is more suitable for embedded systems with limited system resources. But the key is that we have to have a good memory management mechanism, because both the stack and the application operate on the same piece of data.

The implementation of LwIP's API consists of two main parts: one part resides in the user process and the other part resides in the TCP/IP stack process. The two parts communicate with each other through the process communication mechanism (IPC) provided by the operating system simulation layer, which completes the communication between the user process and the protocol stack, including shared memory, messaging, and semaphores. Usually as much work as possible is implemented in the API part of the user process, such as computationally intensive and time-intensive work; The API part of the TCP/IP stack process only does a small amount of work, mainly the communication between processes. The two APIs pass data through shared memory, and the description of shared memory areas is implemented in a similar structure to `pbuf`. In summary, the mechanism of this API implementation can be described in a simple paragraph: the functions in the API library that handle network connections reside in the TCP/IP process. API functions located in the application process use mailbox protocols to deliver messages to API functions that reside in TCP/IP processes. This message includes the type of operation that needs to be performed by the stack and the associated parameters. The API function residing in the TCP/IP process performs this operation and returns the result of the operation to the application via messaging.

The structure `pbuf` was described a long time ago, which is a way of describing packets inside the stack. Here we introduce the data structure `netbuf`, which is a way for APIs to describe data. `Netbuf` is based on `PBUF`, and its structure is shown below, and it is clear that it contains several typical structures.

```
struct netbuf {
    struct pbuf *p, *ptr; struct ip_addr *addr; u16_t port; };

```

Note that the `netbuf` here is just a data header, and it is the `pbuf` linked list that field `P` points to that actually holds the data. The field `ptr` is also a `PBUF` linked list that points to the `netbuf`, but it differs from `p` in that `p` always points to the first `PBUF` structure in the `PBUF` linked list, while `ptr` does not, it may point to other places in the linked list, which is described in the source documentation as a fragment pointer, and the functions closely related to this pointer adjustment are `netbuf_next` and `netbuf_first`. There are also two fields, `addr` and `port`, which represent the IP address and port number of the sending `netbuf` data, respectively, which seem to be of little practical use, and the API defines macros `netbuf_fromaddr` and `netbuf_fromport` to return the values of these two fields in a `netbuf` structure.

There are many handlers related to `netbuf`, and each is described in detail in section 15.2 of the source documentation

Let's talk about the more important ones. `netbuf_new` is used to allocate a new `netbuf` structure, note that this is only a header structure, and the storage data area that is really needed is allocated in function `netbuf_alloc`, the same function `netbuf_delete` to delete a `netbuf` structure, and function `netbuf_free` is called to delete the space of the data area. Simple examples of the above functions are as follows:

<code>struct netbuf *buf;</code>	Clarify the guidelines
<code>buf = netbuf_new();</code>	Request a new netbuf
<code>netbuf_alloc(buf, 200);</code>	Allocate 200 bytes of space for netbuf
<code>.....</code>	Use buf to do related things
<code>netbuf_delete(buf);</code>	Remove buf

After talking about API memory management, let's talk about specific API functions. As in the previous correspondence, the API function consists of two parts, in the files `api_lib.c` and `api_msg.c`. The former includes API interface functions that are directly called by the user program, and the latter includes API functions that communicate with the stack process, and the user program cannot call them directly. These two parts of the API function communicate with each other through mailbox messages. There are two more important data structures to be involved here: the `API` part provides to the parent application to describe a network connection data structure `netconn`, and the `api_msg` that describes the message structure passed between the two API functions.

To establish a connection using an API function, an application should first establish a `netconn` structure that describes the various properties of the connection. The `netconn` structure is as follows, which removes unnecessary compilation options and comments.

```

struct netconn {
    enum netconn_type type;  Types of connections, including TCP, UDP, etc
    enum netconn_state state; The state of the connection
    union {                  A community, the kernel is used to describe a connection
        struct ip_pcb *ip;
        struct tcp_pcb *tcp;
        struct udp_pcb *udp;
        struct raw_pcb *raw;
    } pcb;
    err_t err;               The last time the connection was incorrectly encoded
    sys_sem_t op_completed;  Semaphores for synchronization between two-part APIs
    sys_mbox_t recvmbox;     The mailbox that receives the data
    sys_mbox_t acceptmbox;   The mailbox int socket used by the server to accept external connections; This
                             field is only used in the socket implementation
    u16_t recv_avail;        //struct
    api_msg_msg *write_msg;  If the data cannot be processed normally, the
                             information is saved int write_offset; Same as above, indicating the amount of
                             data that has been processed
    netconn_callback callback; callback function,
                             which can be called when an event related to that netconn occurs};
    write_msg is a pointer of type api_msg_msg, which is discussed later;
    netconn_callback is a function pointer type defined by the API, which is defined as:

```

```

typedef void (* netconn_callback)(struct netconn *, enum netconn_evt, u16_t len);

```

The function of the keyword `typedef` is to define a new type. Here it is used to define a type of `netconn_callback` that is a pointer to a function that has three types of input parameters and returns type `void`. After this definition, you can use `netconn_callback` like `int`, `char`, i.e. it also represents a data type. So the `callback` field above represents a function pointer, and when a function name is assigned to this field, the field records the starting address of the function.

Other fields of netconn will be explained in detail when they are used later. Let's take a look at the api_msg of the message structure passed between the two API functions:

```

    struct api_msg {
        void (* function)(struct api_msg_msg *msg);    function pointer
        struct api_msg_msg msg;    The parameters required for the execution of the function
    }
}
field function is a function pointer, usually when the message is constructed, the
field is populated as some API function in api_msg.c that interfaces with the stack,
and then the message is delivered to the stack process for processing, which parses and
executes the function, so that the communication between the application and the stack
is completed. The field msg contains all the parameters required for the execution of
this function, api_msg_msg is an enum type, so msg has a different structure for
different functions. api_msg_msg structure looks like this: struct api_msg_msg {
    struct netconn *conn; A connection related to a message
    union {
        struct netbuf *b; Function do_send arguments
        struct {
            u8_t
            proto;} n;    Function do_newconn arguments
        struct {
            Functions do_bind and do_connect parameters
            struct ip_addr *ipaddr; u16_t port;
        } bc; struct { // function
            do_getaddr arguments
            struct ip_addr *ipaddr;
            u16_t *port; u8_t local; }
            ad; struct { // argument of
            function do_write
            const void *dataptr; int
            len; u8_t apiflags;} w;
            struct { // 函数 do_recv 的参
            数
            u16_t len;}
            r;} msg;};

```

This structure contains only two fields: conn, which describes the connection information, and msg, which is an enum type. Saving the conn field in the api_msg_msg is necessary because the conn structure contains information such as mailboxes and semaphores related to the connection, which the stack process needs to use to synchronize and communicate with the application process. The individual members of an enum type msg are closely related to the function that calls it. Therefore, when constructing a message, the API should first fill in the function field of the message, and write parameters to the enumeration field of api_msg_msg for a specific function type. This means that for the API function that constructs the message and the function function that parses the message, their understanding of the number of parameters and structure is pre-agreed, and there can be no other variables. point

It reflects the rigid side of LwIP.

29 API message mechanism message mechanism message mechanism message mechanism

Now it is necessary to see what the process of the kernel agreement stack mentioned earlier looks like. The function is called `tcpip_thread`, and its source code is as follows, with irrelevant compilation options and non-focused parts removed.

```
static void
tcpip_thread(void *arg) {

    struct tcpip_msg *msg; sys_timeout(IP_TMR_INTERVAL, ip_reass_timer,
    NULL); Create an IP shard reinstall timeout event sys_timeout
    (ARP_TMR_INTERVAL, arp_timer, NULL); Create an ARP timeout event while
    (1) { // process loop

        sys_mbox_fetch(mbox, (void*)&msg); Blocking messages to be processed on the mailbox
        switch (msg->type) { // Judge the message type

            case TCPIP_MSG_API: For API messages, call the function function inside the message
                msg->msg.apimg->function (&(msg->msg.apimg->msg));
                break; case TCPIP_MSG_INPKT: // 若是接收到 IP 层递交的
                数据包
                    if (msg->msg.inp.netif->flags & NETIF_FLAG_ETHARP) // 支持 ARP
                        ethernet_input(msg->msg.inp.p, msg->msg.inp.netif); // 先进行 ARP 处
                        and then decide whether to submit the IP layer for processing

                // 否则直接递交给 IP 层 ip_input(msg->msg.inp.p, msg-
                >msg.inp.netif); memp_free(MEMP_TCPIP_MSG_INPKT, msg);
                break; ..... default: break; } // switch } // while } //
```

The mailbox `mbox` is a mailbox established when the protocol stack is initialized to receive messages for `tcpip_thread`, and the message types that this function can recognize are `tcpip_msg` structured, so whether it is the API part or the IP packet input part, it must encapsulate its information into a `tcpip_msg` structure. `struct tcpip_msg {`

`enum tcpip_msg_type type;` enumeration structure, message type `sys_sem_t *sem;` semaphore pointer, this field doesn't seem to be used much in union { // community, different message types use different structures

```
    struct api_msg *apimg; API news is a needle
    struct netifapi_msg *netifapimg; No discussion
    struct {
        struct pbuf *p;
        struct netif
        *netif; } inp;
```

```

// 不讨论void (*f)
(void *ctx); void
*ctx;} cb; struct {
不讨论

u32_t msec;
sys_timeout_handler h;
void *arg; } tmo; } msg;
name of the enum type};

```

This structure is very similar to the `api_msg_msg` discussed in the previous section, and `api_msg_msg` also contains a struct called `msg`. The internal members of the enum `tcip_msg_type` are the `TCPIP_MSG_API` and `TCPIP_MSG_INPKT` we see in the function `tcip_thread`, and only these two types are discussed here.

The API function is called `tcip_apimg` that is used to send messages to the kernel process, which populates a `TCPIP_MSG_API` type `tcip_msg` structure and delivers it to the stack-blocking mailbox `mbox`:

```

err_t tcip_apimg(struct api_msg
*apimg) {
    struct tcip_msg msg; Define a message variable
    if (mbox != SYS_MBOX_NULL) {
        msg.type = TCPIP_MSG_API; Message type
        msg.msg.apimg = apimg; Use the msg.apimg field in tcip_msg to record the relevant information
    }
    sys_mbox_post(mbox, &msg); Delivery message
    sys_arch_sem_wait(apimg->msg.conn->op_completed, 0); blocking, waiting for the kernel to finish processing
    the return ERR_OK; }return ERR_VAL; The underlying delivery of received IP packets
    to the kernel process is achieved by calling the function pointed to by the pointer
    input in the network interface structure netif (see the section described in netif
    earlier). Usually this function is tcip_input. err_t tcip_input(struct pbuf *p, struct
    netif *inp){

```

```

struct tcip_msg *msg; Define a message pin if (mbox != SYS_MBOX_NULL) {

```

```

msg = memp_malloc(MEMP_TCIP_MSG_INPKT); Request space for new messages
if (msg == NULL) {
    return ERR_MEM;}msg->type = TCPIP_MSG_INPKT; 消息类型msg->
msg.inp.p = p; 使用 tcip_msg 中的 msg.inp 字段记录相关信息
msg->msg.inp.netif = inp; if (sys_mbox_trypost(mbox, msg) !=
ERR_OK) { // 投递一次消息

```

```

memp_free(MEMP_TCIP_MSG_INPKT, msg); If the delivery is unsuccessful, the message will
be deleted ERR_MEM.

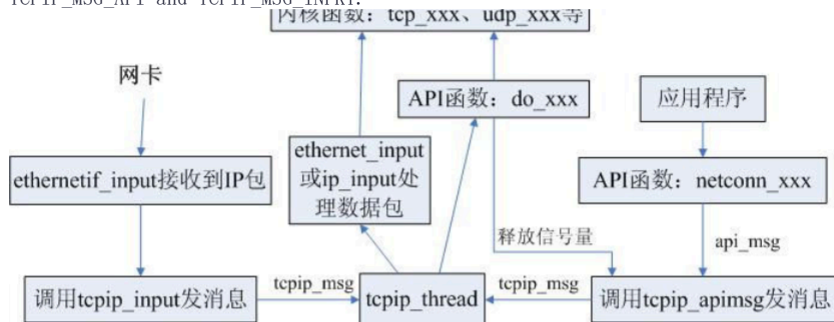
```

```

}return
ERR_OK;}return
ERR_VAL;}

```

Hey, this process is very complicated and entangled, and the following diagram may be easier to understand. Here the function `tcpip_thread` still only deals with two types of messages, `TCPIP_MSG_API` and `TCPIP_MSG_INPKT`.



From the diagram above, you can clearly see the interaction between the two API functions, as well as between the application and the kernel function. The API function `netconn_xxx` is in the file `api_lib.c`, while the other part of the API implementation `do_xxx` in `api_msg.c`.

In the next section, we'll focus on the individual functions in `api_lib.c` without focusing too much on how each `do_xxx` in `api_msg.c` interacts with the kernel functions to accomplish related tasks. `netconn_xxx` functions are also described in subsection 16 of the LwIP documentation, and I intend to repeat the functions of each function and their implementation process.

API Functions and Programming Instances

Functions and Programming Examples

The function `netconn_new` is used to create a new connection structure. The type of connection structure can be TCP or UDP, etc. The prototype of the function structure is shown below, the parameter type describes the type of connection, which can be `NETCONN_TCP` or `NETCONN_UDP`, etc., all of which are discussed in TCP.

`struct netconn* netconn_new(enum netconn_type type)` This function first calls `netconn_alloc` to assign and initialize a `netconn` structure. The initialization process involves setting the `netconn` structure type field, creating a semaphore for the structure's `op_completed` and a receiving mailbox for the `recvmbx` field. It's strange that the `netconn_alloc` function is not in the file `api_lib.c`, but in `api_msg.c`, messy! Next, the function `netconn_new` constructs a `api_msg` message that requires the kernel to execute the function `do_newconn`, and finally the function `tcpip_apimsg` is used to wrap the message in a `tcpip_msg` structure and send it out. `tcpip_thread` function parses the message and calls function `do_newconn`, `do_newconn` finally calls function `tcp_new` to create a TCP control block based on the type of parameter. `tcpip_apimsg` blocks on a semaphore until it is released `do_newconn`.

The function `netconn_delete` is used to remove a connection structure `netconn`. The same process as the previous one, it tells the kernel to perform `do_delconn` via a message, calling the `tcp_close` function to close the TCP connection. The `netconn_delete` then calls the `netconn_free` function to free up space in the `netconn` structure. Note that the `netconn_free` function here is the same as the `netconn_alloc` function, not in the file `api_lib.c`, but in the `api_msg.c`, although they are both `netconn_xxx` structures.

`netconn_bind` is used to bind an IP address and port number to the structure `netconn`. In fact, the kernel is used to call `tcp_bind` through the function `do_bind` to complete the binding of the corresponding TCP control block.

`netconn_connect` functions are typically called on the client side to bind the server-side IP address and port number to the local `netconn` structure. This function corresponds to the kernel `tcp_connect` function.

`netconn_listen` functions are typically called on the server side to put a TCP control block into a LISTEN state. A similar function `do_listen` is called, which has two important jobs: to create a mailbox for the structure `netconn` field `acceptmbx`, which is used to accept external connections; Register a callback function `accept_function` the `accept` field in the PCB control block of the corresponding TCP, and when a new connection appears on the PCB, the callback function will be called to send a message to the `acceptmbx` mailbox above to tell the application that a new connection is coming and that the information of the new connection is saved in the mailbox in the form of a `netconn` structure.

The `netconn_accept` function is used on the server to receive a connection from the far end, which is mainly blocked on the `acceptmbx` mailbox described above, and when a new connection is received, the connected `netconn` structure is removed from that mailbox and returned.

`netconn_recv` function is used to receive data, and the received data is encapsulated in a `netbuf` structure. Here, the kernel function `tcp_recved` will be called by the protocol stack to notify the kernel that the data is received normally, so the kernel adjusts the sending window structure, returns ACK confirmation, etc.

The function `netconn_write` is used to send data to the corresponding TCP connection, and the function used to send UDP data is called `netconn_send`, which will not be discussed here. The `netconn_write` function prototype is as follows, which is used to put the size data pointed by `Dataptr` to the send queue connected to the `conn`, and the `apiflags` are used to describe it

```
err_t netconn_write (struct netconn *conn, const void *dataptr, int size, u8_t apiflags)
to operate on this data, including whether to copy or not, and whether to send it
immediately. Finally, the netconn_close function is used to actively close the connection.
```

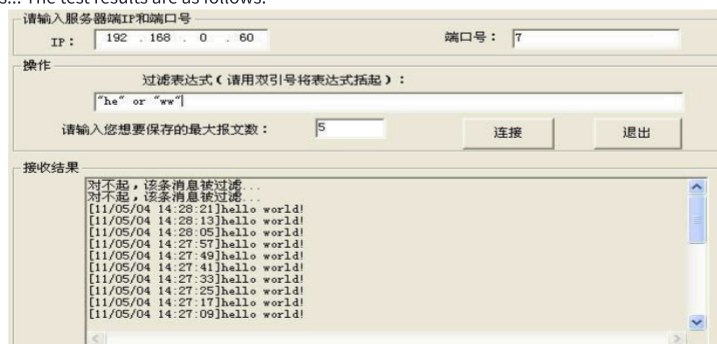
That's just a little bit of the API function. Let's use these API functions to construct a server program. This server program is simple, it responds to a client's connection and data request, and sends a fixed string to the client. The task code is as follows: const uint8 data[] = "hello world!"; string to be sent void mytcp_thread(void *arg) {

```
struct netconn * conn, *newconn; // API 描述的连接结构 struct netbuf * buf; // API 数据缓冲
```

```
conn = netconn_new(NETCONN_TCP); Create a new TCP connection structure netconn_bind  
(conn, NULL, 7); The connection is bound to port 7 netconn_listen (conn); Put the structure  
to listening state newconn = netconn_accept(conn); A new connection is received
```

```
while(  
1) {  
    buf = netconn_recv(newconn); A piece of data is received on the new connection  
    netbuf_delete(buf); Delete the data received  
    netconn_write(newconn,data,sizeof(data), NETCONN_COPY); The client that sends  
    the string}}
```

The server program is designed this way for testing convenience, because there happens to be a small program that can be used to test whether the server program and our LwIP protocol stack are working properly. This mini program was written when I participated in the ZTE Programming Competition, and the name was Message Monitor. It can receive data from a TCP connection and filter it according to the user's requirements, removing data that the user does not care about. Big mouth Dong Ge and Peng Peng in the dormitory. O(∩_∩)O~ When I saw this program, I thought of you, great heroes... The test results are as follows:



The content in the filter expression edit box is a filter condition entered by the user, and when the received data string meets the filter conditions, the string will not be displayed in the received results. Filter conditions are a series of strings enclosed in quotation marks, which can be connected by or, and, not, parentheses, etc. to form a very complex filter condition. I won't talk about it.

First, set the filter to empty and show all the data received from the server "hello word!", as shown below in the image. Then set the filter to "he" or "ww": That is, the string containing the word "he" or "ww" in the string will be filtered out and not displayed. This is as shown in the first two lines in the receiving results. OK... The test results are normal,

Our LwIP is running steadily! However, there are other testing methods here, the more common method is to build an http server and then use our browser to connect to the server, which has many routines and detailed instructions in the LwIP porting manual, so let's not go into details.

It can be seen that using the LwIP API can easily complete all TCP communication related tasks. In addition, LwIP implements the BSD Socket API function with its own API function. Because many software is written based on BSD sockets, which are simpler and easier to understand and widely used, it is still necessary to implement the Socket API. But the LwIP documentation reads:

This section provides a simple implementation of the BSD Socket API using the LwIP API. This implementation can only be used as a reference and cannot be used in actual programming, because it is not perfect, such as it does not have a fault tolerance mechanism. Also, this simple implementation does not support the select() and poll() functions in the BSD Socket API, as the LwIP API does not have any functions that can be used for their implementations. To implement both functions, the BSD socket implementation needs to communicate directly with the LwIP stack instead of using its API.

So I won't discuss the BSD Socket API in detail here, but using the LwIP API can do the related work, and the programming work is very simple.

At this point, we have gone through the LwIP protocol from beginning to end, from the network interface layer to the ARP layer, then to the IP layer, then to the TCP layer, and finally to the API layer. In practice, TCP packets are usually processed in this order. LwIP has a lot of other things that haven't been discussed yet, first UDP, followed by PPP, SILP, DNS, IGMP, DHCP, SNMP, IPV6, and so on. These are only used on some special occasions and have no commonality, so I won't touch on them here.

The end of the whole play...